geometry-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `geometry_ai999x.about()` for the same answer.

Every capability geometry-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import geometry_ai999x

result = geometry_ai999x.solve('rectangle_measures', length=8, width=5)
result.show()
# area = 40, perimeter = 26, diagonal = 9.43398
```

By topic

topic	capabilities
Computational geometry / convex hulls	Convex hull of points in the plane
Computational geometry / enclosing shapes	Smallest enclosing circle of points, Minimum-area enclosing rectangle
Computational geometry / intersections	Intersection of two line segments
Computational geometry / point location	Point in polygon test
Computational geometry / polygon clipping	Area of the intersection of two convex polygons
Computational geometry / proximity	Closest pair of points, Farthest pair of points (diameter of a point set), Distance from a point to a segment, Nearest neighbour of a query point
Computational geometry / shape comparison	Hausdorff distance between two point sets
Computational geometry / triangulations	Delaunay triangulation and Voronoi diagram: counts
Conic sections / ellipses	Perimeter of an ellipse (exact, by the complete elliptic integral)
Conic sections / general second-degree equation	Classify a conic section from its general equation
Coordinate geometry / circles	Intersection of a circle and a line, Intersection of two circles, Tangents from a point to a circle
Coordinate geometry / lines	Intersection point of two lines, Angle between two lines
Coordinate geometry / paths	Length of a polyline (path through points)

topic	capabilities
Coordinate geometry / triangle centres	Circumcircle: the circle through three points, Incircle of a triangle: incentre and inradius, Orthocentre of a triangle
Coordinate geometry / triangles	Triangle from vertex coordinates: area, perimeter, centroid, angles, Barycentric coordinates of a point in a triangle
Curves / arc length	Arc length of a parabola
Curves / curvature	Curvature of a polynomial graph at a point, Curvature of an ellipse at a point
Curves / space curves	Helix: curvature, torsion and length per turn
Plane figures / circles	Circle: area, circumference and diameter, Circular sector: arc length, area and chord, Area of an annulus (ring), Circular segment: area, chord, height, arc
Plane figures / quadrilaterals	Rectangle: area, perimeter and diagonal, Trapezoid: area and midsegment, Rhombus from its diagonals: area, side, perimeter
Plane figures / triangles	Area of a triangle from base and height, Classify a triangle by its sides and angles, Medians, altitudes and angle bisectors of a triangle
Polygons / convexity	Is a polygon convex?
Polygons / coordinates	Polygon from vertices: area, perimeter, orientation, Centroid of a polygon, Interior angles of a polygon from its vertices
Polygons / lattice polygons	Pick's theorem: lattice points of a polygon
Polygons / regular polygons	Polygon angles and diagonals, Regular polygon: area, perimeter, apothem, circumradius
Polyhedra / regular polyhedra	Platonic solids: counts, Euler characteristic, volume, surface, radii
Solid coordinate geometry / lines	Distance between two lines in space (skew or parallel)
Solid coordinate geometry / lines and planes	Intersection of a line and a plane
Solid coordinate geometry / planes	Plane through three points, Distance from a point to a plane, Angle between two planes
Solid coordinate geometry / polyhedra	Tetrahedron from four vertices: volume and surface area
Solid coordinate geometry / spheres	Sphere through four points (circumsphere of a tetrahedron)
Solid coordinate geometry / triangles	Triangle in space: area and perimeter from vertices
Solid geometry / solid angles	Solid angle of a cone, Solid angle subtended by a triangle
Solids / cones	Cone: volume, slant height and surface area, Frustum of a cone: volume and surface area
Solids / cylinders	Cylinder: volume and surface area
Solids / ellipsoids	Spheroid (ellipsoid of revolution): surface area and volume
Solids / prisms	Cuboid (rectangular box): volume, surface area, space diagonal, Right prism on a regular polygon base: volume and surface area
Solids / pyramids	Right rectangular (or square) pyramid: volume and surface area
Solids / solids of revolution	Torus: volume and surface area
Solids / spheres	Sphere: volume and surface area, Spherical cap: volume, curved area, base radius
Spherical geometry / distances	Great-circle distance between two points on a sphere
Spherical geometry / navigation	Initial bearing (forward azimuth) along a great circle
Spherical geometry / spherical triangles	Area and spherical excess of a spherical triangle

topic	capabilities
Surfaces / curvature	Gaussian and mean curvature of a quadratic surface z = f(x, y), Curvature of a torus at a point
Transformations / dilations	Dilation (enlargement) of a point
Transformations / reflections	Reflection of a point in a line
Transformations / rotations	Rotation of a point
Transformations / rotations in space	Rotation of a point in space about an axis

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.

- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
rectangle_measures	Rectangle: area, perimeter and diagonal	length, width	area, perimeter, diagonal
triangle_area_base_height	Area of a triangle from base and height	base, height	area
circle_measures	Circle: area, circumference and diameter	radius	area, circumference, diameter
trapezoid_measures	Trapezoid: area and midsegment	a, b, height	area, midsegment
rhombus_measures	Rhombus from its diagonals: area, side, perimeter	d1, d2	area, side, perimeter
sector_measures	Circular sector: arc length, area and chord	radius, angle	arc_length, sector_area, chord
regular_polygon_angles	Polygon angles and diagonals	n	interior_angle, exterior_angle, interior_sum, diagonals
regular_polygon_measures	Regular polygon: area, perimeter, apothem, circumradius	n, side	area, perimeter, apothem, circumradius
cuboid_measures	Cuboid (rectangular box): volume, surface area, space diagonal	length, width, height	volume, surface_area, space_diagonal
cylinder_measures	Cylinder: volume and surface area	radius, height	volume, lateral_area, total_area
cone_measures	Cone: volume, slant height and surface area	radius, height	volume, slant_height, lateral_area, total_area
sphere_measures	Sphere: volume and surface area	radius	volume, surface_area
triangle_classify	Classify a triangle by its sides and angles	a, b, c	by_sides, by_angles
polygon_area	Polygon from vertices: area, perimeter, orientation	points	area, perimeter, orientation
polygon_centroid	Centroid of a polygon	points	cx, cy
triangle_coordinates	Triangle from vertex coordinates: area, perimeter, centroid, angles	points	area, perimeter, gx, gy, angle_A, angle_B, angle_C
triangle_circumcircle	Circumcircle: the circle through three points	points	cx, cy, radius
triangle_incircle	Incircle of a triangle: incentre and inradius	points	ix, iy, radius

id	operation	inputs	outputs
triangle_orthocenter	Orthocentre of a triangle	points	hx, hy
line_intersection	Intersection point of two lines	a1, b1, c1, a2, b2, c2	x, y
angle_between_lines	Angle between two lines	a1, b1, c1, a2, b2, c2	angle
reflect_point_line	Reflection of a point in a line	px, py, a, b, c	x, y
rotate_point	Rotation of a point	px, py, angle, cx, cy	x, y
dilate_point	Dilation (enlargement) of a point	px, py, k, cx, cy	x, y
circle_line_intersection	Intersection of a circle and a line	cx, cy, r, a, b, c	count, x1, y1, x2, y2
circle_circle_intersection	Intersection of two circles	x1, y1, r1, x2, y2, r2	count, x1, y1, x2, y2
tangent_from_point	Tangents from a point to a circle	cx, cy, r, px, py	length, x1, y1, x2, y2
conic_classify	Classify a conic section from its general equation	A, B, C, D, E, F	type
point_in_polygon	Point in polygon test	px, py, points	location
convex_hull	Convex hull of points in the plane	points	hull, n_vertices, area, perimeter
closest_pair	Closest pair of points	points	distance
farthest_pair	Farthest pair of points (diameter of a point set)	points	distance
segment_intersection	Intersection of two line segments	points	status, x, y
point_segment_distance	Distance from a point to a segment	px, py, x1, y1, x2, y2	distance, qx, qy
polyline_length	Length of a polyline (path through points)	points	length, segments
polygon_is_convex	Is a polygon convex?	points	convex
pick_lattice_points	Pick's theorem: lattice points of a polygon	points	area, boundary_points, interior_points
triangle_cevians	Medians, altitudes and angle bisectors of a triangle	a, b, c	median_a, median_b, median_c, altitude_a, altitude_b, altitude_c, bisector_a, bisector_b, bisector_c
annulus_area	Area of an annulus (ring)	outer, inner	area
circular_segment	Circular segment: area, chord, height, arc	radius, angle	area, chord, height, arc_length
frustum_measures	Frustum of a cone: volume and surface area	r1, r2, height	volume, slant_height, lateral_area, total_area
pyramid_measures	Right rectangular (or square) pyramid: volume and surface area	length, width, height	volume, lateral_area, total_area
regular_prism	Right prism on a regular polygon base: volume and	n, side, height	volume, lateral_area, total_area, base_area

id	operation	inputs	outputs
	surface area		
platonic_solid	Platonic solids: counts, Euler characteristic, volume, surface, radii	solid, edge	vertices, edges, faces, euler_characteristic, volume, surface_area, circumradius, inradius
spherical_cap	Spherical cap: volume, curved area, base radius	radius, height	volume, cap_area, base_radius
torus_measures	Torus: volume and surface area	major, minor	volume, surface_area
spheroid_measures	Spheroid (ellipsoid of revolution): surface area and volume	a, c	surface_area, volume, shape
ellipse_perimeter	Perimeter of an ellipse (exact, by the complete elliptic integral)	a, b	perimeter, area
triangle_area_3d	Triangle in space: area and perimeter from vertices	points	area, perimeter
tetrahedron_measures	Tetrahedron from four vertices: volume and surface area	points	volume, surface_area
plane_through_points	Plane through three points	points	a, b, c, d
point_plane_distance	Distance from a point to a plane	px, py, pz, a, b, c, d	distance, fx, fy, fz
line_plane_intersection	Intersection of a line and a plane	x0, y0, z0, dx, dy, dz, a, b, c, d	x, y, z, t
skew_lines_distance	Distance between two lines in space (skew or parallel)	p1, d1, p2, d2	distance, parallel
angle_between_planes	Angle between two planes	a1, b1, c1, a2, b2, c2	angle
rotate_point_3d	Rotation of a point in space about an axis	point, axis, angle	x, y, z
sphere_four_points	Sphere through four points (circumsphere of a tetrahedron)	points	cx, cy, cz, radius
great_circle_distance	Great-circle distance between two points on a sphere	lat1, lon1, lat2, lon2, radius	central_angle, distance
initial_bearing	Initial bearing (forward azimuth) along a great circle	lat1, lon1, lat2, lon2	bearing
spherical_triangle_area	Area and spherical excess of a spherical triangle	points, radius	excess, area
solid_angle_cone	Solid angle of a cone	half_angle	solid_angle, fraction_of_sphere
solid_angle_triangle	Solid angle subtended by a triangle	points	solid_angle
curvature_of_graph	Curvature of a polynomial graph at a point	coefficients, x0	curvature, radius_of_curvature, slope
helix_measures	Helix: curvature, torsion and length per turn	radius, pitch	curvature, torsion, length_per_turn

id	operation	inputs	outputs
ellipse_curvature	Curvature of an ellipse at a point	a, b, t	curvature, radius_of_curvature, x, y
surface_curvature	Gaussian and mean curvature of a quadratic surface z = f(x, y)	A, B, C, D, E, x0, y0, F	gaussian, mean
torus_curvature	Curvature of a torus at a point	major, minor, v	gaussian, mean, k1, k2
parabola_arc_length	Arc length of a parabola	a, b, x0, x1, c	length
min_enclosing_circle	Smallest enclosing circle of points	points	cx, cy, radius
min_area_rectangle	Minimum-area enclosing rectangle	points	area, width, length
delaunay_voronoi_counts	Delaunay triangulation and Voronoi diagram: counts	points	triangles, edges, hull_vertices, voronoi_vertices, unbounded_cells
convex_polygon_intersection	Area of the intersection of two convex polygons	p, q	area
hausdorff_distance	Hausdorff distance between two point sets	a, b	distance, from_a, from_b
nearest_neighbor	Nearest neighbour of a query point	px, py, points	index, distance, nx, ny
barycentric_coordinates	Barycentric coordinates of a point in a triangle	px, py, points	l1, l2, l3
polygon_interior_angles	Interior angles of a polygon from its vertices	points	angles, angle_sum, reflex_vertices

Rectangle: area, perimeter and diagonal, `rectangle_measures`

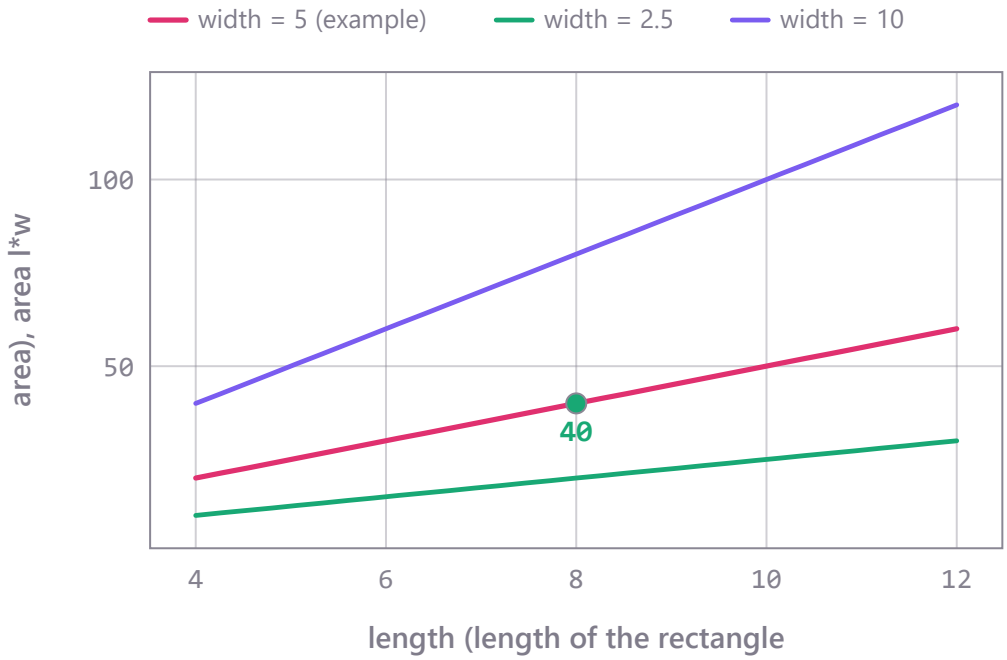
Area, perimeter and diagonal of a rectangle (a square when length = width).

$$A = lw, \quad P = 2(l + w), \quad d = \sqrt{l^2 + w^2}$$

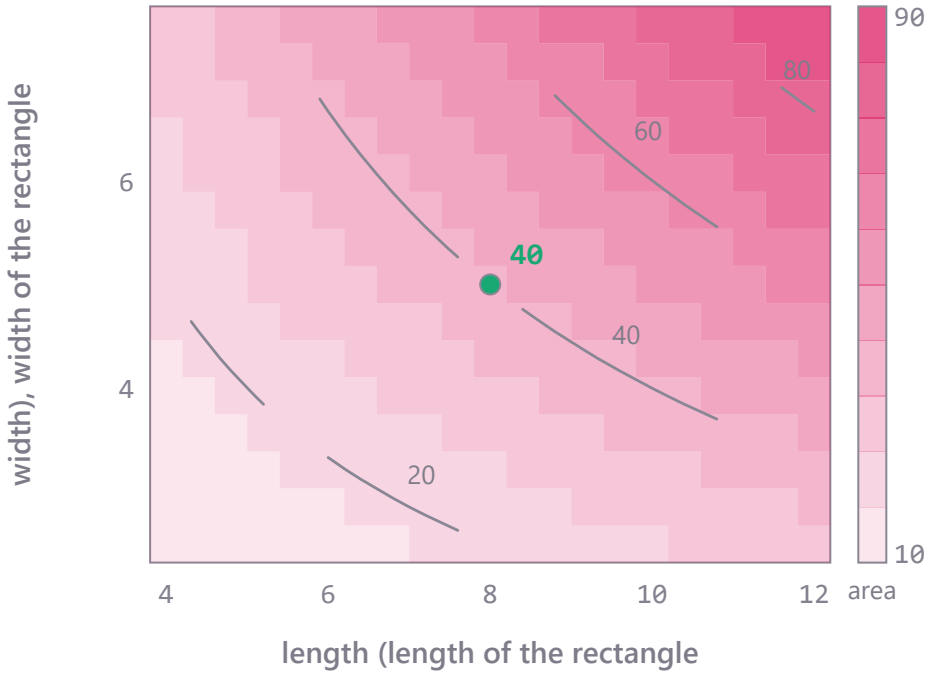
input	meaning	condition
`length`	length of the rectangle	
`width`	width of the rectangle	

output	meaning
`area`	area lw
`perimeter`	perimeter 2(l + w)
`diagonal`	diagonal sqrt(l^2 + w^2)

- **Method:** $A = l$ w, $P = 2$(l + w), $d = sqrt$(l^2 + w^2) (Pythagoras)
- **Assumes:** length > 0; width > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



area (area l*w) against length (length of the rectangle) from 4 to 12. One curve for each value of width (width of the rectangle): 5, 2.5, 10. 31 points per curve computed by geometry-ai999x; marked: the worked example, length = 8, area = 40.



area (area l*w) over length (length of the rectangle) from 4 to 12 and width (width of the rectangle) from 2.5 to 7.5: each of the 21 × 15 cells is one run of geometry-ai999x, shaded by its value (light 10 to dark 90); iso-lines at area = 20, 40, 60, 80. Marked: the worked example, length = 8, width = 5, area = 40.

```
geometry_ai999x.solve('rectangle_measures', length=8, width=5)
# -> area = 40, perimeter = 26, diagonal = 9.43398   (SOLVED)
```

```
geometry_ai999x.solve('rectangle_measures', length=6, width=8)
# -> area = 48, perimeter = 28, diagonal = 10   (SOLVED)
```

```
geometry_ai999x.solve('rectangle_measures', length=7, width=7)
# -> area = 49, perimeter = 28, diagonal = 9.89949   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the area and perimeter of a rectangle with length 8 and width 5.	SOLVED	area = 40, perimeter = 26, diagonal = 9.43398
What is the diagonal of a 6 by 8 rectangle?	SOLVED	area = 48, perimeter = 28, diagonal = 10
Find the perimeter of a square with side 2.5 cm.	SOLVED	area = 6.25, perimeter = 10, diagonal = 3.53553
A rectangle is 12 m long and 5 m wide. What is its diagonal?	SOLVED	area = 60, perimeter = 34, diagonal = 13
Find the area of a rectangle with length 3 m and width 40 cm.	OUT_OF_SCOPE	Out of scope: lengths are given in different units; state every length in one unit.
Find the area of a rectangle with length -3 and width 4.	INVALID_INPUT	Rectangle: area, perimeter and diagonal: length must be positive.
Find the area of a rectangle with length 3.	NEEDS_INPUT	rectangle_measures needs width (width of the rectangle). Example: solve('rectangle_measures', length=8, width=5).

Area of a triangle from base and height, `triangle_area_base_height`

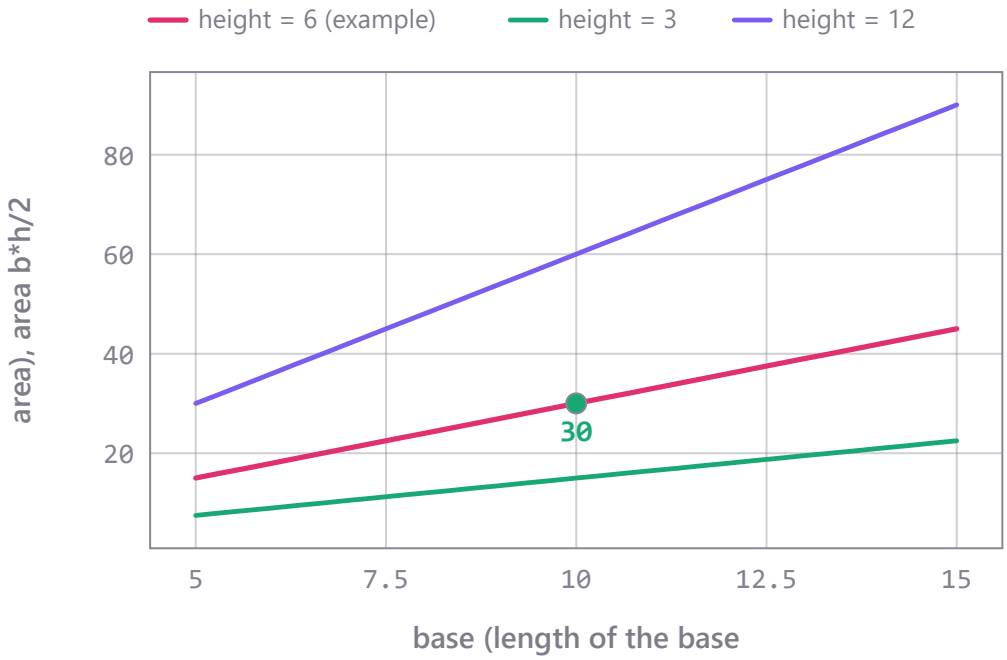
The area of a triangle from a base and the height (altitude) to that base.

$$A = \tfrac{1}{2} b h$$

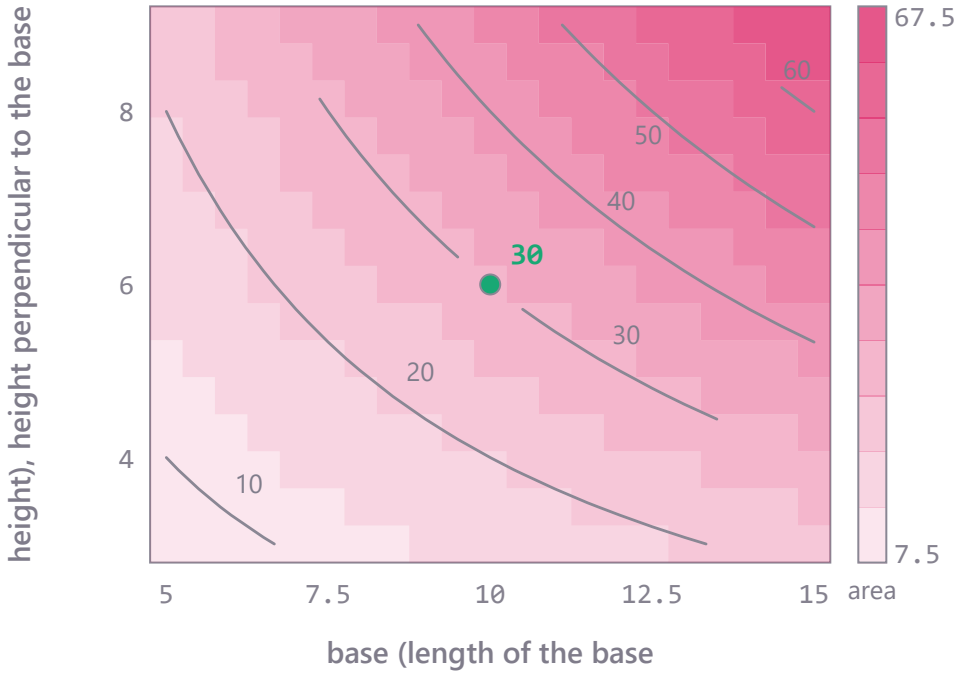
input	meaning	condition
`base`	length of the base	
`height`	height perpendicular to the base	

output	meaning
`area`	area $bh/2$

- **Method:** $A = b\,h/2$
- **Assumes:** base > 0; height > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



area (area b*h/2) against base (length of the base) from 5 to 15. One curve for each value of height (height perpendicular to the base): 6, 3, 12. 31 points per curve computed by geometry-ai999x; marked: the worked example, base = 10, area = 30.



area (area b*h/2) over base (length of the base) from 5 to 15 and height (height perpendicular to the base) from 3 to 9: each of the 21 × 15 cells is one run of geometry-ai999x, shaded by its value (light 7.5 to dark 67.5); iso-lines at area = 10, 20, 30, 40, 50, 60. Marked: the worked example, base = 10, height = 6, area = 30.

```
geometry_ai999x.solve('triangle_area_base_height', base=10, height=6)
# -> area = 30   (SOLVED)
```

```
geometry_ai999x.solve('triangle_area_base_height', base=7, height=4)
# -> area = 14   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the area of a triangle with base 10 and height 6.	SOLVED	area = 30
A triangle has a base of 7 cm and an altitude of 4 cm. What is its area?	SOLVED	area = 14
Find the area of a triangle with base 0 and height 6.	INVALID_INPUT	Area of a triangle from base and height: base must be positive.
Find the area of a triangle with base 10 and height not given.	NEEDS_INPUT	triangle_area_base_height needs height (height perpendicular to the base). Example: solve('triangle_area_base_height', b

Circle: area, circumference and diameter, `circle_measures`

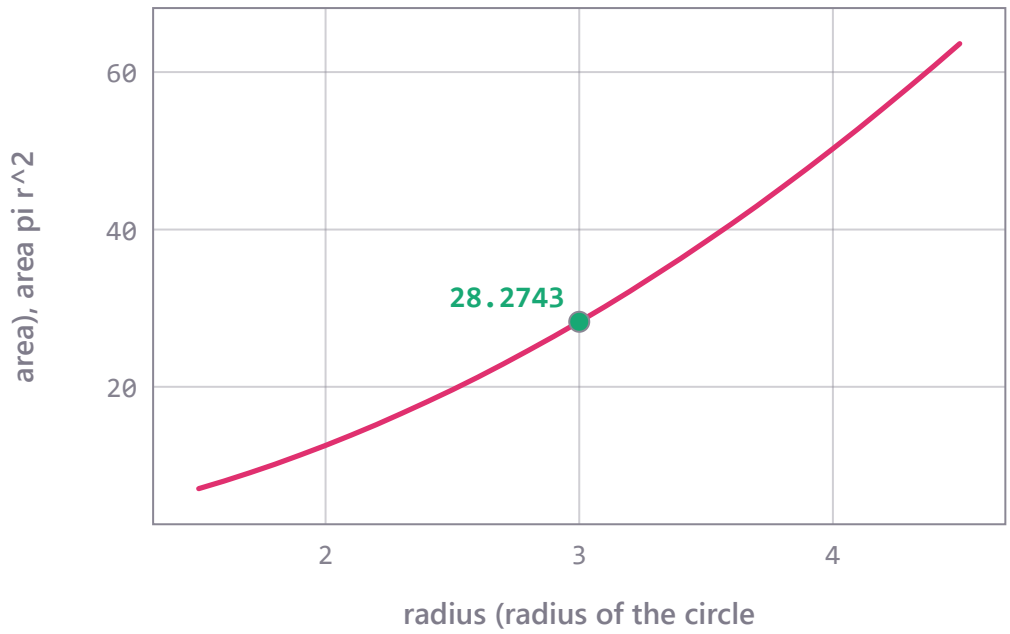
Area, circumference and diameter of a circle from its radius (or diameter).

$$A = \pi r^2, \quad C = 2\pi r$$

input	meaning	condition
`radius`	radius of the circle	

output	meaning
`area`	area pi r^2
`circumference`	circumference 2 pi r
`diameter`	diameter 2r

- **Method:** $A = pi\ r^2$, $C = 2$ pi r
- **Assumes:** radius > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



area (area pi r^2) against radius (radius of the circle) from 1.5 to 4.5. 31 points computed by geometry-ai999x; marked: the worked example, radius = 3, area = 28.2743.

```
geometry_ai999x.solve('circle_measures', radius=3)
# -> area = 28.2743, circumference = 18.8496, diameter = 6   (SOLVED)
```

```
geometry_ai999x.solve('circle_measures', radius=5)
# -> area = 78.5398, circumference = 31.4159, diameter = 10   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the area of a circle with radius 3.	SOLVED	area = 28.2743, circumference = 18.8496, diameter = 6
What is the circumference of a circle with diameter 10?	SOLVED	area = 78.5398, circumference = 31.4159, diameter = 10
A circular garden has a radius of 1 m. Find its area.	SOLVED	area = 3.14159, circumference = 6.28319, diameter = 2
Find the area of a circle with radius -2.	INVALID_INPUT	Circle: area, circumference and diameter: radius must be positive.
Find the area of a circle.	NEEDS_INPUT	circle_measures needs radius (radius of the circle). Example: solve('circle_measures', radius=3).

Trapezoid: area and midsegment, `trapezoid_measures`

Area and midsegment (median) of a trapezoid from its parallel sides and height.

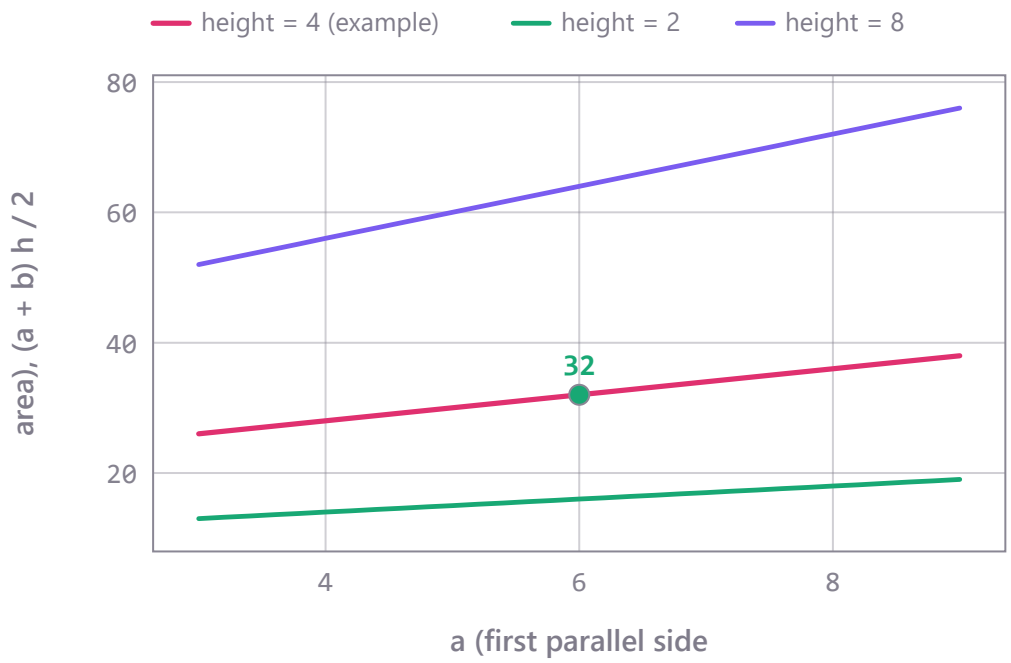
$$A = \frac{(a+b)\,h}{2}$$

input	meaning	condition
`a`	first parallel side	
`b`	second parallel side	

input	meaning	condition
`height`	distance between the parallel sides	

output	meaning
`area`	(a + b) $h/2$
`midsegment`	$(a+b)/2$

- **Method:** $A = (a+b)\,h/2$
- **Assumes:** a, b, height > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



area ((a + b) h / 2) against a (first parallel side) from 3 to 9. One curve for each value of height (distance between the parallel sides): 4, 2, 8. Other inputs as in the worked example: b = 10. 31 points per curve computed by geometry-ai999x; marked: the worked example, a = 6, area = 32.

```
geometry_ai999x.solve('trapezoid_measures', a=6, b=10, height=4)
# -> area = 32, midsegment = 8   (SOLVED)
```

```
geometry_ai999x.solve('trapezoid_measures', a=8, b=12, height=5)
# -> area = 50, midsegment = 10   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the area of a trapezoid with parallel sides 6 and 10 and height 4.	SOLVED	area = 32, midsegment = 8
A trapezium has bases of 8 cm and 12 cm and a height of 5 cm. What is its area?	SOLVED	area = 50, midsegment = 10
Find the area of a trapezoid with parallel sides 6 and 10.	NEEDS_INPUT	trapezoid_measures needs height (distance between the parallel sides). Example: solve('trapezoid_measures', a=6, b=10, h

Rhombus from its diagonals: area, side, perimeter, `rhombus_measures`

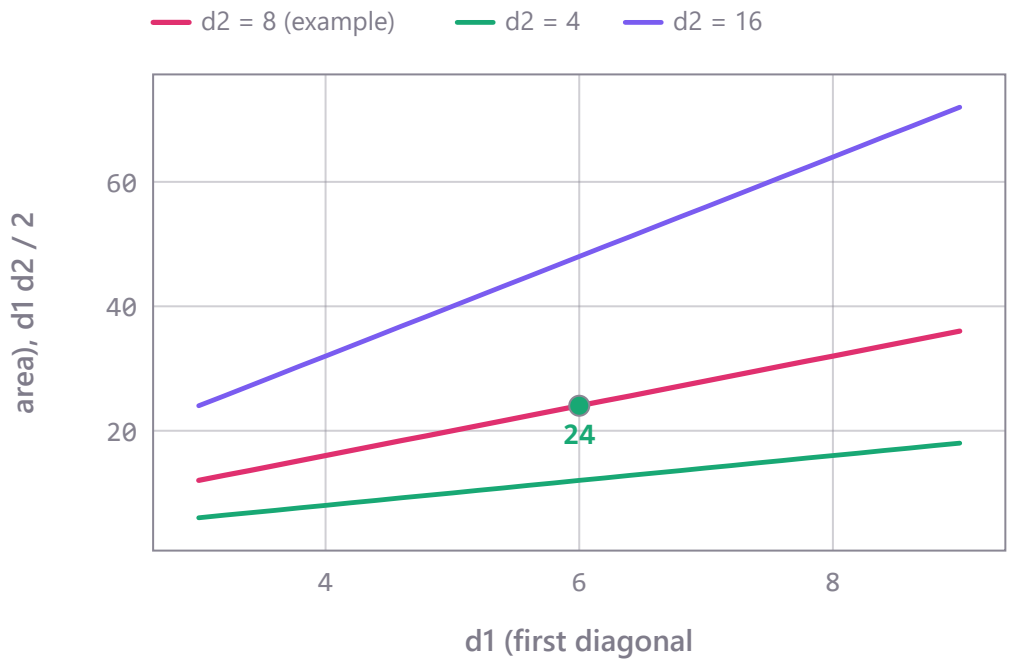
Area, side and perimeter of a rhombus from its two diagonals.

$$A = \tfrac{1}{2} d_1 d_2, \quad s = \tfrac{1}{2}\sqrt{d_1^2 + d_2^2}$$

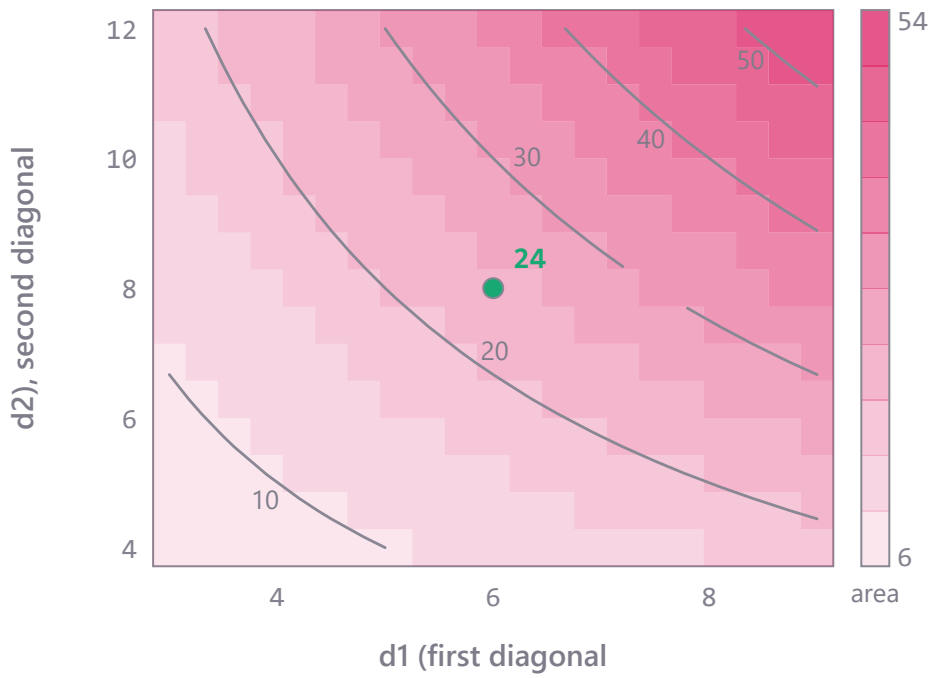
input	meaning	condition
`d1`	first diagonal	
`d2`	second diagonal	

output	meaning
`area`	d1 $d_2/2$
`side`	sqrt(d1^2 + d2^2)/2
`perimeter`	4 * side

- **Method:** $A = d_1\ d_2/2$; the half-diagonals are the legs of a right triangle whose hypotenuse is a side
- **Assumes:** d1, $d_2 > 0$ (the diagonals of a rhombus bisect each other at right angles)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



area (d1 d2 / 2) against d1 (first diagonal) from 3 to 9. One curve for each value of d2 (second diagonal): 8, 4, 16. 31 points per curve computed by geometry-ai999x; marked: the worked example, d1 = 6, area = 24.



area (d1 d2 / 2) over d1 (first diagonal) from 3 to 9 and d2 (second diagonal) from 4 to 12: each of the 21 × 15 cells is one run of geometry-ai999x, shaded by its value (light 6 to dark 54); iso-lines at area = 10, 20, 30, 40, 50. Marked: the worked example, d1 = 6, d2 = 8, area = 24.

```
geometry_ai999x.solve('rhombus_measures', d1=6, d2=8)
# -> area = 24, side = 5, perimeter = 20   (SOLVED)
```

```
geometry_ai999x.solve('rhombus_measures', d1=10, d2=24)
# -> area = 120, side = 13, perimeter = 52   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the area of a rhombus with diagonals 6 and 8.	SOLVED	area = 24, side = 5, perimeter = 20
A rhombus has diagonals of 10 cm and 24 cm. Find its perimeter.	SOLVED	area = 120, side = 13, perimeter = 52
Find the area of a rhombus.	NEEDS_INPUT	rhombus_measures needs d1, d2 (first diagonal; second diagonal). Example: solve('rhombus_measures', d1=6, d2=8).

Circular sector: arc length, area and chord, `sector_measures`

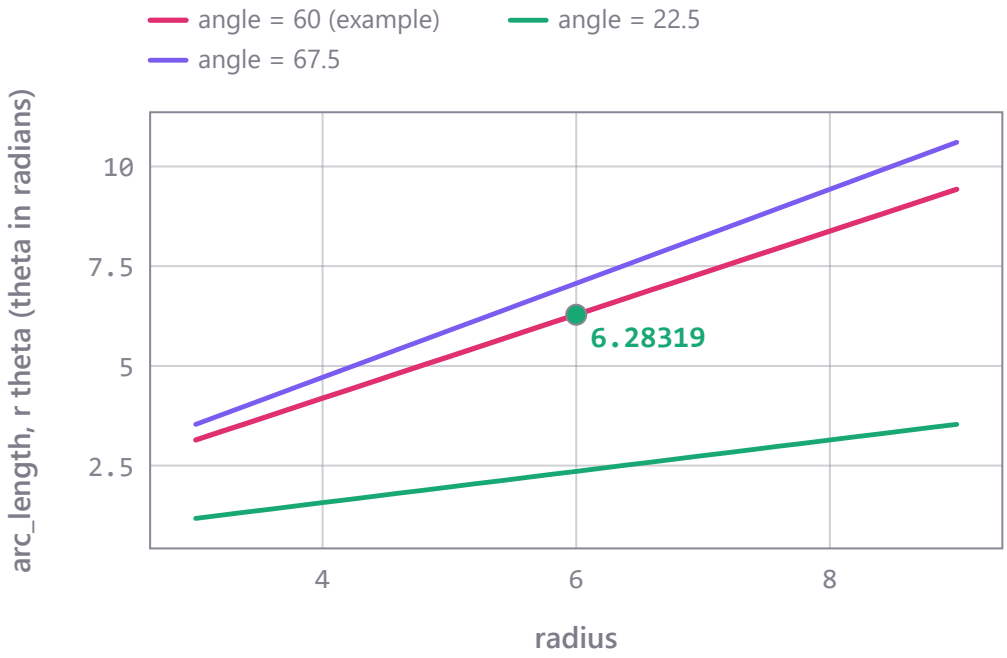
Arc length, sector area and chord of a circular sector from the radius and central angle.

$$s = r\theta, \quad A = \tfrac{1}{2}r^2\theta, \quad c = 2r\sin\tfrac{\theta}{2}$$

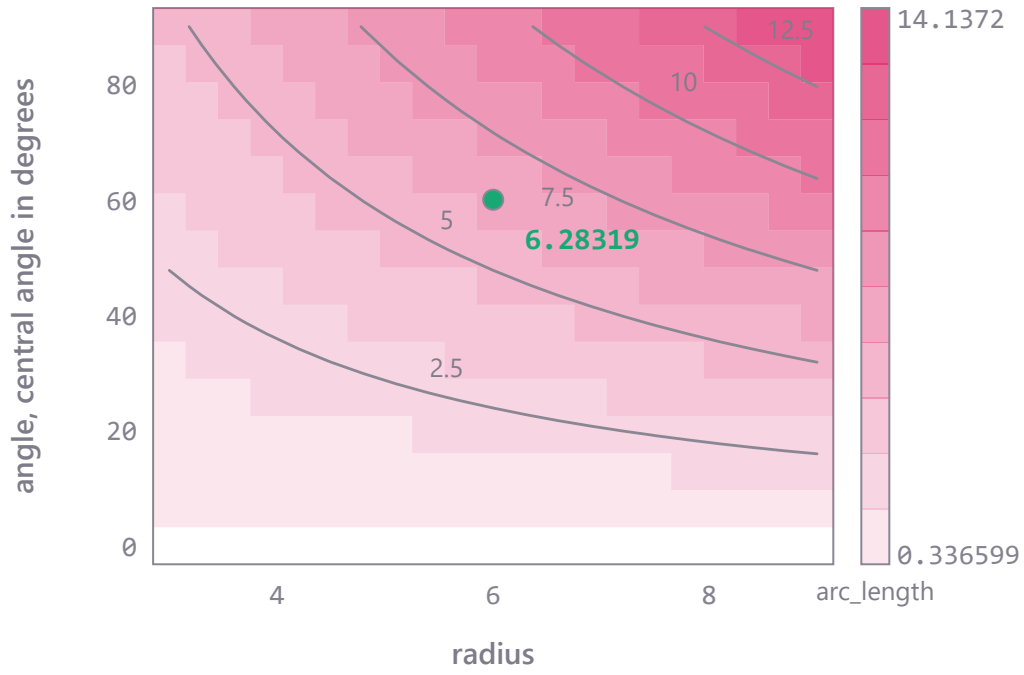
input	meaning	condition
`radius`	radius	
`angle`	central angle in degrees	

output	meaning
`arc_length`	r theta (theta in radians)
`sector_area`	$r^2\ \theta/2$
`chord`	2 r sin(theta/2)

- **Method:** theta = angle * pi/180; $s = r$ theta; $A = r^2\ \theta/2$; chord = 2 r sin(theta/2)
- **Assumes:** radius > 0; 0 < angle <= 360 degrees
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



arc_length (r theta (theta in radians)) against radius (radius) from 3 to 9. One curve for each value of angle (central angle in degrees): 60, 22.5, 67.5. 31 points per curve computed by geometry-ai999x; marked: the worked example, radius = 6, arc_length = 6.28319.



arc_length (r theta (theta in radians)) over radius (radius) from 3 to 9 and angle (central angle in degrees) from 0 to 90: each of the 21 × 15 cells is one run of geometry-ai999x, shaded by its value (light 0.336599 to dark 14.1372); iso-lines at arc_length = 2.5, 5, 7.5, 10, 12.5; blank cells: inputs the capability refuses. Marked: the worked example, radius = 6, angle = 60, arc_length = 6.28319.

```
geometry_ai999x.solve('sector_measures', radius=6, angle=60)
# -> arc_length = 6.28319, sector_area = 18.8496, chord = 6   (SOLVED)
```

```
geometry_ai999x.solve('sector_measures', radius=10, angle=90)
# -> arc_length = 15.708, sector_area = 78.5398, chord = 14.1421   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the arc length and area of a sector with radius 6 and central angle 60 degrees.	SOLVED	arc_length = 6.28319, sector_area = 18.8496, chord = 6
A sector of a circle has radius 10 cm and angle 90°. What is its area?	SOLVED	arc_length = 15.708, sector_area = 78.5398, chord = 14.1421
Find the arc length of a sector with radius 2 and angle 3.14159265358979 rad.	SOLVED	arc_length = 6.28319, sector_area = 6.28319, chord = 4
Find the area of a sector with radius 4 and central angle 400 degrees.	INVALID_INPUT	Circular sector: arc length, area and chord: the central angle must be between 0 and 360 degrees.
Find the area of a sector with radius 4.	NEEDS_INPUT	sector_measures needs angle (central angle in degrees). Example: solve('sector_measures', radius=6, angle=60).

Polygon angles and diagonals, `regular_polygon_angles`

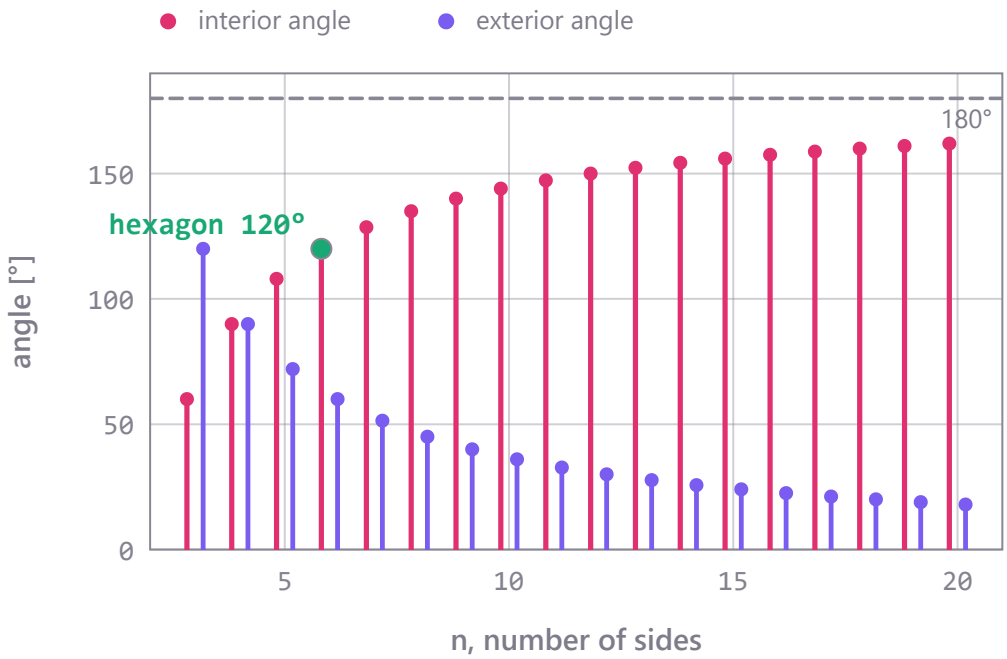
Each interior and exterior angle of a regular n-gon, the interior angle sum of any n-gon and its number of diagonals.

$$\theta_{int} = \frac{(n-2)\,180^\circ}{n}, \quad \theta_{ext} = \frac{360^\circ}{n}, \quad D = \frac{n(n-3)}{2}$$

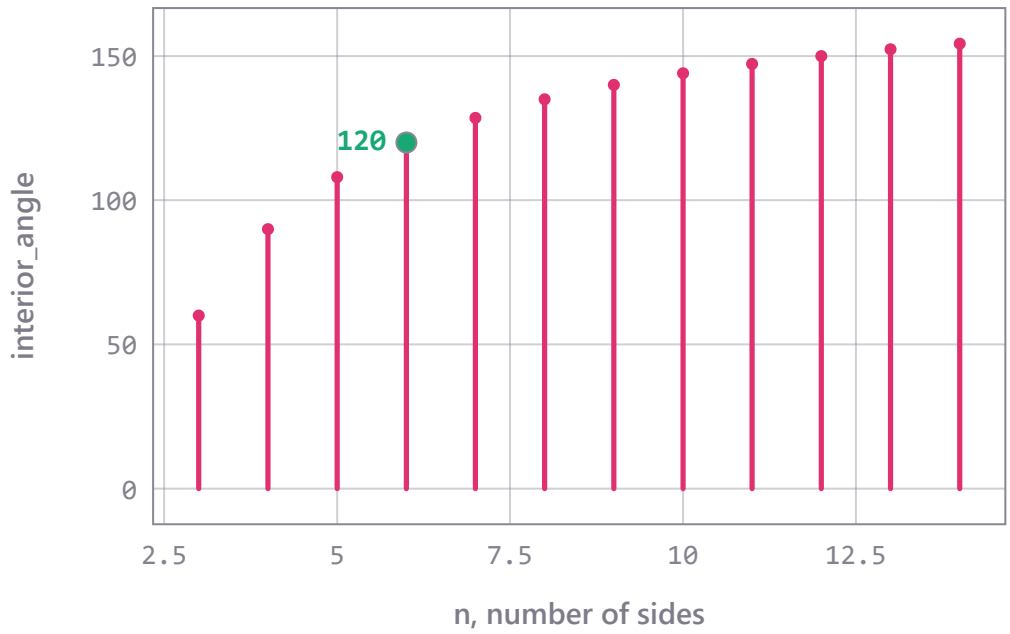
input	meaning	condition
`n`	number of sides	

output	meaning
`interior_angle`	each interior angle of the regular polygon, degrees
`exterior_angle`	each exterior angle, degrees
`interior_sum`	sum of the interior angles, degrees
`diagonals`	number of diagonals n(n-3)/2

- **Method:** sum = (n - 2) 180; interior = sum / n; exterior = 360 / n; diagonals = n(n - 3)/2
- **Assumes:** $3 \le n \le 1000$ (whole number); each single angle assumes a regular polygon
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Interior and exterior angles of the regular polygons with 3 to 20 sides, each computed by the package: the interior angle climbs towards 180° (triangle 60°, 20-gon 162°) while the exterior angle 360°/n falls; marked: n = 6, interior angle = 120°.



interior_angle (each interior angle of the regular polygon, degrees) against n (number of sides) from 1 to 14. 12 points computed by geometry-ai999x; marked: the worked example, n = 6, interior_angle = 120.

```
geometry_ai999x.solve('regular_polygon_angles', n=6)
# -> interior_angle = 120, exterior_angle = 60, interior_sum = 720, diagonals = 9   (SOLVED)
```

```
geometry_ai999x.solve('regular_polygon_angles', n=10)
# -> interior_angle = 144, exterior_angle = 36, interior_sum = 1440, diagonals = 35   (SOLVED)
```

```
geometry_ai999x.solve('regular_polygon_angles', n='5')
# -> interior_angle = 108, exterior_angle = 72, interior_sum = 540, diagonals = 5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is each interior angle of a regular hexagon?	SOLVED	interior_angle = 120, exterior_angle = 60, interior_sum = 720, diagonals = 9
Find the sum of the interior angles of a polygon with 9 sides.	SOLVED	interior_angle = 140, exterior_angle = 40, interior_sum = 1260, diagonals = 27
How many diagonals does a regular decagon have?	SOLVED	interior_angle = 144, exterior_angle = 36, interior_sum = 1440, diagonals = 35
What is the exterior angle of a regular 12-sided polygon?	SOLVED	interior_angle = 150, exterior_angle = 30, interior_sum = 1800, diagonals = 54
What is each interior angle of a regular polygon with 2 sides?	INVALID_INPUT	n: must be at least 3, got 2.
What is each interior angle of a regular polygon?	NEEDS_INPUT	regular_polygon_angles needs n (number of sides). Example: solve('regular_polygon_angles', n=6).

Regular polygon: area, perimeter, apothem, circumradius, `regular_polygon_measures`

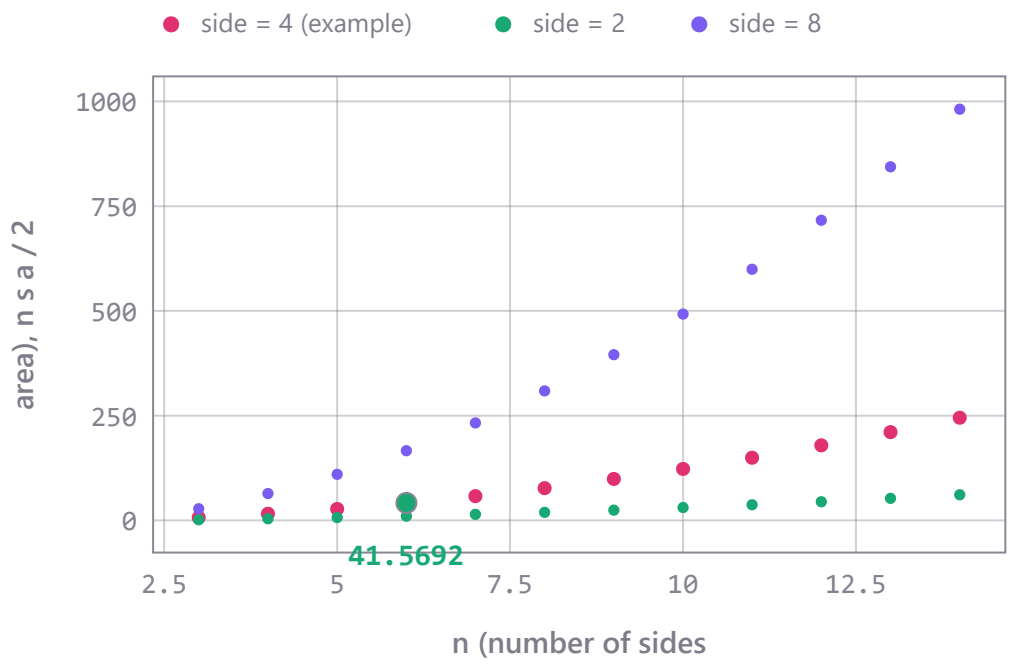
Area, perimeter, apothem (inradius) and circumradius of a regular n-gon from its side length.

$$a = \frac{s}{2\tan(\pi/n)}, \quad R = \frac{s}{2\sin(\pi/n)}, \quad A = \tfrac{1}{2}nsa$$

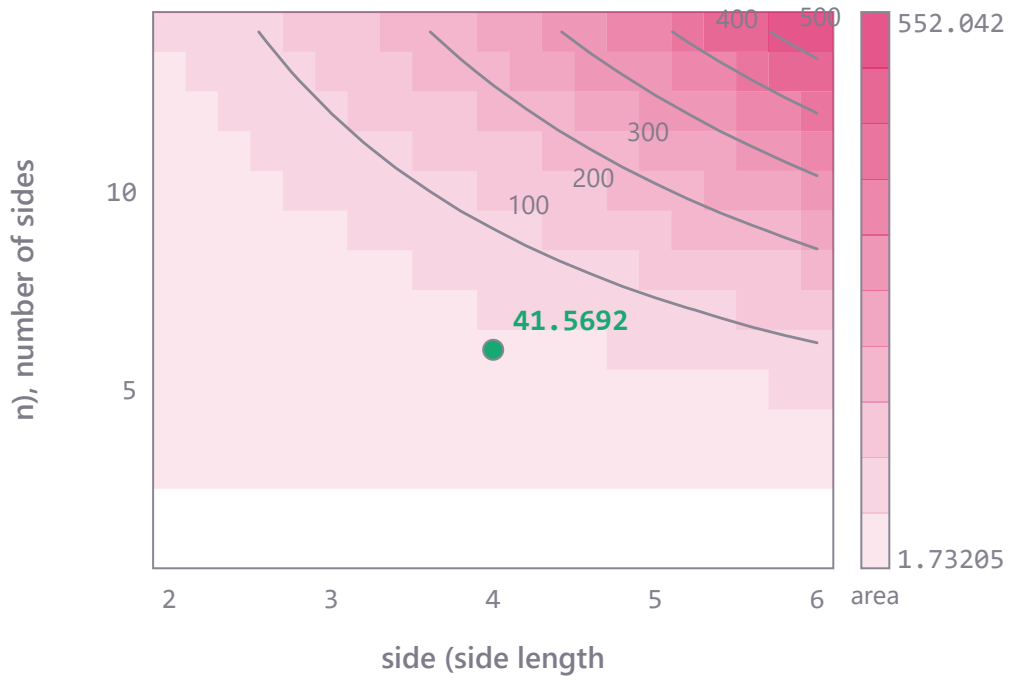
input	meaning	condition
`n`	number of sides	
`side`	side length	

output	meaning
`area`	n s $a/2$
`perimeter`	n s
`apothem`	s / (2 tan(pi/n))
`circumradius`	s / (2 sin(pi/n))

- **Method:** apothem a = s/(2 tan(pi/n)), R = s/(2 sin(pi/n)), A = (perimeter * apothem)/2
- **Assumes:** $3 \le n \le 200$; side > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



area (n s a / 2) against n (number of sides) from 1 to 14. One curve for each value of side (side length): 4, 2, 8. 12 points per curve computed by geometry-ai999x; marked: the worked example, n = 6, area = 41.5692.



area (n s a / 2) over side (side length) from 2 to 6 and n (number of sides) from 1 to 14: each of the 21 × 14 cells is one run of geometry-ai999x, shaded by its value (light 1.73205 to dark 552.042); iso-lines at area = 100, 200, 300, 400, 500; blank cells: inputs the capability refuses. Marked: the worked example, side = 4, n = 6, area = 41.5692.

```
geometry_ai999x.solve('regular_polygon_measures', n=6, side=4)
# -> area = 41.5692, perimeter = 24, apothem = 3.4641, circumradius = 4   (SOLVED)
```

```
geometry_ai999x.solve('regular_polygon_measures', n=4, side=5)
# -> area = 25, perimeter = 20, apothem = 2.5, circumradius = 3.53553   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the area of a regular hexagon with side 4.	SOLVED	area = 41.5692, perimeter = 24, apothem = 3.4641, circumradius = 4
What is the apothem of a regular octagon with side length 10?	SOLVED	area = 482.843, perimeter = 80, apothem = 12.0711, circumradius = 13.0656
Find the area of a regular polygon with 4 sides of length 5.	SOLVED	area = 25, perimeter = 20, apothem = 2.5, circumradius = 3.53553
Find the area of a regular hexagon with side -4.	INVALID_INPUT	Regular polygon: area, perimeter, apothem, circumradius: side must be positive.
Find the area of a regular hexagon.	NEEDS_INPUT	regular_polygon_measures needs side (side length). Example: solve('regular_polygon_measures', n=6, side=4).

Cuboid (rectangular box): volume, surface area, space diagonal, `cuboid_measures`

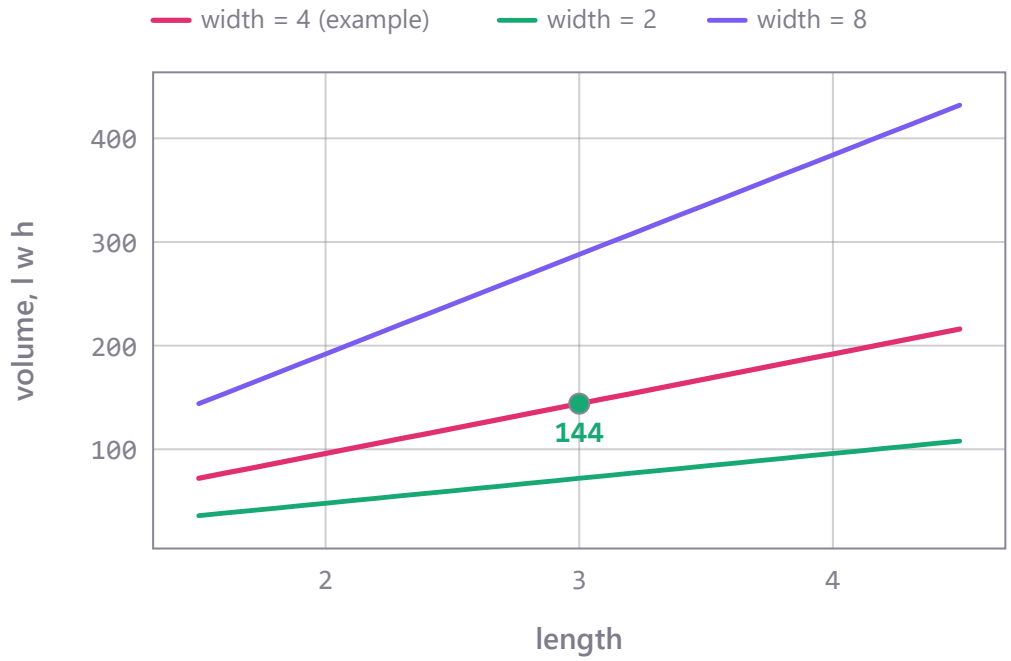
Volume, total surface area and space diagonal of a rectangular box (a cube when all edges are equal).

$$V = lwh, \quad S = 2(lw + lh + wh), \quad d = \sqrt{l^2 + w^2 + h^2}$$

input	meaning	condition
`length`	length	
`width`	width	
`height`	height	

output	meaning
`volume`	l w h
`surface_area`	2(lw + lh + wh)
`space_diagonal`	sqrt(l^2 + w^2 + h^2)

- **Method:** $V = l$ w h; $S = 2$(lw + lh + wh); $d = \sqrt{l^2 + w^2 + h^2}$
- **Assumes:** length, width, height > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



volume (l w h) against length (length) from 1.5 to 4.5. One curve for each value of width (width): 4, 2, 8. Other inputs as in the worked example: height = 12. 31 points per curve computed by geometry-ai999x; marked: the worked example, length = 3, volume = 144.

```
geometry_ai999x.solve('cuboid_measures', length=3, width=4, height=12)
# -> volume = 144, surface_area = 192, space_diagonal = 13   (SOLVED)
```

```
geometry_ai999x.solve('cuboid_measures', length=5, width=5, height=5)
# -> volume = 125, surface_area = 150, space_diagonal = 8.66025   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the volume and surface area of a box 3 by 4 by 12.	SOLVED	volume = 144, surface_area = 192, space_diagonal = 13

question	outcome	result
Find the surface area of a cube with edge 5.	SOLVED	volume = 125, surface_area = 150, space_diagonal = 8.66025
A rectangular prism has length 6, width 4 and height 3. Find its volume.	SOLVED	volume = 72, surface_area = 108, space_diagonal = 7.81025
What is the space diagonal of a box 3 by 4 by 12?	SOLVED	volume = 144, surface_area = 192, space_diagonal = 13
A rectangular prism has length 6 and width 4. Find its volume.	NEEDS_INPUT	cuboid_measures needs height (height). Example: solve('cuboid_measures', length=3, width=4, height=12).

Cylinder: volume and surface area, `cylinder_measures`

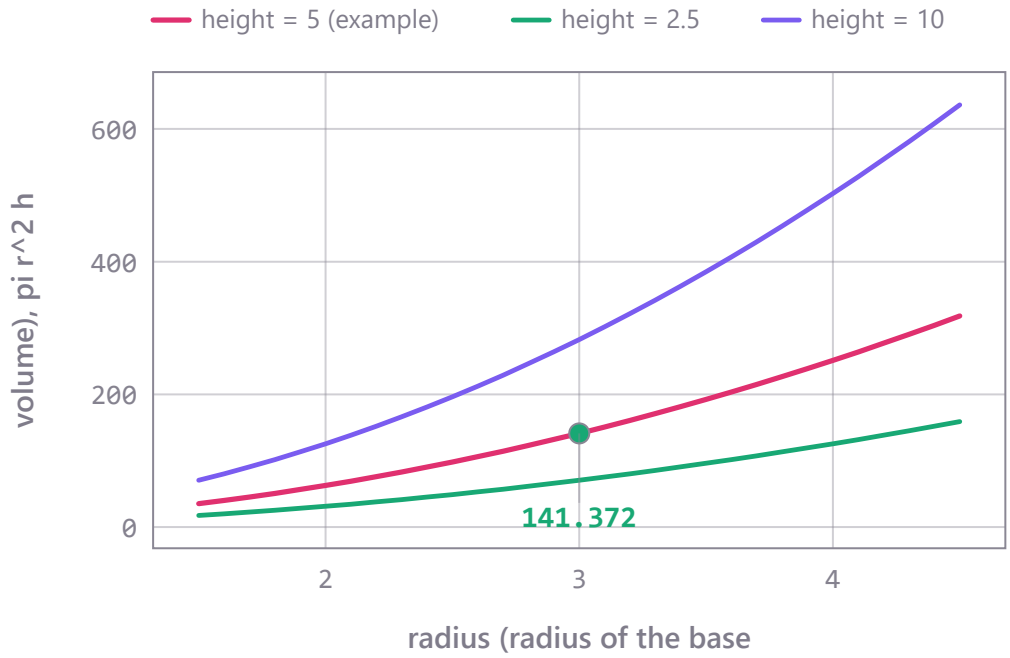
Volume, lateral (curved) area and total surface area of a right circular cylinder.

$$V = \pi r^2 h, \quad A = 2\pi r h + 2\pi r^2$$

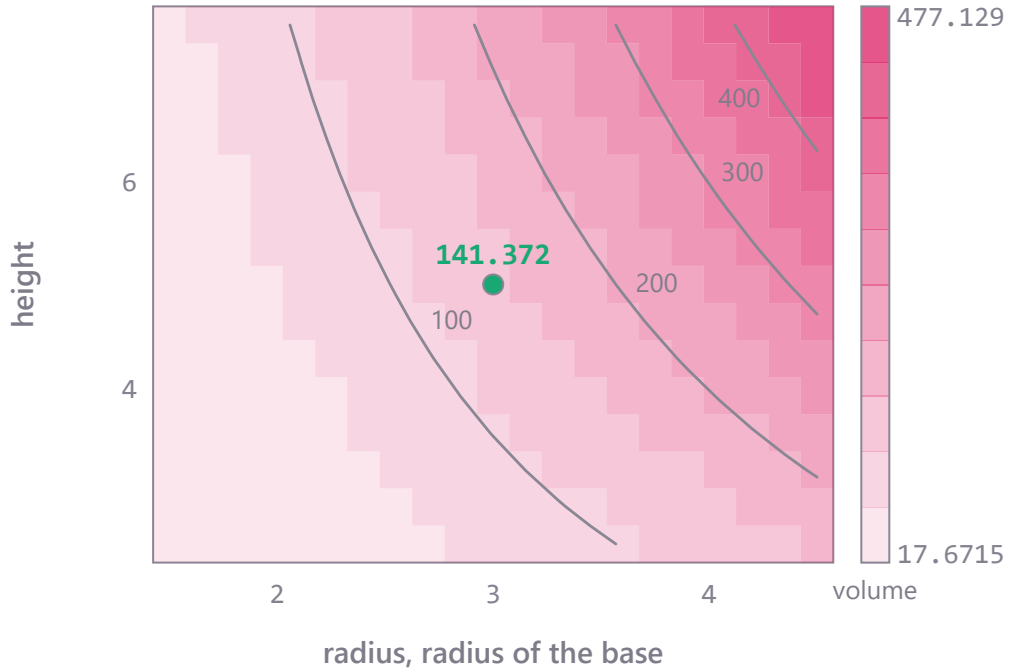
input	meaning	condition
`radius`	radius of the base	
`height`	height	

output	meaning
`volume`	pi r^2 h
`lateral_area`	2 pi r h
`total_area`	2 pi r (r + h)

- **Method:** $V = pi\, r^2$ h; lateral = 2 pi r h; total = lateral + 2 pi r^2
- **Assumes:** radius, height > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



volume (pi r^2 h) against radius (radius of the base) from 1.5 to 4.5. One curve for each value of height (height): 5, 2.5, 10. 31 points per curve computed by geometry-ai999x; marked: the worked example, radius = 3, volume = 141.372.



volume (pi r^2 h) over radius (radius of the base) from 1.5 to 4.5 and height (height) from 2.5 to 7.5: each of the 21 × 15 cells is one run of geometry-ai999x, shaded by its value (light 17.6715 to dark 477.129); iso-lines at volume = 100, 200, 300, 400. Marked: the worked example, radius = 3, height = 5, volume = 141.372.

```
geometry_ai999x.solve('cylinder_measures', radius=3, height=5)
# -> volume = 141.372, lateral_area = 94.2478, total_area = 150.796   (SOLVED)
```

```
geometry_ai999x.solve('cylinder_measures', radius=5, height=20)
# -> volume = 1570.8, lateral_area = 628.319, total_area = 785.398   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the volume of a cylinder with radius 3 and height 5.	SOLVED	volume = 141.372, lateral_area = 94.2478, total_area = 150.796
What is the total surface area of a cylinder with diameter 10 cm and height 20 cm?	SOLVED	volume = 1570.8, lateral_area = 628.319, total_area = 785.398
Find the volume of a cylinder with radius 3 cm and height 2 m.	OUT_OF_SCOPE	Out of scope: lengths are given in different units; state every length in one unit.
Find the volume of a cylinder with radius 3.	NEEDS_INPUT	cylinder_measures needs height (height). Example: solve('cylinder_measures', radius=3, height=5).

Cone: volume, slant height and surface area, `cone_measures`

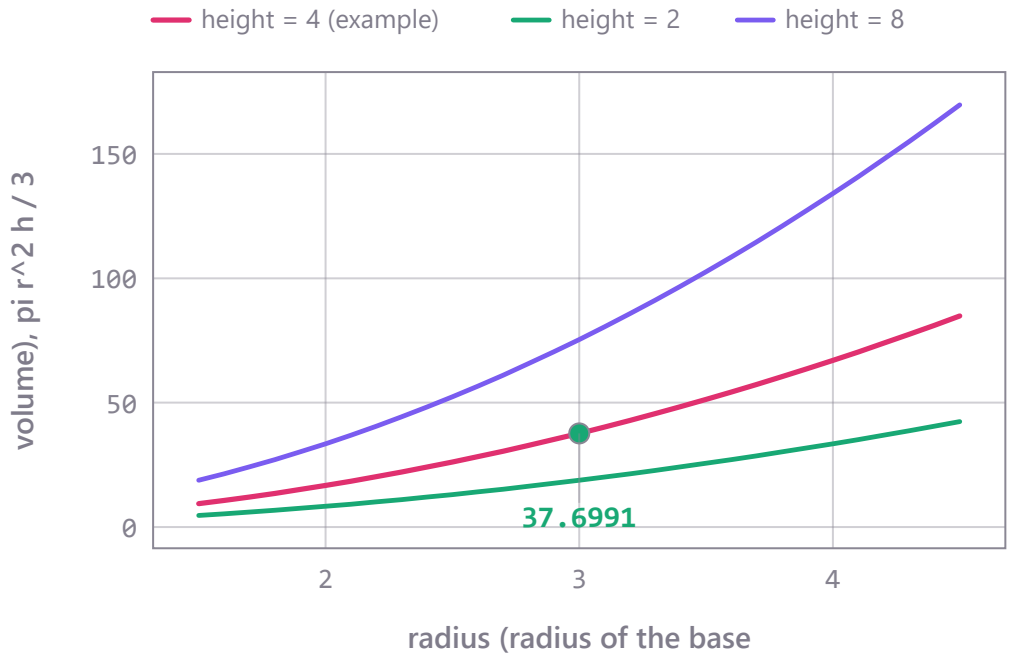
Volume, slant height, lateral area and total surface area of a right circular cone.

$$V = \tfrac{1}{3}\pi r^2 h, \quad l = \sqrt{r^2 + h^2}, \quad A_L = \pi r l$$

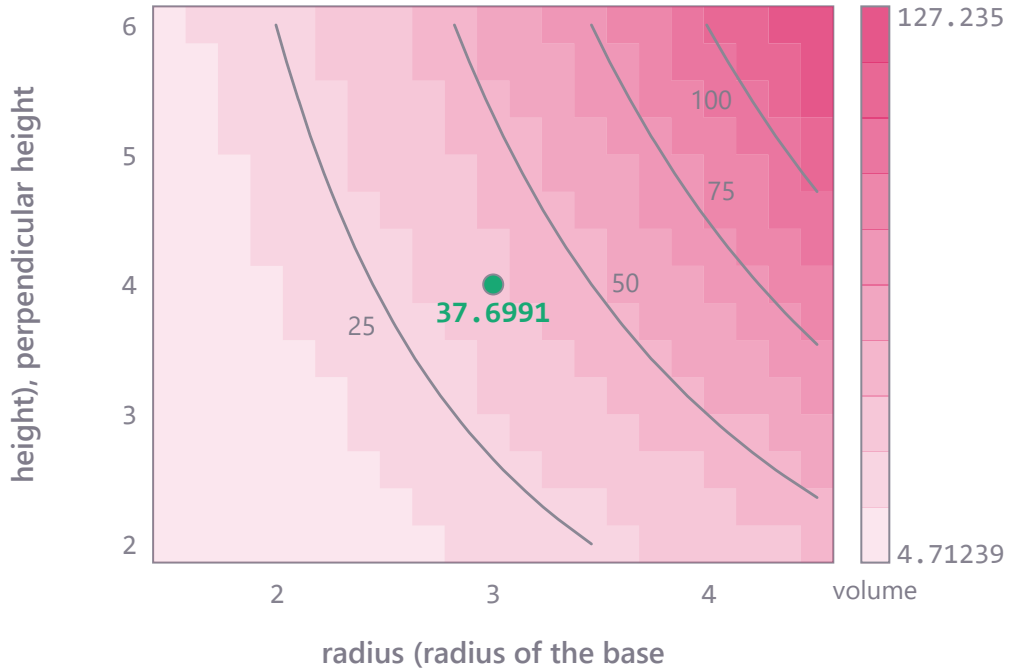
input	meaning	condition
`radius`	radius of the base	
`height`	perpendicular height	

output	meaning
`volume`	pi r^2 $h/3$
`slant_height`	sqrt(r^2 + h^2)
`lateral_area`	pi r l
`total_area`	pi r (r + l)

- **Method:** $V = pi\, r^2\, h/3$; $l = \sqrt{r^2 + h^2}$; lateral = pi r l
- **Assumes:** radius, height > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



volume (pi r^2 h / 3) against radius (radius of the base) from 1.5 to 4.5. One curve for each value of height (perpendicular height): 4, 2, 8. 31 points per curve computed by geometry-ai999x; marked: the worked example, radius = 3, volume = 37.6991.



volume (pi r^2 h / 3) over radius (radius of the base) from 1.5 to 4.5 and height (perpendicular height) from 2 to 6: each of the 21 × 15 cells is one run of geometry-ai999x, shaded by its value (light 4.71239 to dark 127.235); iso-lines at volume = 25, 50, 75, 100. Marked: the worked example, radius = 3, height = 4, volume = 37.6991.

```
geometry_ai999x.solve('cone_measures', radius=3, height=4)
# -> volume = 37.6991, slant_height = 5, lateral_area = 47.1239, total_area = 75.3982   (SOLVED)
```

```
geometry_ai999x.solve('cone_measures', radius=5, height=12)
# -> volume = 314.159, slant_height = 13, lateral_area = 204.204, total_area = 282.743   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the volume of a cone with radius 3 and height 4.	SOLVED	volume = 37.6991, slant_height = 5, lateral_area = 47.1239, total_area = 75.3982
What is the slant height of a cone with base radius 5 cm and height 12 cm?	SOLVED	volume = 314.159, slant_height = 13, lateral_area = 204.204, total_area = 282.743
Find the total surface area of a cone with radius 3 and height 4.	SOLVED	volume = 37.6991, slant_height = 5, lateral_area = 47.1239, total_area = 75.3982
Find the volume of a cone with height 4.	NEEDS_INPUT	cone_measures needs radius (radius of the base). Example: solve('cone_measures', radius=3, height=4).

Sphere: volume and surface area, `sphere_measures`

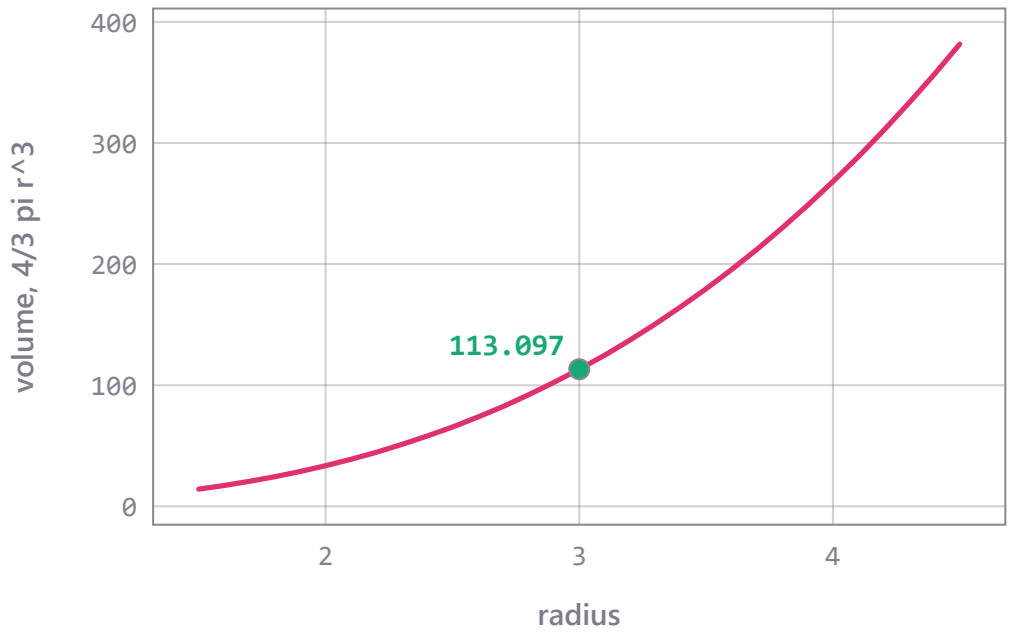
Volume and surface area of a sphere from its radius (or diameter).

$$V = \tfrac{4}{3}\pi r^3, \quad S = 4\pi r^2$$

input	meaning	condition
`radius`	radius	

output	meaning
`volume`	$4/3$ pi r^3
`surface_area`	4 pi r^2

- **Method:** $V = 4/3$ pi r^3; $S = 4$ pi r^2
- **Assumes:** radius > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



volume (4/3 pi r^3) against radius (radius) from 1.5 to 4.5. 31 points computed by geometry-ai999x; marked: the worked example, radius = 3, volume = 113.097.

```
geometry_ai999x.solve('sphere_measures', radius=3)
# -> volume = 113.097, surface_area = 113.097   (SOLVED)
```

```
geometry_ai999x.solve('sphere_measures', radius=5)
# -> volume = 523.599, surface_area = 314.159   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the volume of a sphere with radius 3.	SOLVED	volume = 113.097, surface_area = 113.097
What is the surface area of a ball with diameter 10 cm?	SOLVED	volume = 523.599, surface_area = 314.159
Find the volume of a sphere with radius 0.	INVALID_INPUT	Sphere: volume and surface area: radius must be positive.
Find the volume of a sphere.	NEEDS_INPUT	sphere_measures needs radius (radius). Example: solve('sphere_measures', radius=3).

Classify a triangle by its sides and angles, `triangle_classify`

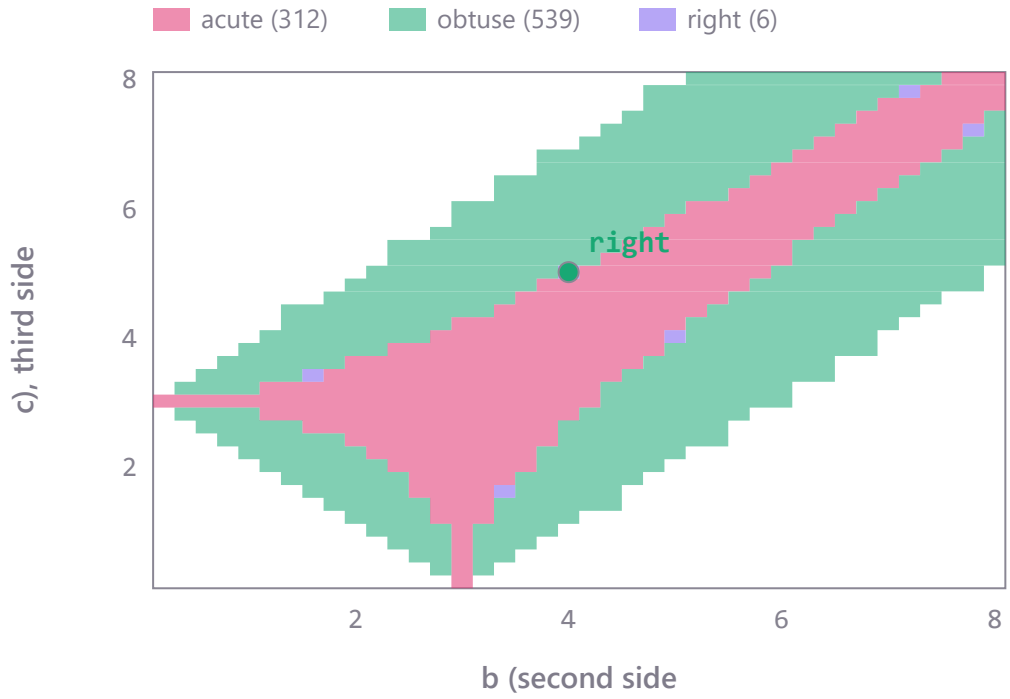
Whether a triangle with sides a, b, c is equilateral, isosceles or scalene, and acute, right or obtuse.

$$x^2 + y^2 \gtreqless z^2 \;\Leftrightarrow\; \text{acute / right / obtuse}$$

input	meaning	condition
`a`	side a	
`b`	side b	
`c`	side c	

output	meaning
`by_sides`	equilateral / isosceles / scalene
`by_angles`	acute / right / obtuse

- **Method:** exact comparison: with z the longest side, $x^2 + y^2 - z^2 > 0$ acute, = 0 right, < 0 obtuse (converse of Pythagoras)
- **Assumes:** a, b, $c > 0$; triangle inequality; the sides are compared exactly as written
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Type by angles of the triangle with sides a = 3 (as in the worked example), b and c from 0.2 to 8. Each of the 40 × 40 cells is one run of the package, coloured by the by_angles it returned: acute: 312 cells, obtuse: 539 cells, right: 6 cells; 743 blank cells: inputs the package refuses. Marked: the worked example b = 4, c = 5 → right (scalene).

```
geometry_ai999x.solve('triangle_classify', a=3, b=4, c=5)
# -> by_sides = scalene, by_angles = right   (SOLVED)
```

```
geometry_ai999x.solve('triangle_classify', a=5, b=5, c=8)
# -> by_sides = isosceles, by_angles = obtuse   (SOLVED)
```

```
geometry_ai999x.solve('triangle_classify', a=0.3, b=0.4, c=0.5)
# -> by_sides = scalene, by_angles = right   (SOLVED)
```

```
geometry_ai999x.solve('triangle_classify', a=6, b=6, c=6)
# -> by_sides = equilateral, by_angles = acute   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Classify the triangle with sides 3, 4 and 5.	SOLVED	by_sides = scalene, by_angles = right
What type of triangle has sides 5, 5 and 8?	SOLVED	by_sides = isosceles, by_angles = obtuse
Is the triangle with sides 7, 8 and 9 acute, right or obtuse?	SOLVED	by_sides = scalene, by_angles = acute
Classify the triangle with sides 1, 1 and 3.	INVALID_INPUT	Classify a triangle by its sides and angles: the sides violate the triangle inequality (each side must be shorter than t

Polygon from vertices: area, perimeter, orientation, `polygon_area`

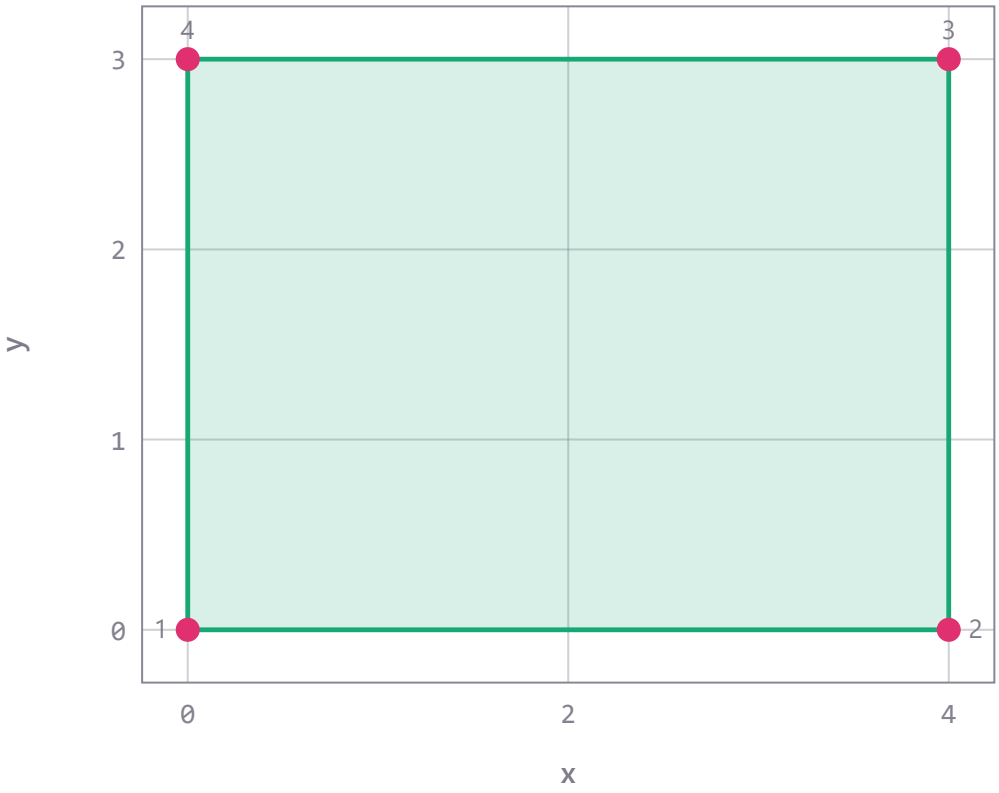
Area (shoelace formula), perimeter and orientation of a simple polygon given its vertices in order.

$$A = \frac{1}{2}\left|\sum_i (x_i y_{i+1} - x_{i+1} y_i)\right|$$

input	meaning	condition
`points`	vertices in order: x1 y1, x2 y2, ...	

output	meaning
`area`	enclosed area
`perimeter`	sum of the edge lengths
`orientation`	counterclockwise or clockwise

- **Method:** shoelace formula A = |sum(x_i y_{$i+1$} - x_{$i+1$} y_i)| / 2; the sign gives the orientation; simplicity checked exactly
- **Assumes:** at least 3 vertices, in order; the polygon is simple (its edges do not cross)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The rows of points (vertices in order: x1 y1, x2 y2, ...) drawn as points (numbered in order), joined in order as a polygon. Returned: area = 12, perimeter = 14.

```
geometry_ai999x.solve('polygon_area', points='0 0, 4 0, 4 3, 0 3')
# -> area = 12, perimeter = 14, orientation = counterclockwise   (SOLVED)
```

```
geometry_ai999x.solve('polygon_area', points=[0, 0, 0, 2, 2, 2, 2, 0])
# -> area = 4, perimeter = 8, orientation = clockwise   (SOLVED)
```

```
geometry_ai999x.solve('polygon_area', points='0 0, 4 0, 4 4, 2 1, 0 4')
# -> area = 10, perimeter = 19.2111, orientation = counterclockwise   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the area of the polygon with vertices (0, 0), (4, 0), (4, 3), (0, 3).	SOLVED	area = 12, perimeter = 14, orientation = counterclockwise
What is the area of the pentagon with vertices (0, 0), (4, 0), (4, 4), (2, 1), (0, 4)?	SOLVED	area = 10, perimeter = 19.2111, orientation = counterclockwise
Find the area of the quadrilateral with vertices (0, 0), (2, 2), (2, 0), (0, 2).	INVALID_INPUT	Polygon from vertices: area, perimeter, orientation: points: the polygon is not simple (edges 1 and 3 meet).

Centroid of a polygon, `polygon_centroid`

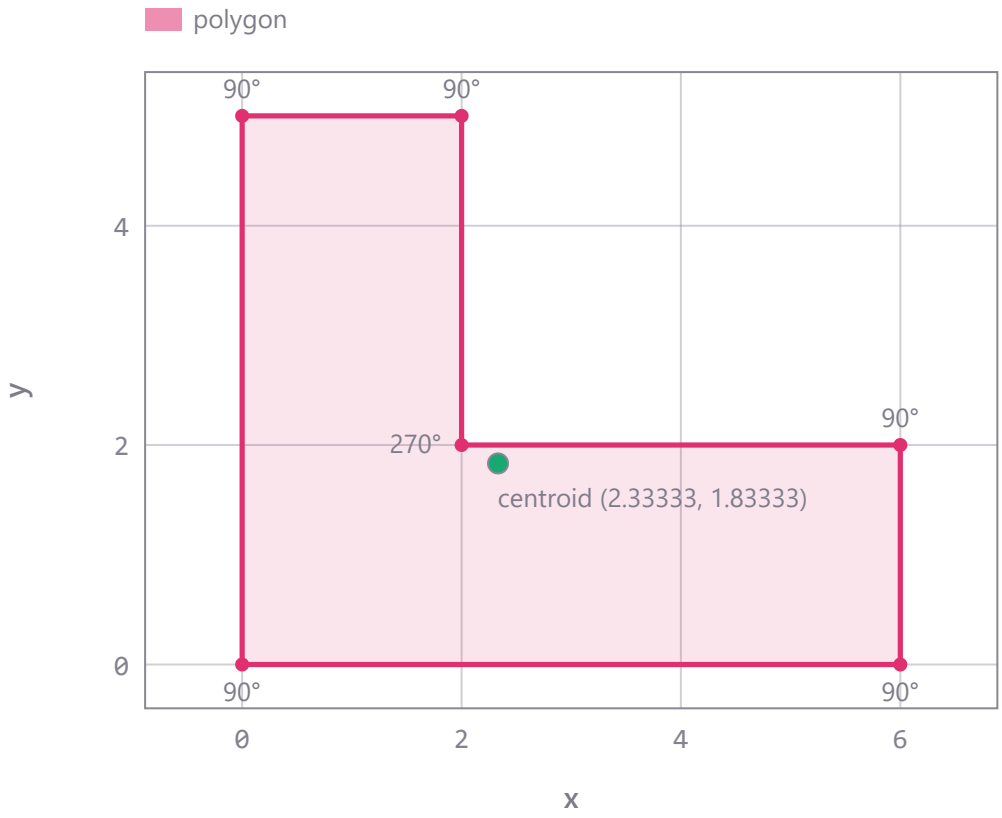
The centroid (centre of mass of the uniform region) of a simple polygon given its vertices in order.

$$C_x = \frac{1}{6A}\sum_i (x_i + x_{i+1})(x_i y_{i+1} - x_{i+1} y_i)$$

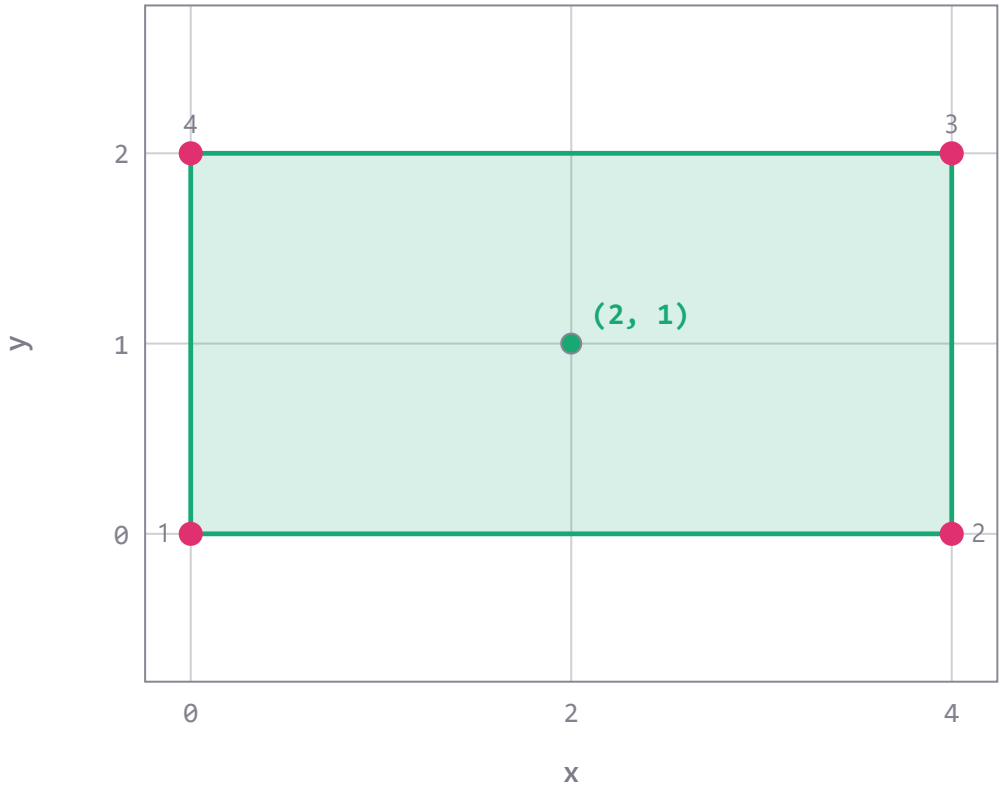
input	meaning	condition
`points`	vertices in order: x1 y1, x2 y2, ...	

output	meaning
`cx`	x of the centroid
`cy`	y of the centroid

- **Method:** area-weighted shoelace sums: Cx = sum((x_i + x_{$i+1$}) w_i) / (3 * sum w_i), $w_i = x_i$ y_{$i+1$} - x_{$i+1$} y_i
- **Assumes:** at least 3 vertices, in order; simple polygon
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The L-shaped polygon 0 0, 6 0, 6 2, 2 2, 2 5, 0 5, with the interior angle at each vertex (sum 720°, 1 reflex), area 18 and perimeter 22; marked: the centroid x = 2.33333, y = 1.83333, all computed by the package. The centroid of a non-convex shape need not be its middle.



The rows of points (vertices in order: x1 y1, x2 y2, ...) drawn as points (numbered in order), joined in order as a polygon, returned (x, y) pairs marked. Returned: cx = 2, cy = 1.

```
geometry_ai999x.solve('polygon_centroid', points='0 0, 4 0, 4 2, 0 2')
# -> cx = 2, cy = 1   (SOLVED)
```

```
geometry_ai999x.solve('polygon_centroid', points='0 0, 6 0, 6 6, 0 3')
# -> cx = 3.33333, cy = 2.33333   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the centroid of the polygon with vertices (0, 0), (4, 0), (4, 2), (0, 2).	SOLVED	cx = 2, cy = 1
Where is the centre of mass of the quadrilateral with vertices (0, 0), (6, 0), (6, 6), (0, 3)?	SOLVED	cx = 3.33333, cy = 2.33333

Triangle from vertex coordinates: area, perimeter, centroid, angles, `triangle_coordinates`

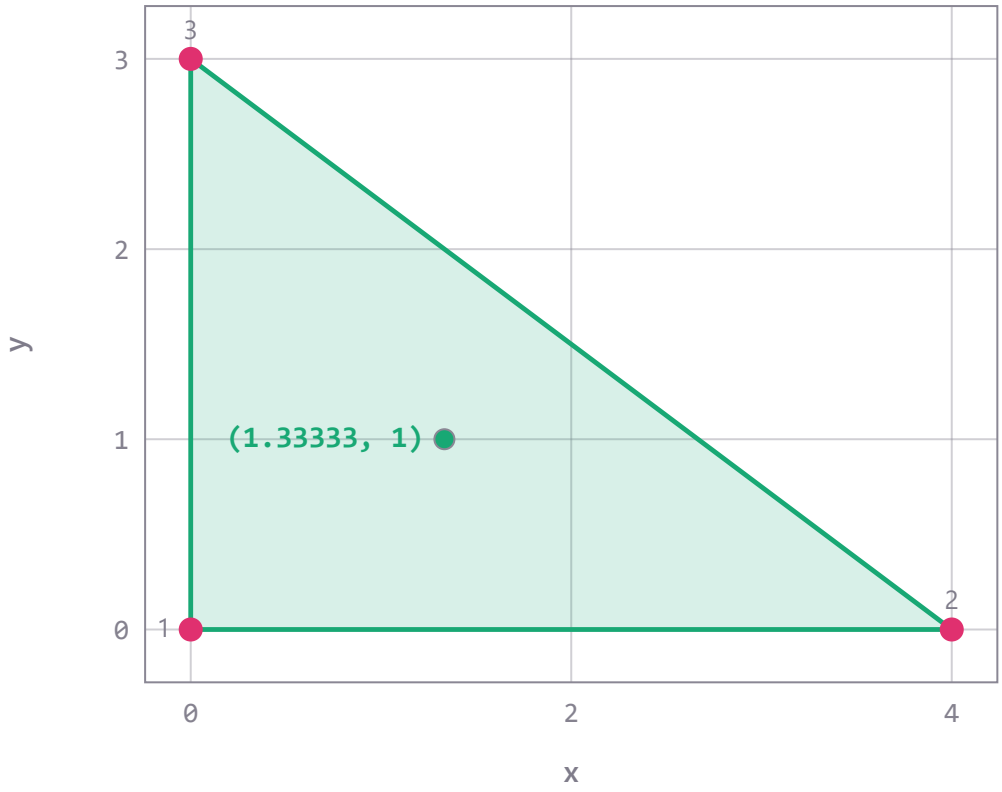
Area, perimeter, centroid and the angles at A, B, C of the triangle with vertices A, B, C.

$$K = \tfrac{1}{2}|(x_B - x_A)(y_C - y_A) - (x_C - x_A)(y_B - y_A)|, \quad G = \tfrac{1}{3}(A + B + C)$$

input	meaning	condition
`points`	the vertices A, B, C: xA yA, xB yB, xC yC	

output	meaning
`area`	area (shoelace)
`perimeter`	perimeter
`gx`	x of the centroid
`gy`	y of the centroid
`angle_A`	angle at A, degrees
`angle_B`	angle at B, degrees
`angle_C`	angle at C, degrees

- **Method:** shoelace area; centroid = mean of the vertices; angles by the law of cosines as atan2(4K, b^2 + c^2 - a^2)
- **Assumes:** exactly 3 vertices; not collinear
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The rows of points (the vertices A, B, C: xA yA, xB yB, xC yC) drawn as points (numbered in order), joined in order as a polygon, returned (x, y) pairs marked. Returned: area = 6, perimeter = 12, gx = 1.33333, gy = 1, angle_A = 90, angle_B = 36.8699, angle_C = 53.1301.

```
geometry_ai999x.solve('triangle_coordinates', points='0 0, 4 0, 0 3')
# -> area = 6, perimeter = 12, gx = 1.33333, gy = 1, angle_A = 90, angle_B = 36.8699, angle_C =
53.1301   (SOLVED)
```

```
geometry_ai999x.solve('triangle_coordinates', points=[0, 0, 2, 0, 1, 1.7320508075688772])
# -> area = 1.73205, perimeter = 6, gx = 1, gy = 0.57735, angle_A = 60, angle_B = 60, angle_C = 60
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the area of the triangle with vertices (0, 0), (4, 0) and (0, 3).	SOLVED	area = 6, perimeter = 12, gx = 1.33333, gy = 1, angle_A = 90, angle_B = 36.8699, angle_C = 53.1301
What is the centroid of the triangle with vertices (1, 1), (5, 1) and (3, 4)?	SOLVED	area = 6, perimeter = 11.2111, gx = 3, gy = 2, angle_A = 56.3099, angle_B = 56.3099, angle_C = 67.3801
Find the area of the triangle with vertices (0, 0), (1, 1) and (2, 2).	INVALID_INPUT	Triangle from vertex coordinates: area, perimeter, centroid, angles: the three points are collinear: they do not form a

Circumcircle: the circle through three points, `triangle_circumcircle`

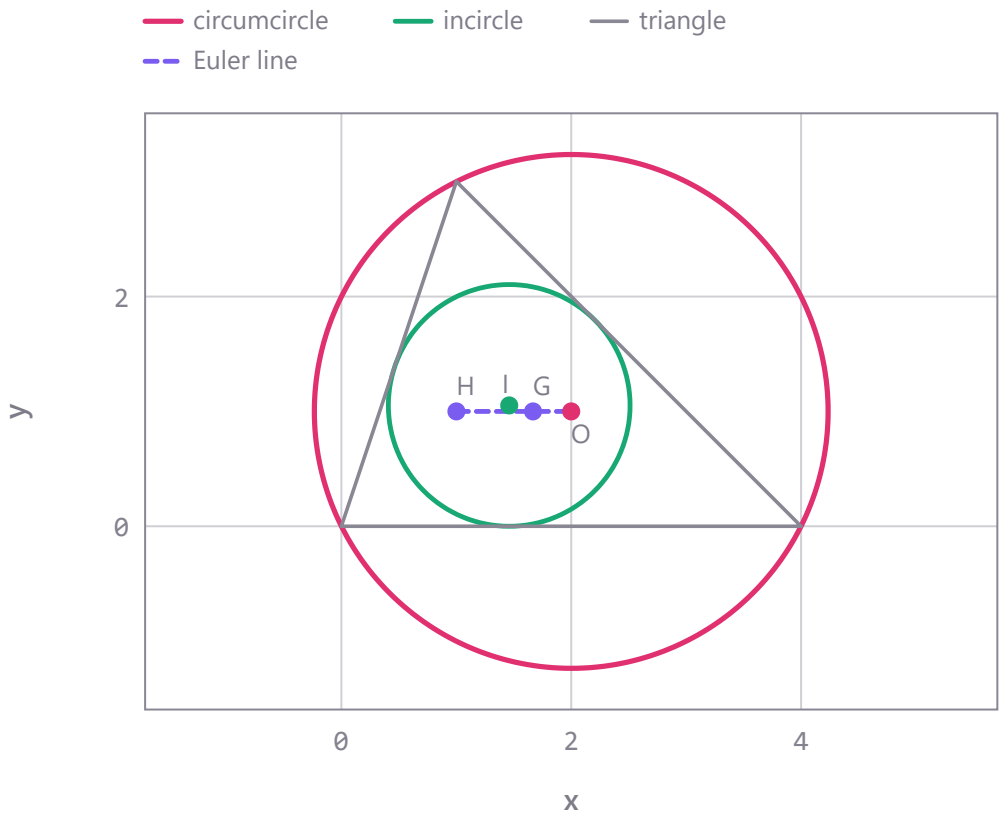
Circumcentre and circumradius of a triangle given by its vertices (the circle through three points).

$$|O-A| = |O-B| = |O-C| = R$$

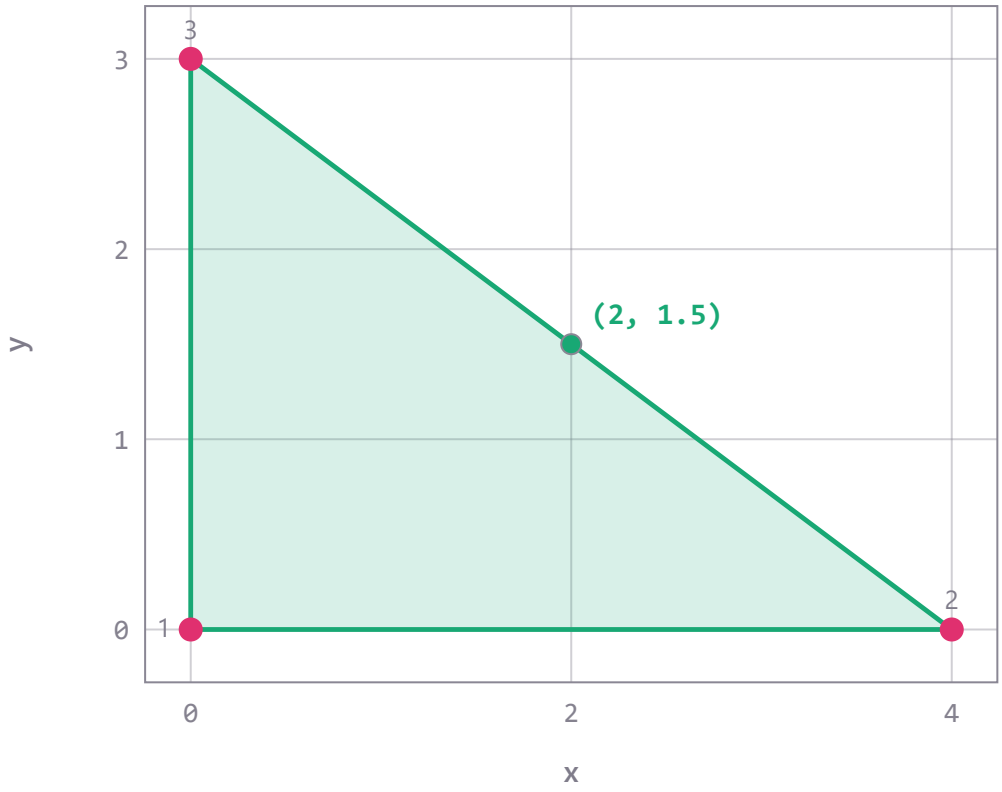
input	meaning	condition
`points`	the three points: x1 y1, x2 y2, x3 y3	

output	meaning
`cx`	x of the circumcentre
`cy`	y of the circumcentre
`radius`	circumradius

- **Method:** intersection of the perpendicular bisectors, solved in coordinates relative to the first point
- **Assumes:** exactly 3 points; not collinear
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The triangle with vertices 0 0, 4 0, 1 3: circumcircle (centre O = (2, 1), radius 2.23607), incircle (centre I = (1.45982, 1.05218), radius 1.05218), orthocentre H = (1, 1) and centroid G = (1.66667, 1), each computed by the package; O, G and H lie on the Euler line.



The rows of points (the three points: x1 y1, x2 y2, x3 y3) drawn as points (numbered in order), joined in order as a polygon, returned (x, y) pairs marked. Returned: cx = 2, cy = 1.5, radius = 2.5.

```
geometry_ai999x.solve('triangle_circumcircle', points='0 0, 4 0, 0 3')
# -> cx = 2, cy = 1.5, radius = 2.5   (SOLVED)
```

```
geometry_ai999x.solve('triangle_circumcircle', points='1 0, 0 1, -1 0')
# -> cx = 0, cy = 0, radius = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the circumcenter and circumradius of the triangle with vertices (0, 0), (4, 0) and (0, 3).	SOLVED	cx = 2, cy = 1.5, radius = 2.5
Find the circle through (1, 0), (0, 1) and (-1, 0).	SOLVED	cx = 0, cy = 0, radius = 1
Find the circle passing through (0, 0), (1, 1) and (2, 2).	INVALID_INPUT	Circumcircle: the circle through three points: the three points are collinear: they do not form a triangle.

Incircle of a triangle: incentre and inradius, `triangle_incircle`

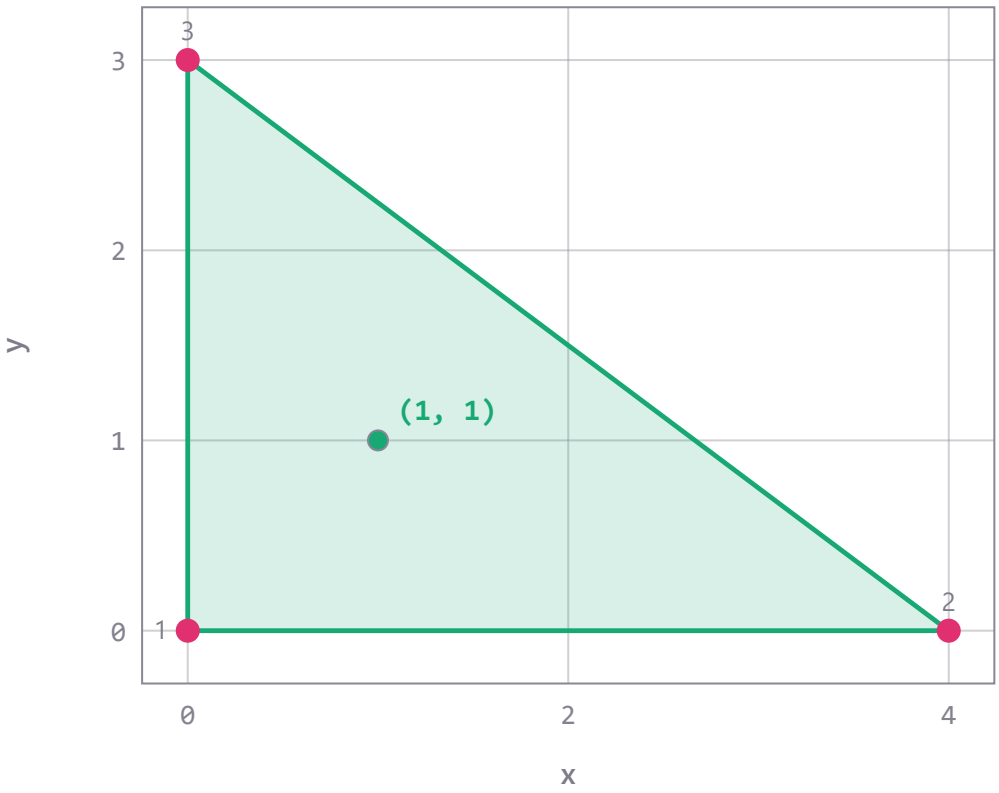
Incentre and inradius of the triangle with the given vertices.

$$I = \frac{aA + bB + cC}{a + b + c}, \quad r = \frac{2K}{a + b + c}$$

input	meaning	condition
`points`	the vertices: xA yA, xB yB, xC yC	

output	meaning
`ix`	x of the incentre
`iy`	y of the incentre
`radius`	inradius

- **Method:** incentre = (a A + b B + c C)/(a + b + c) with a, b, c the opposite side lengths; $r = 2K/(a+b+c)$
- **Assumes:** exactly 3 points; not collinear
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The rows of points (the vertices: xA yA, xB yB, xC yC) drawn as points (numbered in order), joined in order as a polygon, returned (x, y) pairs marked. Returned: ix = 1, iy = 1, radius = 1.

```
geometry_ai999x.solve('triangle_incircle', points='0 0, 4 0, 0 3')
# -> ix = 1, iy = 1, radius = 1   (SOLVED)
```

```
geometry_ai999x.solve('triangle_incircle', points='0 0, 6 0, 0 8')
# -> ix = 2, iy = 2, radius = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the incenter and inradius of the triangle with vertices (0, 0), (4, 0) and (0, 3).	SOLVED	ix = 1, iy = 1, radius = 1
What is the radius of the inscribed circle of the triangle with vertices (0, 0), (6, 0) and (0, 8)?	SOLVED	ix = 2, iy = 2, radius = 2

Orthocentre of a triangle, `triangle_orthocenter`

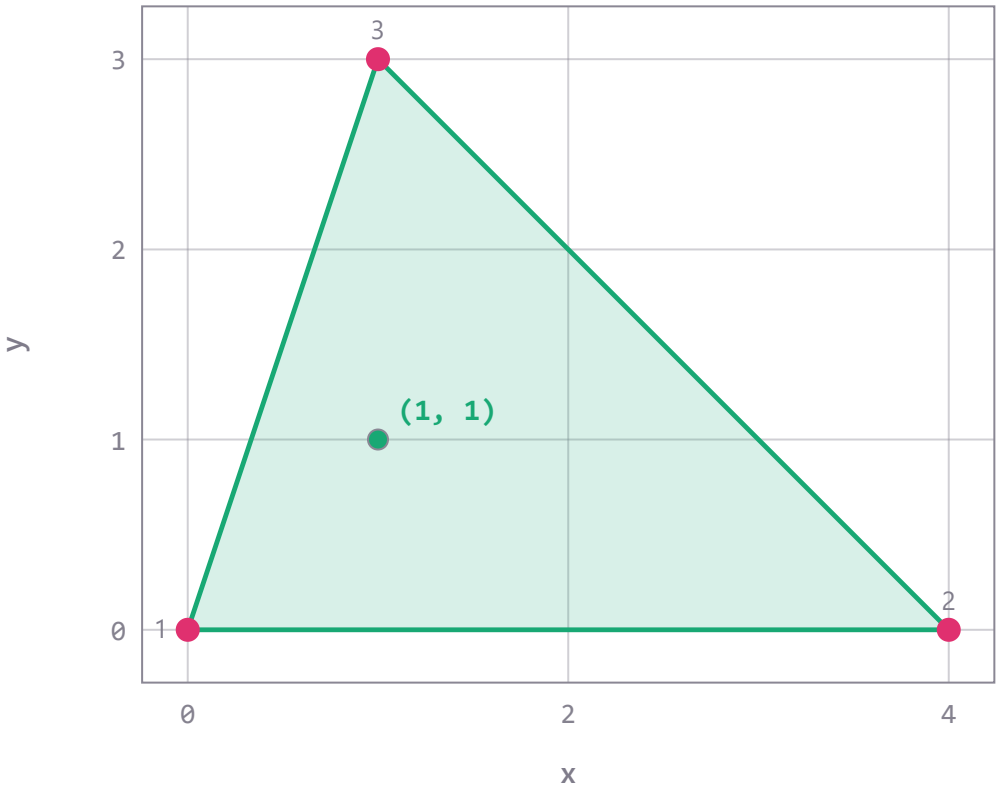
The orthocentre (common point of the three altitudes) of the triangle with the given vertices.

$$(H - A) \cdot (C - B) = 0, \quad (H - B) \cdot (C - A) = 0$$

input	meaning	condition
`points`	the vertices: xA yA, xB yB, xC yC	

output	meaning
`hx`	x of the orthocentre
`hy`	y of the orthocentre

- **Method:** solve (H - A).$(C - B) = 0$ and (H - B).$(C - A) = 0$ (two altitudes) by Cramer's rule
- **Assumes:** exactly 3 points; not collinear
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The rows of points (the vertices: xA yA, xB yB, xC yC) drawn as points (numbered in order), joined in order as a polygon, returned (x, y) pairs marked. Returned: hx = 1, hy = 1.

```
geometry_ai999x.solve('triangle_orthocenter', points='0 0, 4 0, 1 3')
# -> hx = 1, hy = 1   (SOLVED)
```

```
geometry_ai999x.solve('triangle_orthocenter', points='0 0, 4 0, 0 3')
# -> hx = 0, hy = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the orthocenter of the triangle with vertices (0, 0), (4, 0) and (1, 3).	SOLVED	hx = 1, hy = 1
Where do the altitudes meet in the triangle with vertices (0, 0), (4, 0) and (0, 3)?	SOLVED	hx = 0, hy = 0

Intersection point of two lines, `line_intersection`

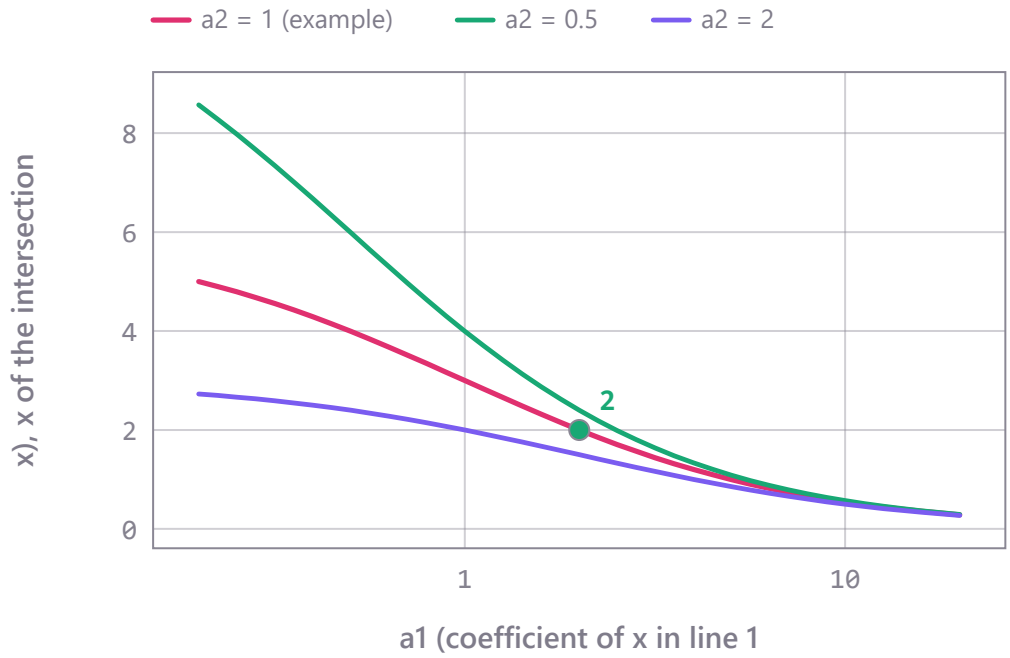
The point where two lines a1 $x + b_1\, y + c_1 = 0$ and a2 $x + b_2\, y + c_2 = 0$ meet.

$$x = \frac{b_1 c_2 - b_2 c_1}{a_1 b_2 - a_2 b_1}, \quad y = \frac{c_1 a_2 - c_2 a_1}{a_1 b_2 - a_2 b_1}$$

input	meaning	condition
`a1`	coefficient of x in line 1 (a1 $x + b_1$ y + c1 = 0)	
`b1`	coefficient of y in line 1	
`c1`	constant of line 1	
`a2`	coefficient of x in line 2 (a2 $x + b_2$ y + c2 = 0)	
`b2`	coefficient of y in line 2	
`c2`	constant of line 2	

output	meaning
`x`	x of the intersection
`y`	y of the intersection

- **Method:** Cramer's rule: x = (b1 $c_2 - b_2$ c1)/D, y = (c1 $a_2 - c_2$ a1)/D, $D = a_1\, b_2 - a_2$ b1
- **Assumes:** each line has a or b nonzero; the lines are not parallel (exact test on the written coefficients)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



x (x of the intersection) against a1 (coefficient of x in line 1 (a1 x + b1 y + c1 = 0)) from 0.2 to 20 on a logarithmic axis. One curve for each value of a2 (coefficient of x in line 2 (a2 x + b2 y + c2 = 0)): 1, 0.5, 2. Other inputs as in the worked example: b1 = 1, c1 = -5, b2 = -1, c2 = -1. 41 points per curve computed by geometry-ai999x; marked: the worked example, a1 = 2, x = 2.

```
geometry_ai999x.solve('line_intersection', a1=2, b1=1, c1=-5, a2=1, b2=-1, c2=-1)
# -> x = 2, y = 1   (SOLVED)
```

```
geometry_ai999x.solve('line_intersection', a1=2, b1=-1, c1=1, a2=1, b2=1, c2=-7)
# -> x = 2, y = 5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the intersection of the lines 2x + y = 5 and x - y = 1.	SOLVED	x = 2, y = 1
Where do the lines y = 2x + 1 and y = -x + 7 meet?	SOLVED	x = 2, y = 5
Find the intersection of the lines y = 2x + 1 and y = 2x + 3.	INVALID_INPUT	Intersection point of two lines: the lines are parallel (or the same line): there is no single intersection point.

Angle between two lines, `angle_between_lines`

The acute (or right) angle between two lines given by equations.

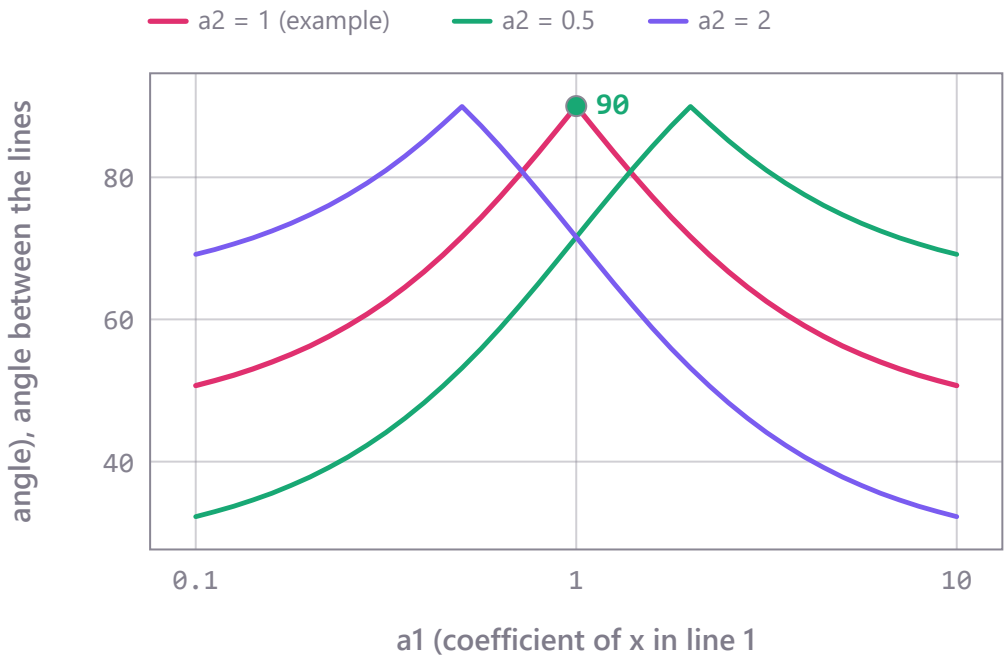
$$\tan\theta = \left|\frac{a_1 b_2 - a_2 b_1}{a_1 a_2 + b_1 b_2}\right|$$

input	meaning	condition
`a1`	coefficient of x in line 1 (a1 x + b_1 y + c1 = 0)	
`b1`	coefficient of y in line 1	
`c1`	constant of line 1	

input	meaning	condition
`a2`	coefficient of x in line 2 (a2 $x + b_2$ y + c2 = 0)	
`b2`	coefficient of y in line 2	
`c2`	constant of line 2	

output	meaning
`angle`	angle between the lines, degrees, 0 to 90

- **Method:** directions (b, -a); angle = atan2(|cross|, |dot|)
- **Assumes:** each line has a or b nonzero; the acute angle is reported (parallel lines: 0)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



angle (angle between the lines, degrees, 0 to 90) against a1 (coefficient of x in line 1 (a1 x + b1 y + c1 = 0)) from 0.1 to 10 on a logarithmic axis. One curve for each value of a2 (coefficient of x in line 2 (a2 x + b2 y + c2 = 0)): 1, 0.5, 2. Other inputs as in the worked example: b1 = -1, c1 = 1, b2 = 1, c2 = -3. 41 points per curve computed by geometry-ai999x; marked: the worked example, a1 = 1, angle = 90.

```
geometry_ai999x.solve('angle_between_lines', a1=1, b1=-1, c1=1, a2=1, b2=1, c2=-3)
# -> angle = 90   (SOLVED)
```

```
geometry_ai999x.solve('angle_between_lines', a1=2, b1=-1, c1=1, a2=3, b2=-1, c2=-2)
# -> angle = 8.1301   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the angle between the lines y = x + 1 and y = -x + 3.	SOLVED	angle = 90
What is the angle between the lines y = 2x + 1 and y = 3x - 2?	SOLVED	angle = 8.1301
Find the angle between the lines x = 0 and y = 0.	SOLVED	angle = 90

Reflection of a point in a line, `reflect_point_line`

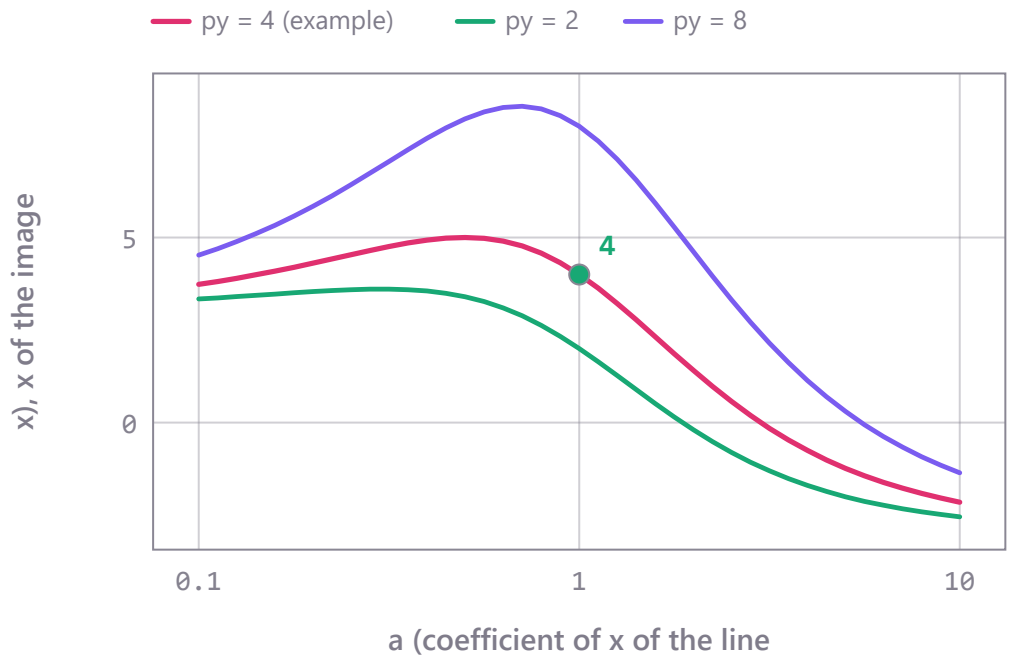
The mirror image of a point in the line a x + b $y + c = 0$ (including the axes and y = x).

$$P' = P - 2\,\frac{ap_x + bp_y + c}{a^2 + b^2}\,(a, b)$$

input	meaning	condition
px	x of the point	
py	y of the point	
a	coefficient of x of the line	
b	coefficient of y of the line	
c	constant of the line	

output	meaning
x	x of the image
y	y of the image

- **Method:** P' = $P - 2$ (a px + b py + c)/(a^2 + b^2) (a, b)
- **Assumes:** a and b not both 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



x (x of the image) against a (coefficient of x of the line) from 0.1 to 10 on a logarithmic axis. One curve for each value of py (y of the point): 4, 2, 8. Other inputs as in the worked example: px = 3, b = -1, c = 0. 41 points per curve computed by geometry-ai999x; marked: the worked example, a = 1, x = 4.

```
geometry_ai999x.solve('reflect_point_line', px=3, py=4, a=1, b=-1, c=0)
# -> x = 4, y = 3   (SOLVED)
```

```
geometry_ai999x.solve('reflect_point_line', px=1, py=2, a=1, b=1, c=-5)
# -> x = 3, y = 4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Reflect the point (3, 4) in the line y = x.	SOLVED	x = 4, y = 3
Find the reflection of (1, 2) across the line x + y - 5 = 0.	SOLVED	x = 3, y = 4
Find the mirror image of (2, -3) in the x-axis.	SOLVED	x = 2, y = 3
Reflect the point (3, 4) in the line.	NEEDS_INPUT	reflect_point_line needs a, b, c (coefficient of x of the line; coefficient of y of the line; constant of the line). Exa

Rotation of a point, `rotate_point`

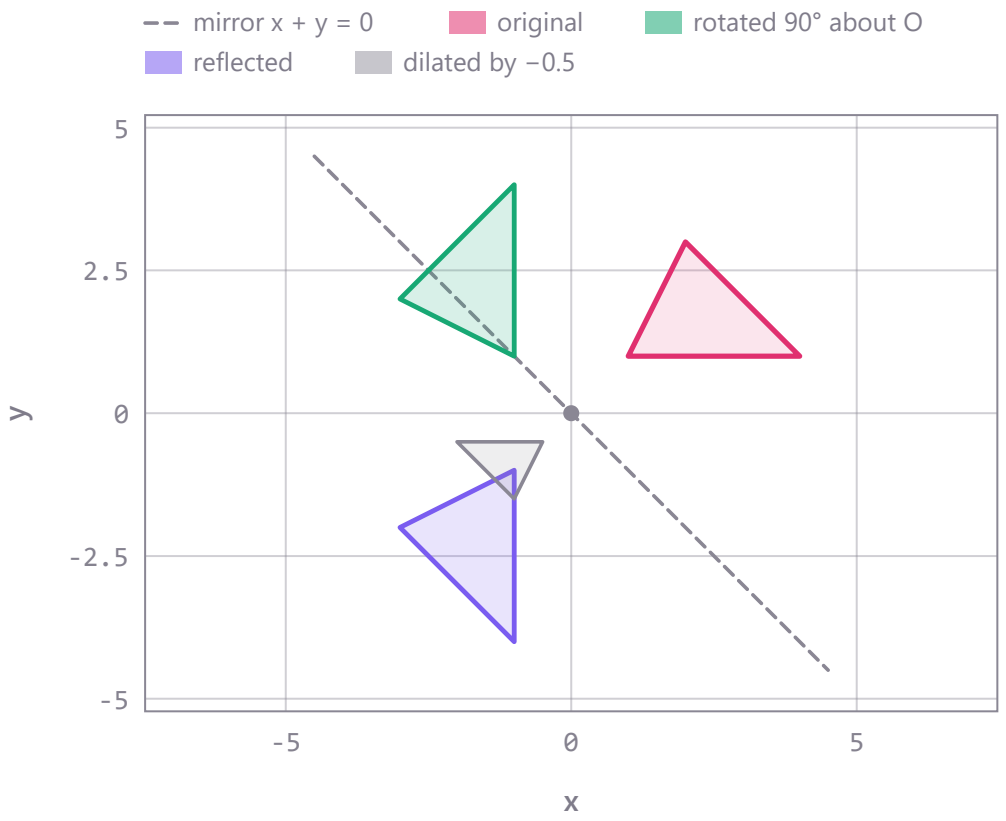
The image of a point rotated by an angle about a centre (the origin unless given); counterclockwise is positive.

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} x - c_x \\ y - c_y \end{pmatrix} + \begin{pmatrix} c_x \\ c_y \end{pmatrix}$$

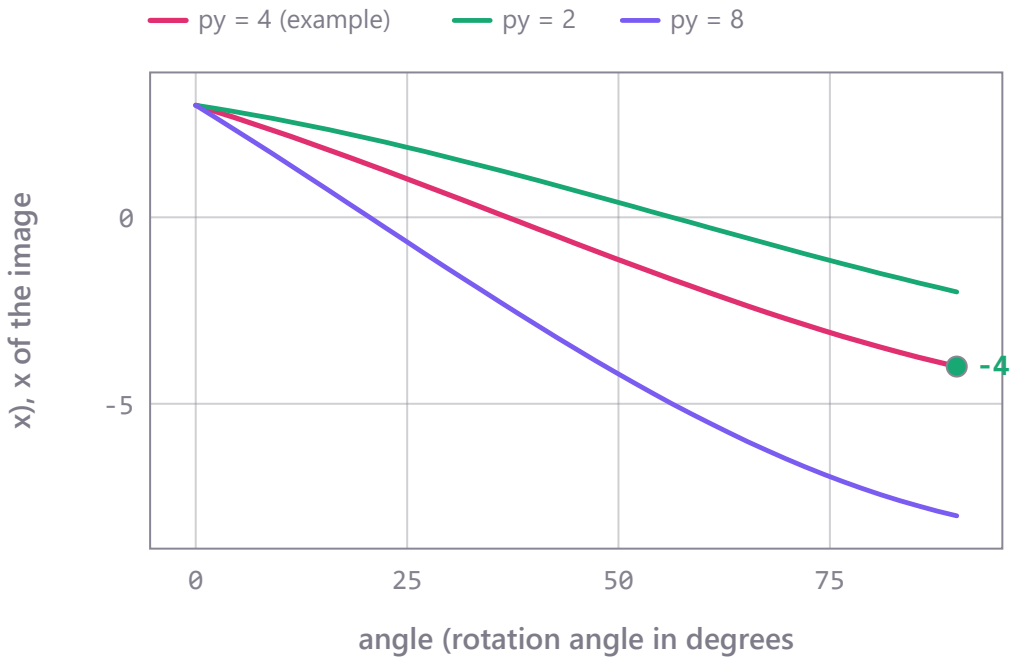
input	meaning	condition
`px`	x of the point	
`py`	y of the point	
`angle`	rotation angle in degrees, counterclockwise positive	
`cx`	x of the centre (default 0)	
`cy`	y of the centre (default 0)	

output	meaning
`x`	x of the image
`y`	y of the image

- **Method:** x' = cx + (x - cx) cos t - (y - cy) sin t, y' = cy + (x - cx) sin t + (y - cy) cos t (exact for multiples of 90 degrees)
- **Assumes:** angle in degrees, counterclockwise positive ('clockwise' in a question makes it negative); centre defaults to the origin
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The triangle (1, 1), (4, 1), (2, 3) and three images, each vertex mapped by the package: rotated 90° about the origin ((-1, 1), (-1, 4), (-3, 2)), reflected in the line x + y = 0 ((-1, -1), (-1, -4), (-3, -2)) and dilated about the origin by −0.5 ((-0.5, -0.5), (-2, -0.5), (-1, -1.5)).



x (x of the image) against angle (rotation angle in degrees, counterclockwise positive) from 0 to 90. One curve for each value of py (y of the point): 4, 2, 8. Other inputs as in the worked example: px = 3. 41 points per curve computed by geometry-ai999x; marked: the worked example, angle = 90, x = -4.

```
geometry_ai999x.solve('rotate_point', px=3, py=4, angle=90)
# -> x = -4, y = 3   (SOLVED)
```

```
geometry_ai999x.solve('rotate_point', px=5, py=1, angle=-30, cx=1, cy=1)
# -> x = 4.4641, y = -1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rotate the point (3, 4) by 90 degrees about the origin.	SOLVED	x = -4, y = 3
Rotate (5, 1) by 30 degrees clockwise about (1, 1).	SOLVED	x = 4.4641, y = -1
Rotate the point (2, 0) by 180 degrees.	SOLVED	x = -2, y = 0
Rotate the point (2, 0) about the origin.	NEEDS_INPUT	rotate_point needs angle (rotation angle in degrees, counterclockwise positive). Example: solve('rotate_point', px=3, py

Dilation (enlargement) of a point, `dilate_point`

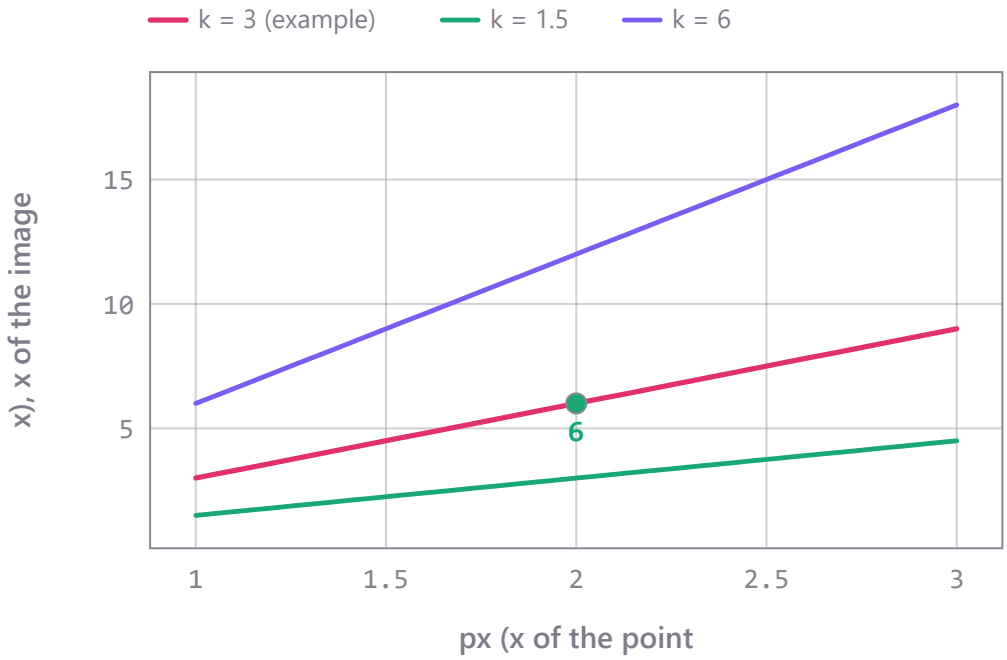
The image of a point under a dilation with scale factor k about a centre (the origin unless given).

$$P' = C + k\,(P - C)$$

input	meaning	condition
`px`	x of the point	
`py`	y of the point	
`k`	scale factor (negative: through the centre)	
`cx`	x of the centre (default 0)	
`cy`	y of the centre (default 0)	

output	meaning
`x`	x of the image
`y`	y of the image

- **Method:** P' = C + k (P - C)
- **Assumes:** $k \neq 0$; centre defaults to the origin
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



x (x of the image) against px (x of the point) from 1 to 3. One curve for each value of k (scale factor (negative: through the centre)): 3, 1.5, 6. Other inputs as in the worked example: py = 3. 31 points per curve computed by geometry-ai999x; marked: the worked example, px = 2, x = 6.

```
geometry_ai999x.solve('dilate_point', px=2, py=3, k=3)
# -> x = 6, y = 9   (SOLVED)
```

```
geometry_ai999x.solve('dilate_point', px=6, py=4, k=0.5, cx=2, cy=2)
# -> x = 4, y = 3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Dilate the point (2, 3) by a scale factor of 3 about the origin.	SOLVED	x = 6, y = 9
Find the image of (6, 4) under an enlargement with scale factor 1/2 and centre (2, 2).	SOLVED	x = 4, y = 3
Dilate the point (2, 3) by a scale factor of 0.	INVALID_INPUT	Dilation (enlargement) of a point: the scale factor must not be 0.

Intersection of a circle and a line, `circle_line_intersection`

The points where the line a x + b $y + c = 0$ meets the circle with centre (cx, cy) and radius r (0, 1 or 2 points).

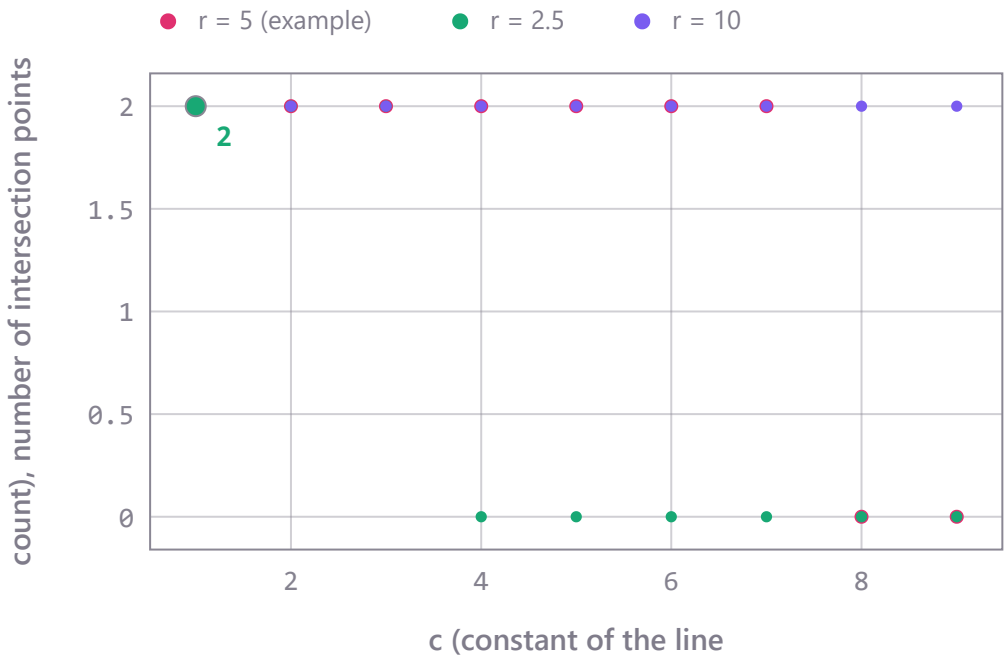
$$d = \frac{|ac_x + bc_y + c|}{\sqrt{a^2 + b^2}}, \quad P = F \pm \sqrt{r^2 - d^2}\,\frac{(-b, a)}{\sqrt{a^2 + b^2}}$$

input	meaning	condition
`cx`	x of the centre	
`cy`	y of the centre	
`r`	radius	

input	meaning	condition
`a`	coefficient of x of the line	
`b`	coefficient of y of the line	
`c`	constant of the line	

output	meaning
`count`	number of intersection points (0, 1 or 2)
`x1`	x of the first point (ordered by x, then y)
`y1`	y of the first point
`x2`	x of the second point
`y2`	y of the second point

- **Method:** foot of the perpendicular from the centre, then the half-chord sqrt(r^2 - d^2) along the line
- **Assumes:** $r > 0$; a and b not both 0; tangency is decided exactly on the written values; points ordered by x, then y; absent points are empty
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



count (number of intersection points (0, 1 or 2)) against c (constant of the line) from 1 to 9. One curve for each value of r (radius): 5, 2.5, 10. Other inputs as in the worked example: cx = 0, cy = 0, a = 1, b = -1. 9 points per curve computed by geometry-ai999x; marked: the worked example, c = 1, count = 2.

```
geometry_ai999x.solve('circle_line_intersection', cx=0, cy=0, r=5, a=1, b=-1, c=1)
# -> count = 2, x1 = -4, y1 = -3, x2 = 3, y2 = 4   (SOLVED)
```

```
geometry_ai999x.solve('circle_line_intersection', cx=0, cy=0, r=5, a=3, b=4, c=-25)
# -> count = 1, x1 = 3, y1 = 4, x2 = none, y2 = none   (SOLVED)
```

```
geometry_ai999x.solve('circle_line_intersection', cx=0, cy=0, r=1, a=0, b=1, c=-2)
# -> count = 0, x1 = none, y1 = none, x2 = none, y2 = none   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Where does the line y = x + 1 intersect the circle x^2 + y^2 = 25?	SOLVED	count = 2, x1 = -4, y1 = -3, x2 = 3, y2 = 4
Find the intersection of the line 3x + 4y - 25 = 0 and the circle with center (0, 0) and radius 5.	SOLVED	count = 1, x1 = 3, y1 = 4, x2 = none, y2 = none
Does the line y = 2 meet the circle (x - 1)^2 + (y + 1)^2 = 4?	SOLVED	count = 0, x1 = none, y1 = none, x2 = none, y2 = none
Where does the line y = x + 1 intersect the circle x^2 + y^2 = -4?	OUT_OF_SCOPE	Out of scope: the equation has no real circle (its radius squared is not positive).

Intersection of two circles, `circle_circle_intersection`

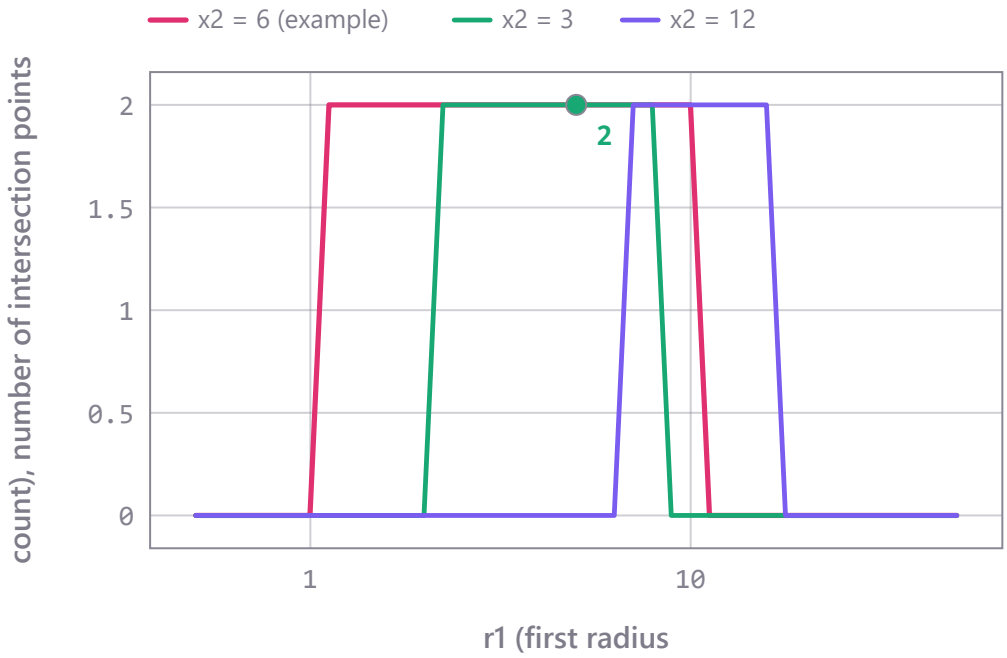
The points where two circles meet (0, 1 or 2 points).

$$a = \frac{d^2 + r_1^2 - r_2^2}{2d}, \quad h = \sqrt{r_1^2 - a^2}$$

input	meaning	condition
`x1`	x of the first centre	
`y1`	y of the first centre	
`r1`	first radius	
`x2`	x of the second centre	
`y2`	y of the second centre	
`r2`	second radius	

output	meaning
`count`	number of intersection points (0, 1 or 2)
`x1`	x of the first point (ordered by x, then y)
`y1`	y of the first point
`x2`	x of the second point
`y2`	y of the second point

- **Method:** radical line: $a = (d^2 + r_1^2 - r_2^2)/(2d)$ along the centre line, $h = sqrt$(r1^2 - a^2) across it
- **Assumes:** r1, $r_2 > 0$; the circles are not identical; tangency decided exactly on the written values; points ordered by x, then y
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



count (number of intersection points (0, 1 or 2)) against r1 (first radius) from 0.5 to 50 on a logarithmic axis. One curve for each value of x2 (x of the second centre): 6, 3, 12. Other inputs as in the worked example: x1 = 0, y1 = 0, y2 = 0, r2 = 5. 41 points per curve computed by geometry-ai999x; marked: the worked example, r1 = 5, count = 2.

```
geometry_ai999x.solve('circle_circle_intersection', x1=0, y1=0, r1=5, x2=6, y2=0, r2=5)
# -> count = 2, x1 = 3, y1 = -4, x2 = 3, y2 = 4   (SOLVED)
```

```
geometry_ai999x.solve('circle_circle_intersection', x1=0, y1=0, r1=5, x2=8, y2=0, r2=3)
# -> count = 1, x1 = 5, y1 = 0, x2 = none, y2 = none   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the intersection points of the circle with center (0, 0) and radius 5 and the circle with center (6, 0) and radius 5.	SOLVED	count = 2, x1 = 3, y1 = -4, x2 = 3, y2 = 4
Where do the circles x^2 + y^2 = 25 and (x - 8)^2 + y^2 = 9 meet?	SOLVED	count = 1, x1 = 5, y1 = 0, x2 = none, y2 = none
Find the intersection of the circle with center (0, 0) and radius 2 and the circle with center (0, 0) and radius 2.	INVALID_INPUT	Intersection of two circles: the circles coincide: they share infinitely many points.

Tangents from a point to a circle, `tangent_from_point`

The tangent length from an external point to a circle and the two points of tangency.

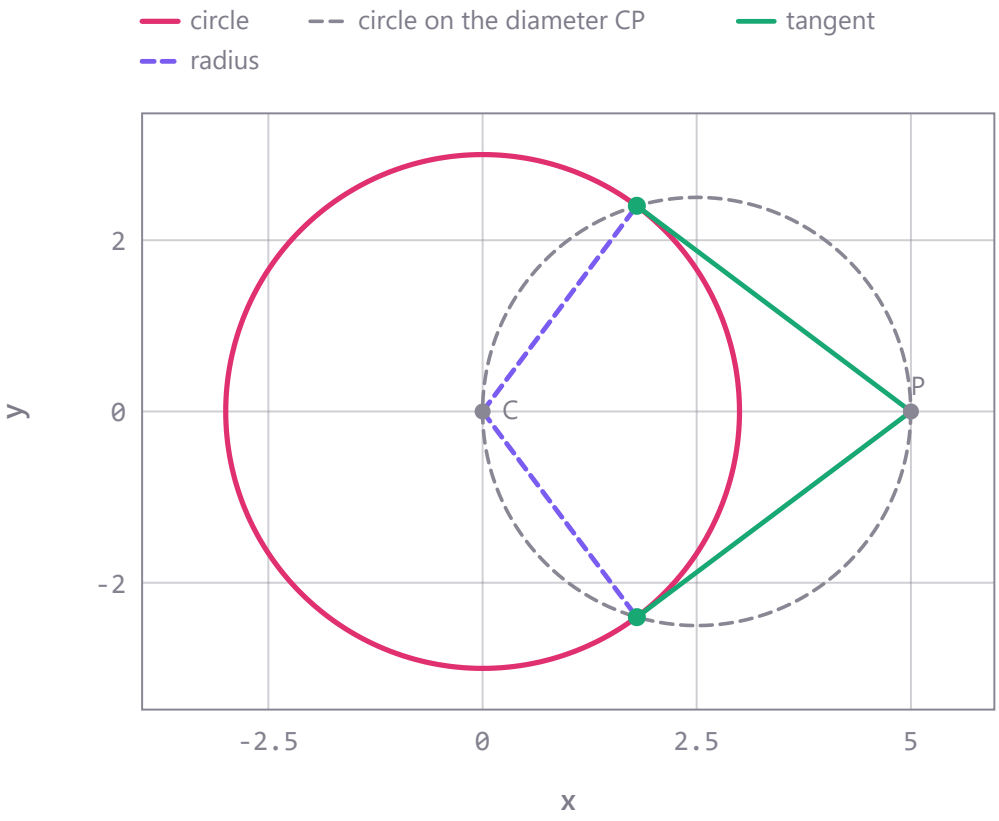
$$L = \sqrt{|P - C|^2 - r^2}, \quad T = C + \frac{r^2}{d^2}(P - C) \pm \frac{rL}{d^2}(P - C)^{\perp}$$

input	meaning	condition
`cx`	x of the centre	
`cy`	y of the centre	
`r`	radius	

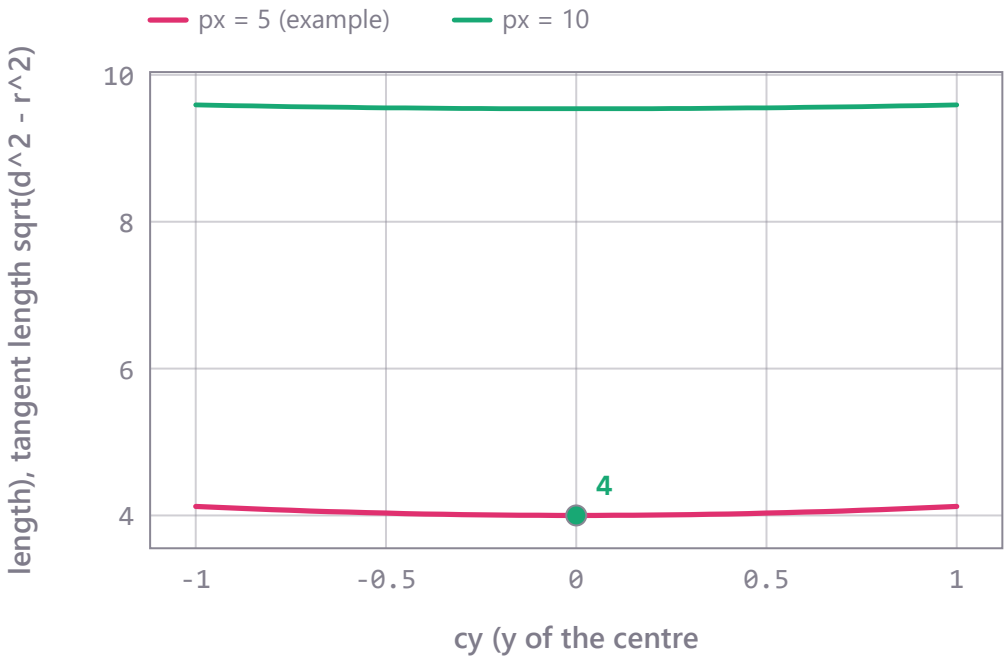
input	meaning	condition
`px`	x of the external point	
`py`	y of the external point	

output	meaning
`length`	tangent length sqrt(d^2 - r^2)
`x1`	x of the first tangency point (ordered by x, then y)
`y1`	y of the first tangency point
`x2`	x of the second tangency point
`y2`	y of the second tangency point

- **Method:** $L = \sqrt{d^2 - r^2}$; $T = C + (r^2/d^2)$(P - C) +- (r L/d^2) perp(P - C)
- **Assumes:** $r > 0$; the point is not inside the circle (on the circle: length 0, both points at the point)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Tangents from P = (5, 0) to the circle of centre C = (0, 0) and radius 3: the package gives the touching points (1.8, -2.4) and (1.8, 2.4) and the tangent length 4. The circle on the diameter CP meets the circle at the same two points (its intersection, also computed by the package: (1.8, -2.4) and (1.8, 2.4)).



length (tangent length sqrt(d^2 - r^2)) against cy (y of the centre) from -1 to 1. One curve for each value of px (x of the external point): 5, 10. Other inputs as in the worked example: cx = 0, r = 3, py = 0. 41 points per curve computed by geometry-ai999x; marked: the worked example, cy = 0, length = 4.

```
geometry_ai999x.solve('tangent_from_point', cx=0, cy=0, r=3, px=5, py=0)
# -> length = 4, x1 = 1.8, y1 = -2.4, x2 = 1.8, y2 = 2.4   (SOLVED)
```

```
geometry_ai999x.solve('tangent_from_point', cx=0, cy=0, r=6, px=0, py=10)
# -> length = 8, x1 = -4.8, y1 = 3.6, x2 = 4.8, y2 = 3.6   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the length of the tangent from the point (5, 0) to the circle with center (0, 0) and radius 3.	SOLVED	length = 4, x1 = 1.8, y1 = -2.4, x2 = 1.8, y2 = 2.4
Find the points of tangency of the tangents from (0, 10) to the circle x^2 + y^2 = 36.	SOLVED	length = 8, x1 = -4.8, y1 = 3.6, x2 = 4.8, y2 = 3.6
Find the length of the tangent from the point (1, 0) to the circle with center (0, 0) and radius 3.	INVALID_INPUT	Tangents from a point to a circle: the point is inside the circle: no tangent passes through it.

Classify a conic section from its general equation, `conic_classify`

Whether A x^2 + B xy + C y^2 + D x + E y + F = 0 is a circle, ellipse, parabola, hyperbola, a degenerate conic or has no real points.

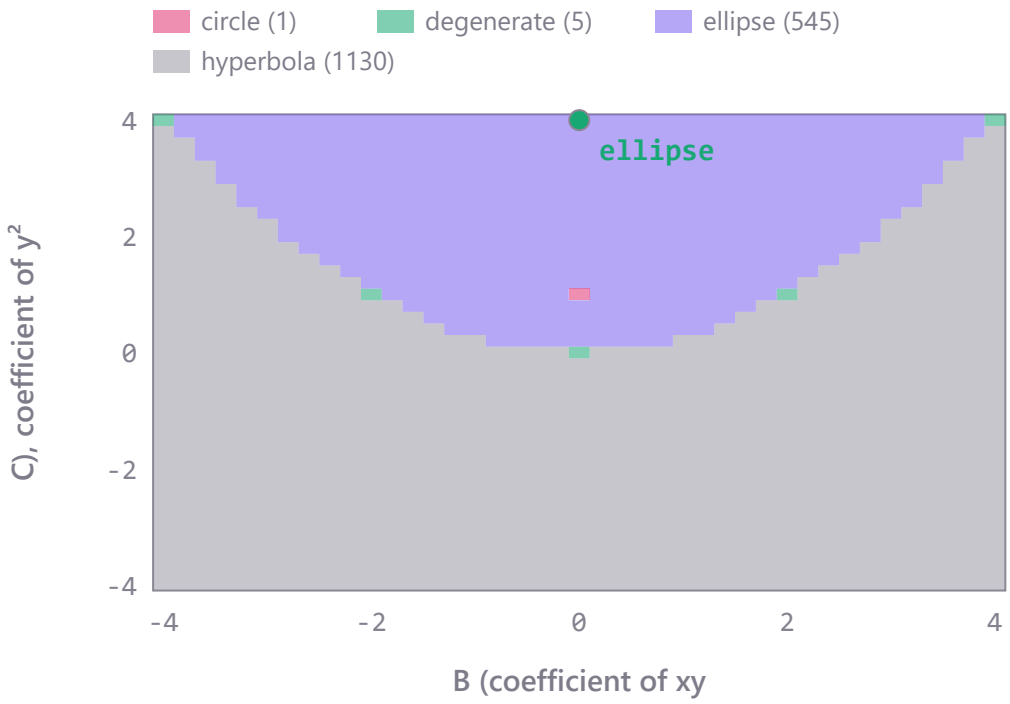
$$\Delta = \det \begin{pmatrix} A & B/2 & D/2 \\ B/2 & C & E/2 \\ D/2 & E/2 & F \end{pmatrix}, \quad B^2 - 4AC \lessgtr 0$$

input	meaning	condition
`A`	coefficient of x^2	

input	meaning	condition
`B`	coefficient of xy	
`C`	coefficient of y^2	
`D`	coefficient of x	
`E`	coefficient of y	
`F`	constant term	

output	meaning
`type`	circle / ellipse / parabola / hyperbola / degenerate / empty

- **Method:** discriminant $B^2 - 4AC$ and the 3x3 determinant of the conic's matrix, exact in fractions
- **Assumes:** A, B, C not all 0; exact on the written coefficients; 'degenerate' = a point, a pair of lines or a degenerate equation (zero 3x3 determinant); 'empty' = an ellipse with no real points
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Type of the conic A x^2 + B xy + C y^2 + D x + E y + F = 0 over B and C from −4 to 4, with A = 1, D = 0, E = 0, F = -4 as in the worked example. Each of the 41 × 41 cells is one run of the package, coloured by the type it returned: circle: 1 cells, degenerate: 5 cells, ellipse: 545 cells, hyperbola: 1130 cells. Marked: the worked example B = 0, C = 4 → ellipse.

```
geometry_ai999x.solve('conic_classify', A=1, B=0, C=4, D=0, E=0, F=-4)
# -> type = ellipse   (SOLVED)
```

```
geometry_ai999x.solve('conic_classify', A=1, B=0, C=-1, D=2, E=0, F=-3)
# -> type = hyperbola   (SOLVED)
```

```
geometry_ai999x.solve('conic_classify', A=1, B=0, C=0, D=0, E=-1, F=0)
# -> type = parabola   (SOLVED)
```

```
geometry_ai999x.solve('conic_classify', A=1, B=0, C=1, D=0, E=0, F=1)
# -> type = empty   (SOLVED)
```

```
geometry_ai999x.solve('conic_classify', A=1, B=0, C=-1, D=0, E=0, F=0)
# -> type = degenerate   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Classify the conic x^2 + 4y^2 - 4 = 0.	SOLVED	type = ellipse
What type of conic is x^2 - y^2 + 2x - 3 = 0?	SOLVED	type = hyperbola
Identify the conic 2x^2 + 2y^2 - 4x + 8y - 1 = 0.	SOLVED	type = circle
Classify the conic x^2 + 2xy + y^2 + x = 0.	SOLVED	type = parabola
What kind of conic is x^2 + y^2 + 1 = 0?	SOLVED	type = empty

Point in polygon test, `point_in_polygon`

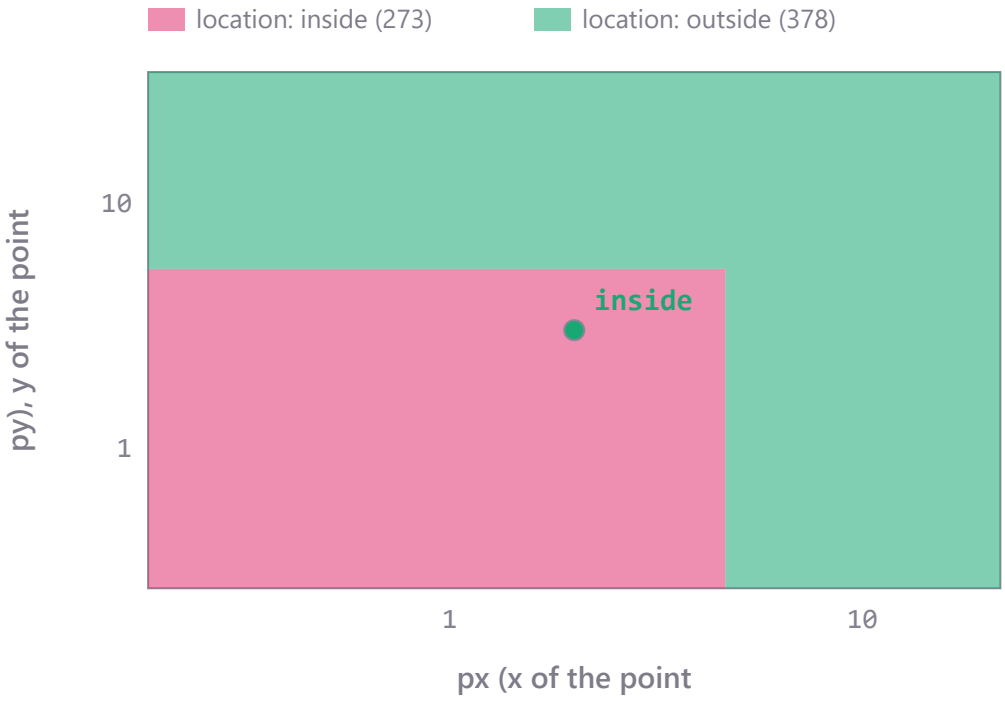
Whether a point lies inside, outside or on the boundary of a simple polygon.

$$\text{inside} \iff \#\{\text{edges crossed by the ray } y = p_y, x > p_x\} \text{ is odd}$$

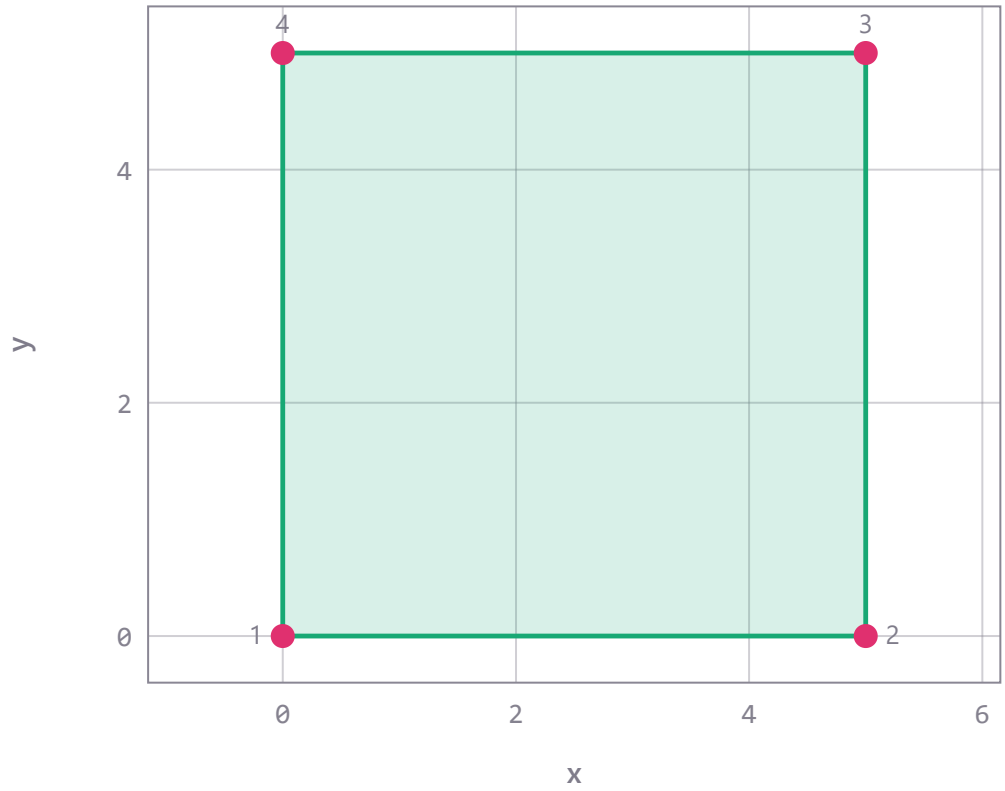
input	meaning	condition
`px`	x of the point	
`py`	y of the point	
`points`	polygon vertices in order: x1 y1, x2 y2, ...	

output	meaning
`location`	inside / outside / boundary

- **Method:** exact boundary test, then the even-odd ray-crossing rule with exact rational arithmetic
- **Assumes:** simple polygon with at least 3 vertices; decided exactly on the written coordinates
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



inside / outside / boundary (location) over px (x of the point) from 0.2 to 20 and py (y of the point) from 0.3 to 30 (logarithmic axes): each of the 31 × 21 cells is one run of geometry-ai999x, coloured by the verdict it returned, inside: 273 cells, outside: 378 cells. Other inputs as in the worked example: points = 0 0, 5 0, 5 5, 0 5. Marked: the worked example, px = 2, py = 3 → inside.



The rows of points (polygon vertices in order: x1 y1, x2 y2, ...) drawn as points (numbered in order), joined in order as a polygon. Other inputs: px = 2, py = 3.

```
geometry_ai999x.solve('point_in_polygon', px=2, py=3, points='0 0, 5 0, 5 5, 0 5')
# -> location = inside   (SOLVED)
```

```
geometry_ai999x.solve('point_in_polygon', px=3, py=3, points='0 0, 4 0, 0 4')
# -> location = outside   (SOLVED)
```

```
geometry_ai999x.solve('point_in_polygon', px=2, py=2, points='0 0, 4 0, 0 4')
# -> location = boundary   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Is the point (2, 3) inside the polygon with vertices (0, 0), (5, 0), (5, 5), (0, 5)?	SOLVED	location = inside
Does the triangle with vertices (0, 0), (4, 0), (0, 4) contain the point (3, 3)?	SOLVED	location = outside
Is the point (2, 2) inside the polygon with vertices (0, 0), (4, 0), (4, 4), (2, 1.5), (0, 4)?	SOLVED	location = outside
Is the point (2, 2) inside the polygon with vertices (0, 0), (4, 4), (4, 0), (0, 4)?	INVALID_INPUT	Point in polygon test: points: the polygon is not simple (edges 1 and 3 meet).

Convex hull of points in the plane, `convex_hull`

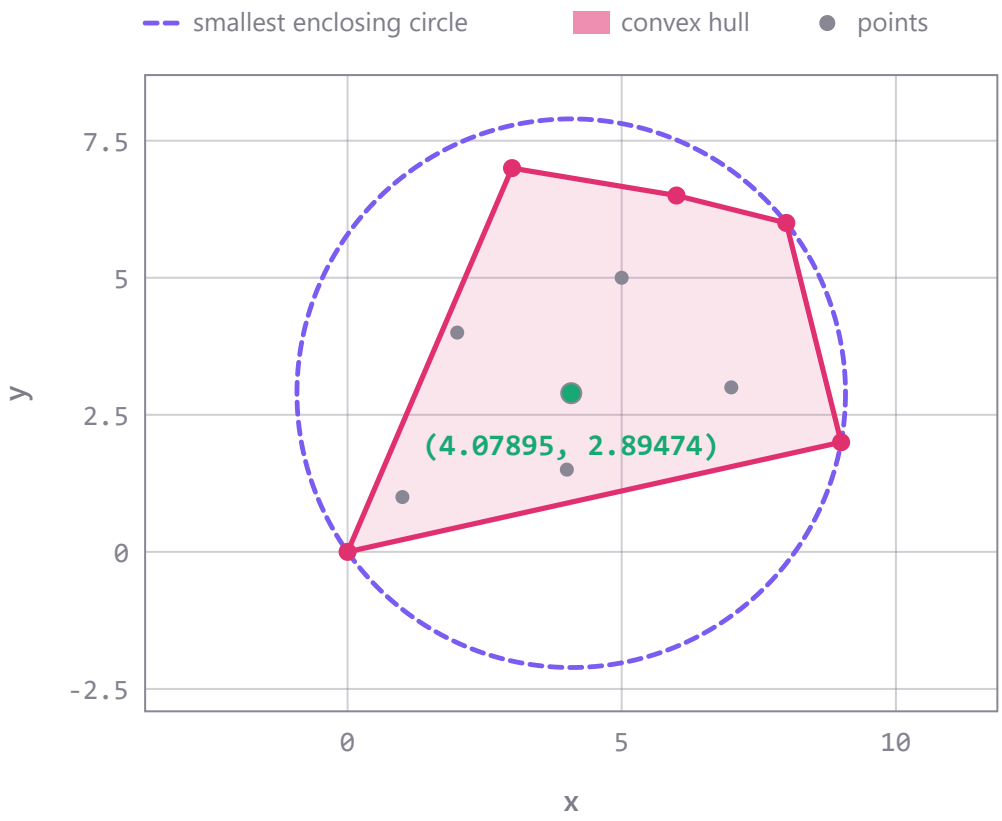
The convex hull of a set of points: its vertices (counterclockwise), their number, its area and perimeter.

$$\mathrm{conv}(S) = \{\sum \lambda_i p_i : \lambda_i \geq 0, \sum \lambda_i = 1\}$$

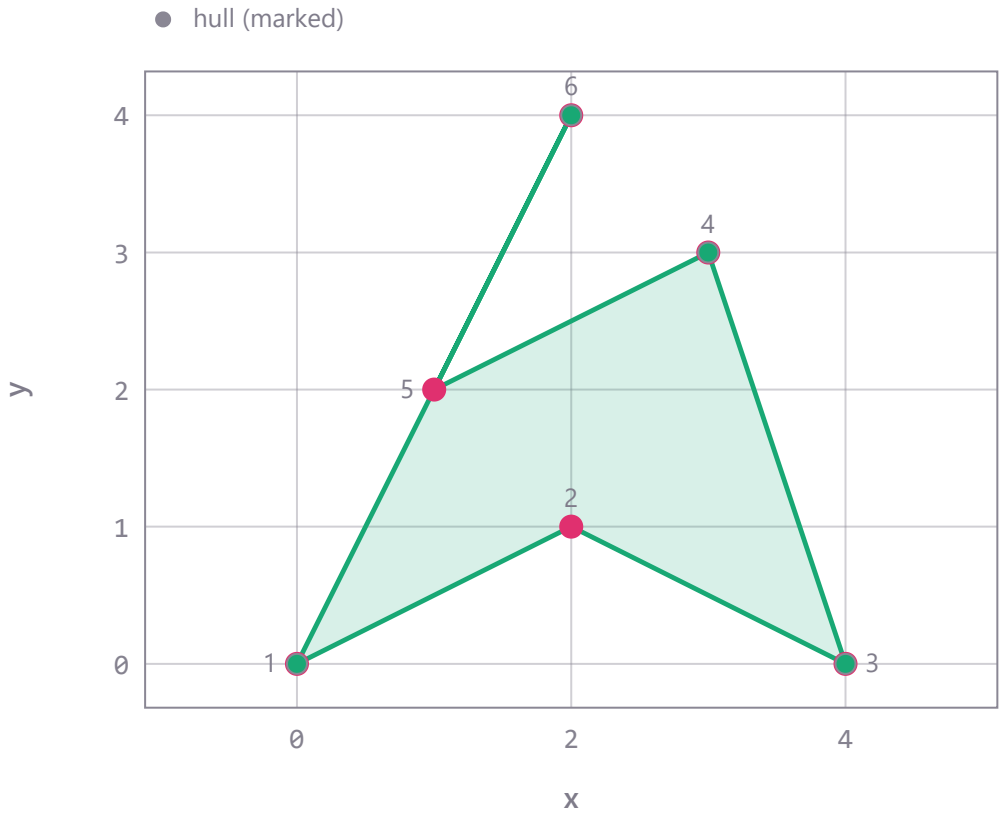
input	meaning	condition
`points`	the points: x1 y1, x2 y2, ...	

output	meaning
`hull`	hull vertices [x, y], counterclockwise from the smallest x (then smallest y)
`n_vertices`	number of hull vertices
`area`	area of the hull
`perimeter`	perimeter of the hull

- **Method:** Andrew's monotone chain with exact (rational) orientation tests; shoelace area
- **Assumes:** at least 3 points, not all collinear, at most 5000; points on a hull edge between two vertices are not vertices; repeated points count once
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Ten points (0 0, 5 5, 1 1, 9 2, 6 6.5, 3 7, 7 3, 2 4, 8 6, 4 1.5), their convex hull as returned by the package (5 vertices, area 38.25, perimeter 26.0614), and the smallest circle enclosing them; marked: its centre x = 4.07895, y = 2.89474, radius 5.00173.



The rows of points (the points: x1 y1, x2 y2, ...) drawn as points (numbered in order), joined in order as a polygon, returned (x, y) pairs marked; marked: the returned hull (0, 0), (4, 0), (3, 3), (2, 4). Returned: n_vertices = 4, area = 9, perimeter = 13.0486.

```
geometry_ai999x.solve('convex_hull', points='0 0, 2 1, 4 0, 3 3, 1 2, 2 4')
# -> hull = [[0.0, 0.0], [4.0, 0.0], [3.0, 3.0], [2.0, 4.0]], n_vertices = 4, area = 9, perimeter =
13.0486   (SOLVED)
```

```
geometry_ai999x.solve('convex_hull', points='0 0, 1 1, 2 0, 2 2, 0 2, 1 0')
# -> hull = [[0.0, 0.0], [2.0, 0.0], [2.0, 2.0], [0.0, 2.0]], n_vertices = 4, area = 4, perimeter =
8   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the convex hull of the points (0, 0), (2, 1), (4, 0), (3, 3), (1, 2), (2, 4).	SOLVED	hull = [[0.0, 0.0], [4.0, 0.0], [3.0, 3.0], [2.0, 4.0]], n_vertices = 4, area = 9, perimeter = 13.0486
What is the area of the convex hull of (0, 0), (1, 1), (2, 0), (2, 2), (0, 2)?	SOLVED	hull = [[0.0, 0.0], [2.0, 0.0], [2.0, 2.0], [0.0, 2.0]], n_vertices = 4, area = 4, perimeter = 8
Find the convex hull of the points (0, 0), (1, 1), (2, 2).	INVALID_INPUT	Convex hull of points in the plane: all points are collinear (or fewer than 3 distinct): the hull has no area.

Closest pair of points, `closest_pair`

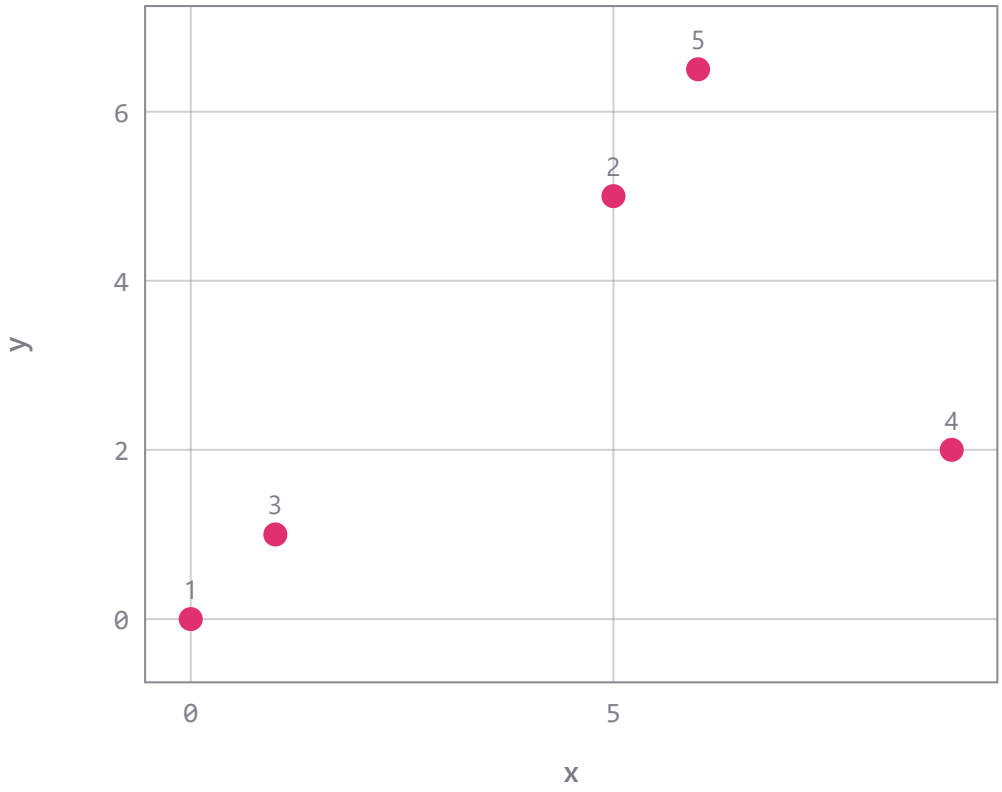
The smallest distance between any two of a set of points in the plane.

$$\delta = \min_{i<j} \|p_i - p_j\|$$

input	meaning	condition
`points`	the points: x1 y1, x2 y2, ...	

output	meaning
`distance`	smallest pairwise distance

- **Method:** sort by x and sweep, stopping each scan once the x-gap exceeds the best distance
- **Assumes:** at least 2 points, at most 2000; a repeated point gives distance 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The rows of points (the points: x1 y1, x2 y2, ...) drawn as points (numbered in order). Returned: distance = 1.41421.

```
geometry_ai999x.solve('closest_pair', points='0 0, 5 5, 1 1, 9 2, 6 6.5')
# -> distance = 1.41421   (SOLVED)
```

```
geometry_ai999x.solve('closest_pair', points='0 0, 3 4, 10 0')
# -> distance = 5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the closest pair distance among the points (0, 0), (5, 5), (1, 1), (9, 2), (6, 6.5).	SOLVED	distance = 1.41421
What is the smallest distance between any two of the points (0, 0), (3, 4), (10, 0)?	SOLVED	distance = 5

Farthest pair of points (diameter of a point set), `farthest_pair`

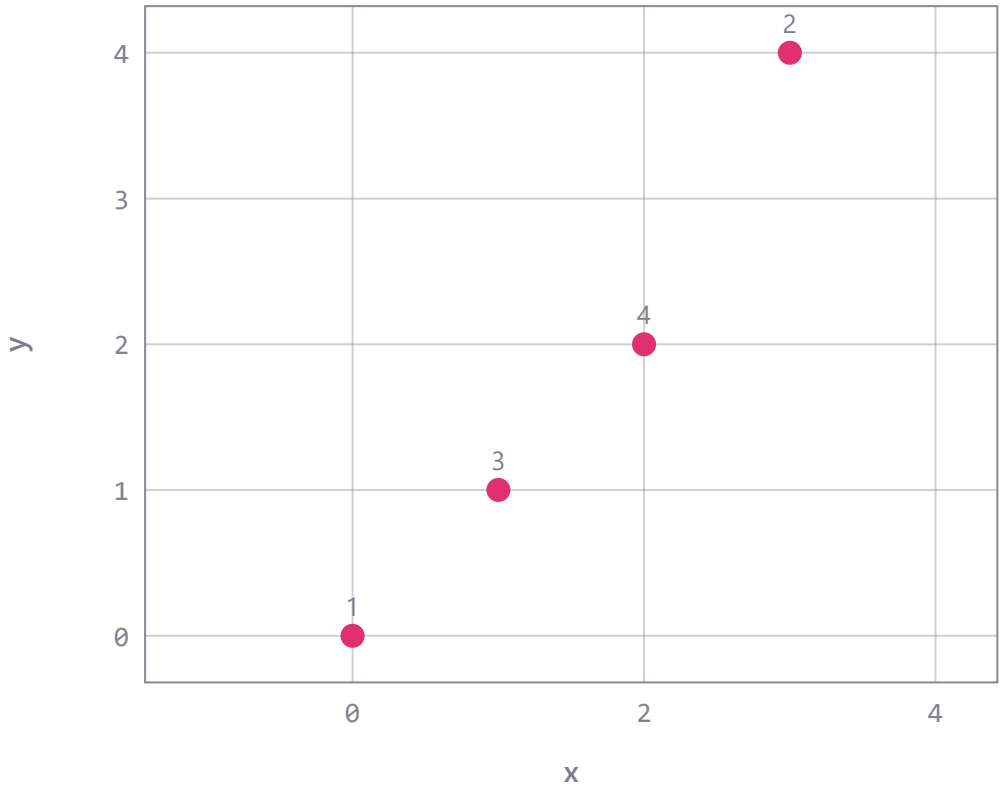
The largest distance between any two of a set of points in the plane.

$$D = \max_{i<j} \|p_i - p_j\|$$

input	meaning	condition
`points`	the points: x1 y1, x2 y2, ...	

output	meaning
`distance`	largest pairwise distance

- **Method:** the farthest pair are vertices of the convex hull: hull first (monotone chain), then all hull-vertex pairs
- **Assumes:** at least 2 points, at most 5000
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The rows of points (the points: x1 y1, x2 y2, ...) drawn as points (numbered in order). Returned: distance = 5.

```
geometry_ai999x.solve('farthest_pair', points='0 0, 3 4, 1 1, 2 2')
# -> distance = 5   (SOLVED)
```

```
geometry_ai999x.solve('farthest_pair', points='0 0, 6 8')
# -> distance = 10   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the farthest pair distance among the points (0, 0), (3, 4), (1, 1), (2, 2).	SOLVED	distance = 5
What is the largest distance between any two of the points (0, 0), (6, 8), (3, 3)?	SOLVED	distance = 10

Intersection of two line segments, `segment_intersection`

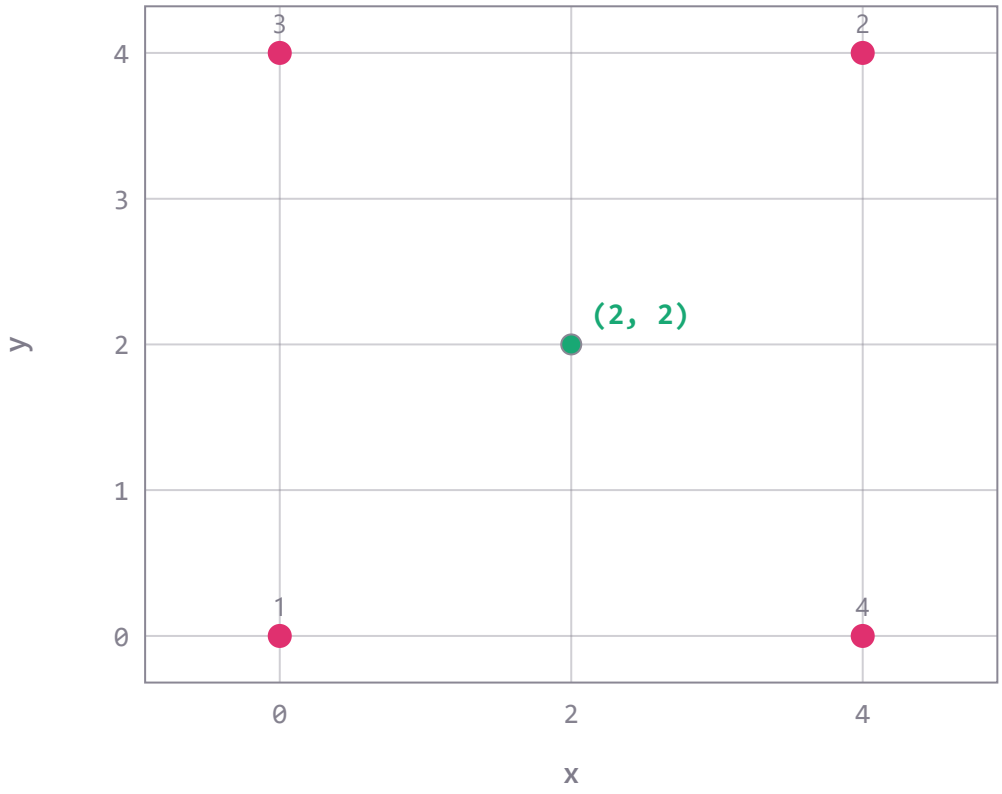
Whether segments AB and CD meet, and where: at one point, along an overlap, or not at all.

$$A + t(B - A) = C + u(D - C), \quad 0 \le t, u \le 1$$

input	meaning	condition
`points`	the endpoints A, B, C, D: xA yA, xB yB, xC yC, xD yD	

output	meaning
`status`	point / overlap / none
`x`	x of the common point (point only)
`y`	y of the common point (point only)

- **Method:** exact orientation tests, then the intersection parameter in rational arithmetic; collinear overlaps by interval
- **Assumes:** each segment has distinct endpoints; decided exactly on the written coordinates (touching counts as meeting)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The rows of points (the endpoints A, B, C, D: xA yA, xB yB, xC yC, xD yD) drawn as points (numbered in order), returned (x, y) pairs marked. Returned: x = 2, y = 2.

```
geometry_ai999x.solve('segment_intersection', points='0 0, 4 4, 0 4, 4 0')
# -> status = point, x = 2, y = 2   (SOLVED)
```

```
geometry_ai999x.solve('segment_intersection', points='0 0, 1 1, 2 0, 3 -1')
# -> status = none, x = none, y = none   (SOLVED)
```

```
geometry_ai999x.solve('segment_intersection', points='0 0, 2 0, 1 0, 3 0')
# -> status = overlap, x = none, y = none   (SOLVED)
```

```
geometry_ai999x.solve('segment_intersection', points='0 0, 2 0, 2 0, 2 5')
# -> status = point, x = 2, y = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Do the segments from (0, 0) to (4, 4) and from (0, 4) to (4, 0) intersect?	SOLVED	status = point, x = 2, y = 2
Do the segments from (0, 0) to (1, 1) and from (2, 0) to (3, -1) intersect?	SOLVED	status = none, x = none, y = none
Do the segments from (0, 0) to (0, 0) and from (2, 0) to (3, -1) intersect?	INVALID_INPUT	Intersection of two line segments: a segment has two equal endpoints.

Distance from a point to a segment, `point_segment_distance`

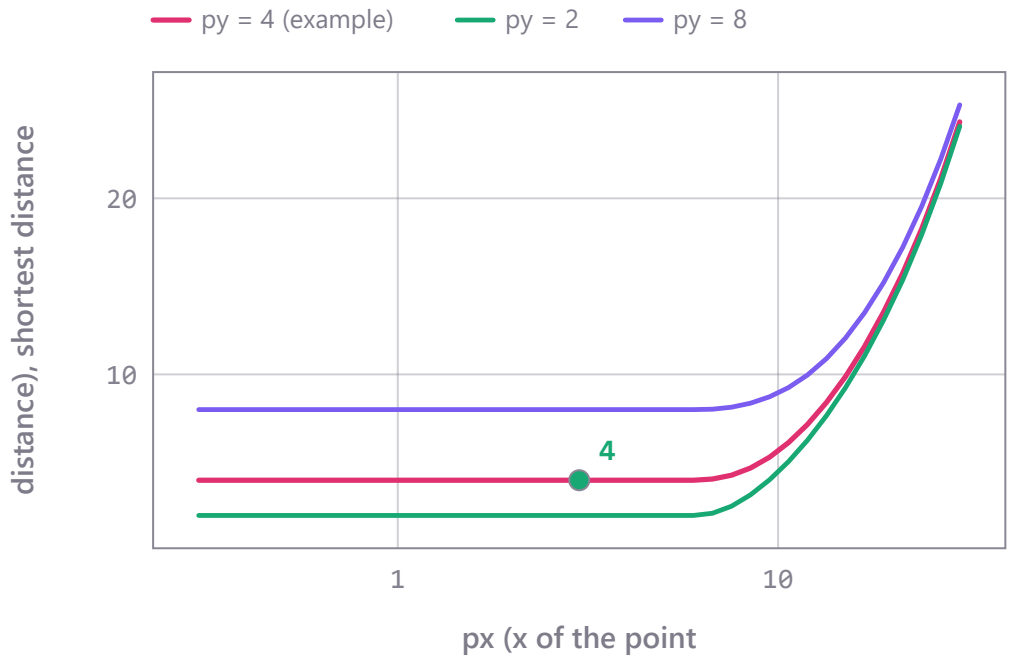
The shortest distance from a point to the segment AB and the closest point of the segment.

$$t^* = \operatorname{clamp}\left(\frac{(P-A)\cdot(B-A)}{\|B-A\|^2}, 0, 1\right), \quad d = \|P - (A + t^*(B-A))\|$$

input	meaning	condition
`px`	x of the point	
`py`	y of the point	
`x1`	x of A	
`y1`	y of A	
`x2`	x of B	
`y2`	y of B	

output	meaning
`distance`	shortest distance
`qx`	x of the closest point
`qy`	y of the closest point

- **Method:** project P onto AB, clamp the parameter to [0, 1]
- **Assumes:** $A \neq B$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



distance (shortest distance) against px (x of the point) from 0.3 to 30 on a logarithmic axis. One curve for each value of py (y of the point): 4, 2, 8. Other inputs as in the worked example: x1 = 0, y1 = 0, x2 = 6, y2 = 0. 41 points per curve computed by geometry-ai999x; marked: the worked example, px = 3, distance = 4.

```
geometry_ai999x.solve('point_segment_distance', px=3, py=4, x1=0, y1=0, x2=6, y2=0)
# -> distance = 4, qx = 3, qy = 0   (SOLVED)
```

```
geometry_ai999x.solve('point_segment_distance', px=8, py=3, x1=0, y1=0, x2=4, y2=0)
# -> distance = 5, qx = 4, qy = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the distance from the point (3, 4) to the segment from (0, 0) to (6, 0).	SOLVED	distance = 4, qx = 3, qy = 0
What is the distance from the point (8, 3) to the segment joining (0, 0) and (4, 0)?	SOLVED	distance = 5, qx = 4, qy = 0

Length of a polyline (path through points), `polyline_length`

The total length of the open path that visits the points in the order given.

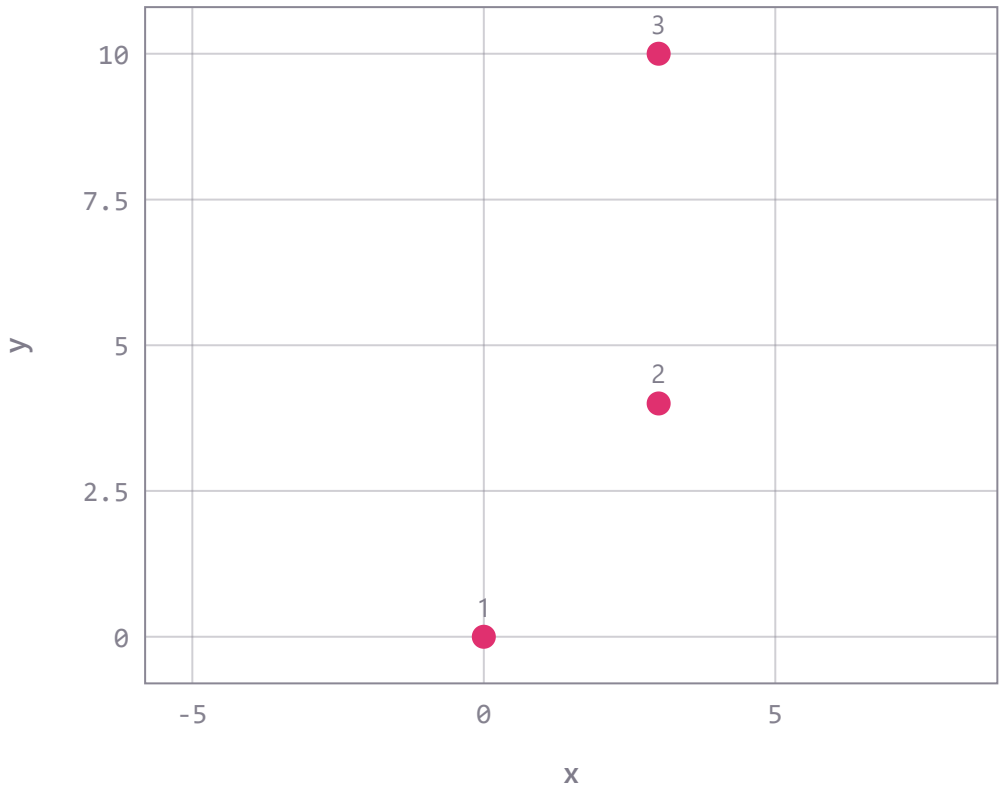
$$L = \sum_{i=1}^{n-1} \|p_{i+1} - p_i\|$$

input	meaning	condition
`points`	the points in order: x1 y1, x2 y2, ...	

output	meaning
`length`	sum of the segment lengths
`segments`	number of segments

- **Method:** sum of the distances between consecutive points

- **Assumes:** at least 2 points; the path is open (not closed back to the start)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The rows of points (the points in order: x1 y1, x2 y2, ...) drawn as points (numbered in order). Returned: length = 11, segments = 2.

```
geometry_ai999x.solve('polyline_length', points='0 0, 3 4, 3 10')
# -> length = 11, segments = 2   (SOLVED)
```

```
geometry_ai999x.solve('polyline_length', points=[0, 0, 1, 0, 1, 1, 0, 1])
# -> length = 3, segments = 3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the length of the path through (0, 0), (3, 4) and (3, 10).	SOLVED	length = 11, segments = 2
A route goes from (1, 1) to (4, 5) to (10, 13). How long is it?	SOLVED	length = 15, segments = 2

Is a polygon convex?, `polygon_is_convex`

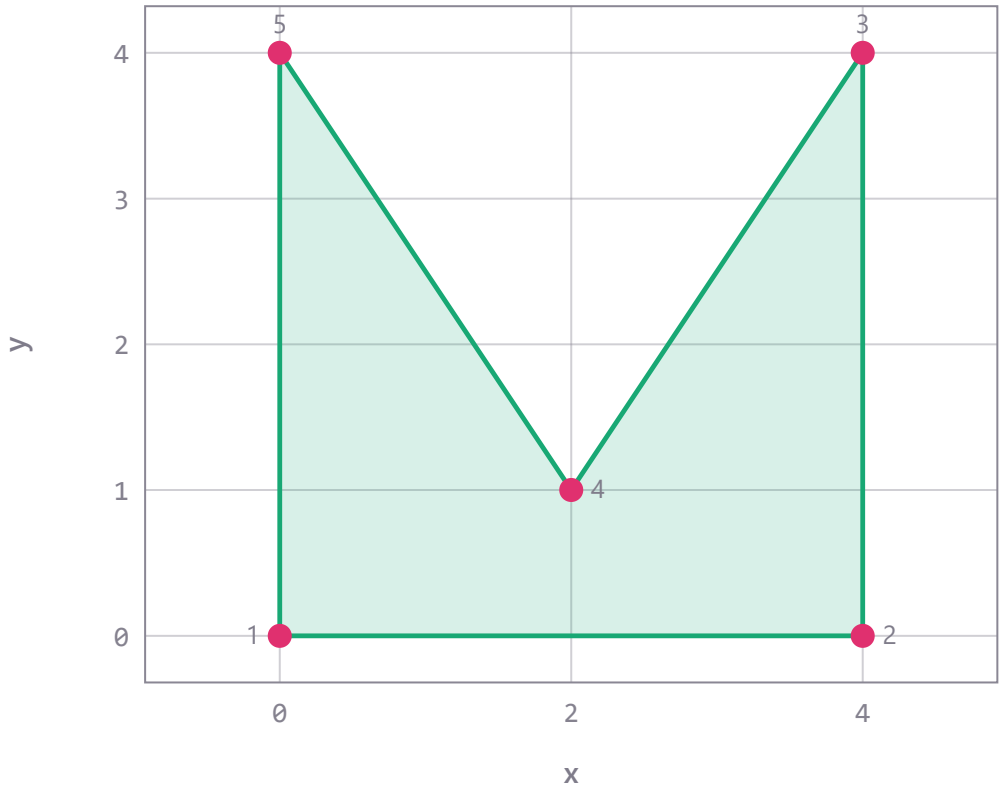
Whether a simple polygon given by its vertices is convex.

$$\text{sign}\left((p_{i+1}-p_i)\times(p_{i+2}-p_{i+1})\right)\ \text{constant}$$

input	meaning	condition
`points`	vertices in order: x1 y1, x2 y2, ...	

output	meaning
`convex`	yes / no

- **Method:** every turn (cross product of consecutive edges) has the same sign, computed exactly
- **Assumes:** simple polygon; a vertex on a straight edge (180 degrees) does not make it non-convex; exact on the written coordinates
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The rows of points (vertices in order: x1 y1, x2 y2, ...) drawn as points (numbered in order), joined in order as a polygon.

```
geometry_ai999x.solve('polygon_is_convex', points='0 0, 4 0, 4 4, 2 1, 0 4')
# -> convex = no   (SOLVED)
```

```
geometry_ai999x.solve('polygon_is_convex', points='0 0, 4 0, 4 4, 0 4')
# -> convex = yes   (SOLVED)
```

```
geometry_ai999x.solve('polygon_is_convex', points='0 0, 2 0, 4 0, 4 4, 0 4')
# -> convex = yes   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Is the polygon with vertices (0, 0), (4, 0), (4, 4), (2, 1), (0, 4) convex?	SOLVED	convex = no
Is the quadrilateral with vertices (0, 0), (4, 0), (4, 4), (0, 4) convex?	SOLVED	convex = yes

Pick's theorem: lattice points of a polygon, `pick_lattice_points`

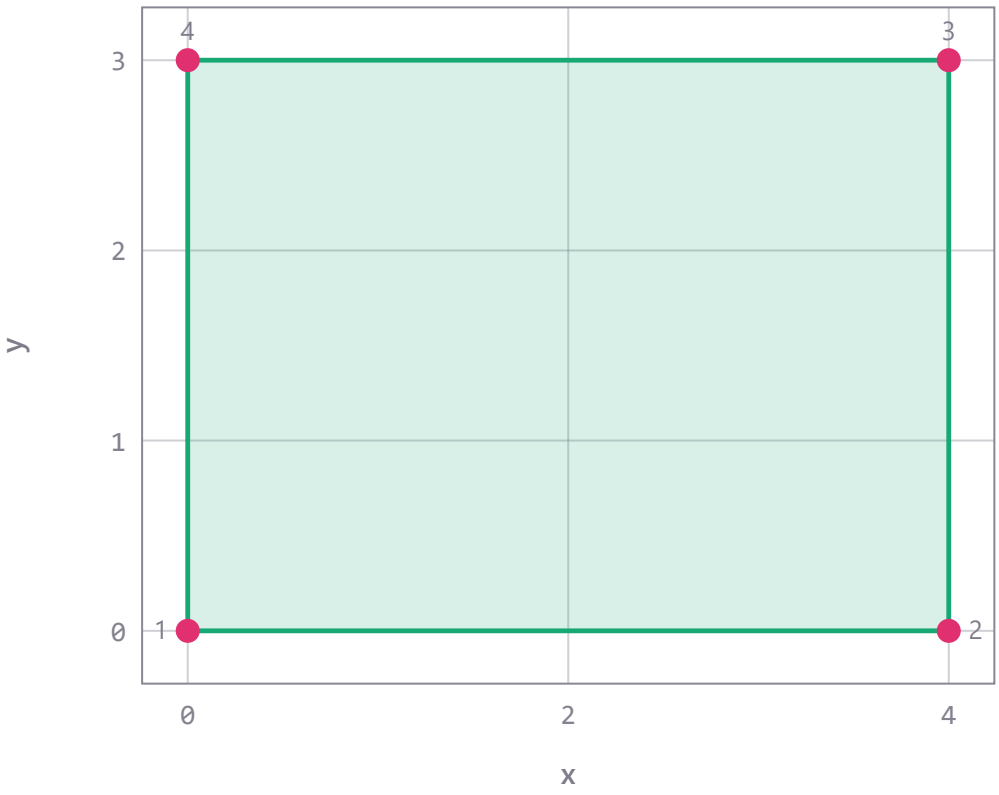
Area, boundary lattice points and interior lattice points of a simple polygon with integer vertices.

$$A = I + \frac{B}{2} - 1$$

input	meaning	condition
`points`	integer vertices in order: x1 y1, x2 y2, ...	

output	meaning
`area`	area
`boundary_points`	lattice points on the boundary
`interior_points`	lattice points strictly inside

- **Method:** B = sum of gcd(|dx|, |dy|) over the edges; shoelace area A; Pick: $I = A - B/2 + 1$
- **Assumes:** integer coordinates; simple polygon
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The rows of points (integer vertices in order: x1 y1, x2 y2, ...) drawn as points (numbered in order), joined in order as a polygon. Returned: area = 12, boundary_points = 14, interior_points = 6.

```
geometry_ai999x.solve('pick_lattice_points', points='0 0, 4 0, 4 3, 0 3')
# -> area = 12, boundary_points = 14, interior_points = 6   (SOLVED)
```

```
geometry_ai999x.solve('pick_lattice_points', points='0 0, 5 0, 0 5')
# -> area = 12.5, boundary_points = 15, interior_points = 6   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
How many lattice points are inside the polygon with vertices (0, 0), (4, 0), (4, 3), (0, 3)?	SOLVED	area = 12, boundary_points = 14, interior_points = 6
Use Pick's theorem for the triangle with vertices (0, 0), (5, 0), (0, 5).	SOLVED	area = 12.5, boundary_points = 15, interior_points = 6
Count the lattice points inside the polygon with vertices (0, 0), (2.5, 0), (0, 2).	INVALID_INPUT	Pick's theorem: lattice points of a polygon: Pick's theorem needs integer (lattice) coordinates.

Medians, altitudes and angle bisectors of a triangle, `triangle_cevians`

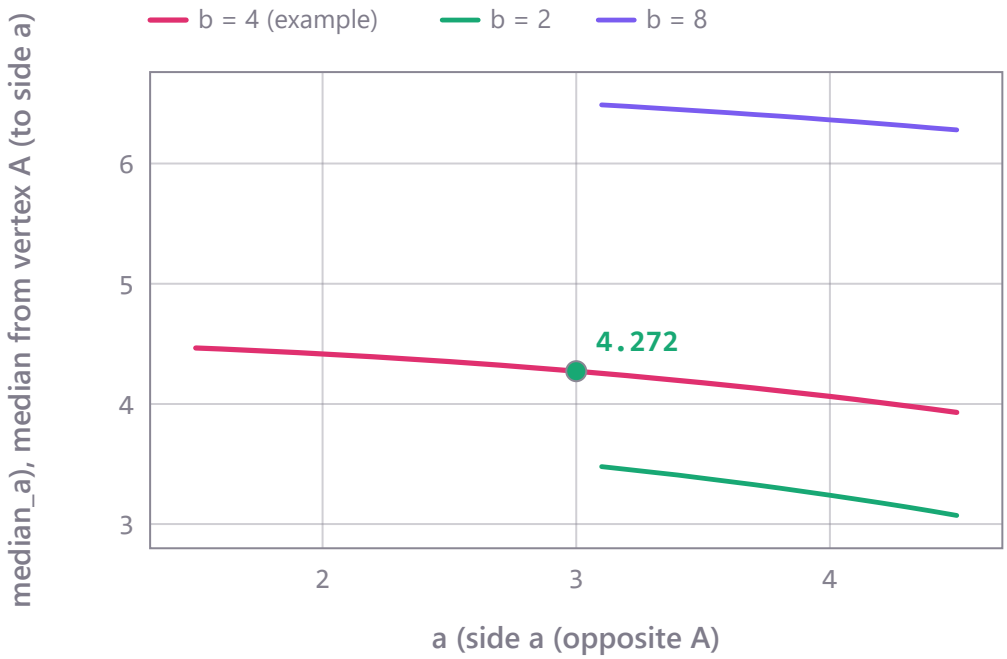
The lengths of the three medians, three altitudes and three internal angle bisectors of a triangle from its sides.

$$m_a = \tfrac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}, \quad h_a = \frac{2K}{a}, \quad t_a = \frac{\sqrt{bc\,[(b+c)^2 - a^2]}}{b+c}$$

input	meaning	condition
`a`	side a (opposite A)	
`b`	side b (opposite B)	
`c`	side c (opposite C)	

output	meaning
`median_a`	median from vertex A (to side a)
`median_b`	median from vertex B (to side b)
`median_c`	median from vertex C (to side c)
`altitude_a`	altitude from vertex A (to side a)
`altitude_b`	altitude from vertex B (to side b)
`altitude_c`	altitude from vertex C (to side c)
`bisector_a`	bisector from vertex A (to side a)
`bisector_b`	bisector from vertex B (to side b)
`bisector_c`	bisector from vertex C (to side c)

- **Method:** $m_a = sqrt$(2b^2 + 2c^2 - a^2)/2; $h_a = 2K/a$ (Heron); $t_a = sqrt$(bc((b + c)^2 - a^2))/(b + c)
- **Assumes:** a, b, $c > 0$; triangle inequality
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



median_a (median from vertex A (to side a)) against a (side a (opposite A)) from 1.5 to 4.5. One curve for each value of b (side b (opposite B)): 4, 2, 8. Other inputs as in the worked example: c = 5. 31 points per curve computed by geometry-ai999x; marked: the worked example, a = 3, median_a = 4.272.

```
geometry_ai999x.solve('triangle_cevians', a=3, b=4, c=5)
# -> median_a = 4.272, median_b = 3.60555, median_c = 2.5, altitude_a = 4, altitude_b = 3,
altitude_c = 2.4, bisector_a = 4.21637, bisector_b = 3.3541, bisector_c = 2.42437   (SOLVED)
```

```
geometry_ai999x.solve('triangle_cevians', a=13, b=14, c=15)
# -> median_a = 12.9711, median_b = 12.1655, median_c = 11.2361, altitude_a = 12.9231, altitude_b =
12, altitude_c = 11.2, bisector_a = 12.9538, bisector_b = 12.0934, bisector_c = 11.2173   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the medians of a triangle with sides 3, 4 and 5.	SOLVED	median_a = 4.272, median_b = 3.60555, median_c = 2.5, altitude_a = 4, altitude_b = 3, altitude_c = 2.4, bisector_a = 4.21637, bisector_b = 3.3541, bisector_c = 2.42437
Find the altitudes of a triangle with sides 13, 14 and 15.	SOLVED	median_a = 12.9711, median_b = 12.1655, median_c = 11.2361, altitude_a = 12.9231, altitude_b = 12, altitude_c = 11.2, bisector_a = 12.9538, bisector_b = 12.0934, bisector_c = 11.2173
Find the angle bisectors of a triangle with sides 2, 3 and 9.	INVALID_INPUT	Medians, altitudes and angle bisectors of a triangle: the sides violate the triangle inequality (each side must be short

Area of an annulus (ring), `annulus_area`

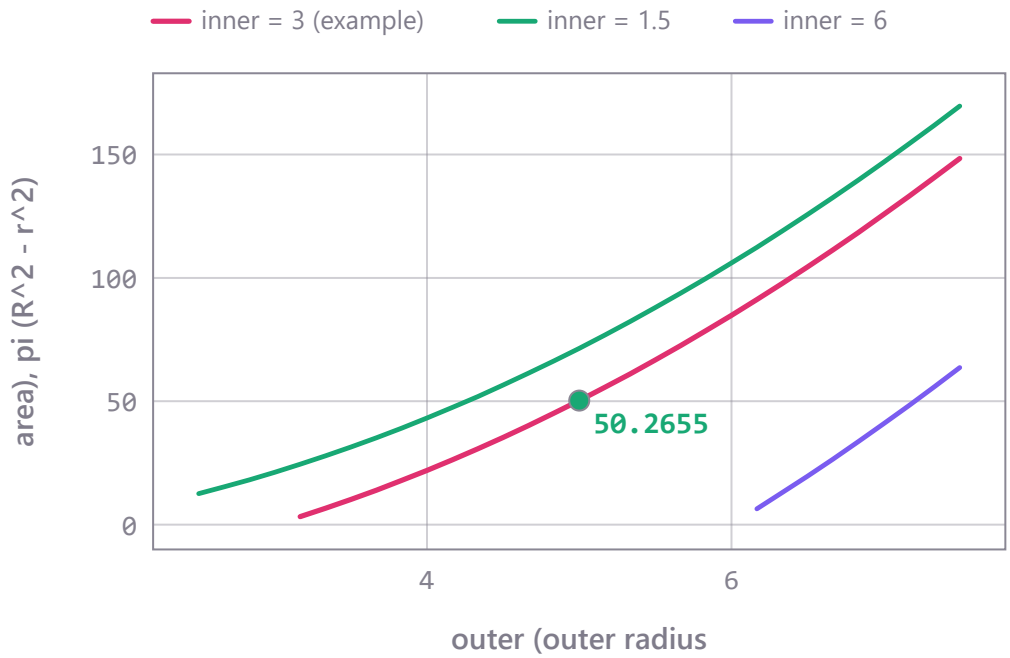
The area between two concentric circles.

$$A = \pi(R^2 - r^2)$$

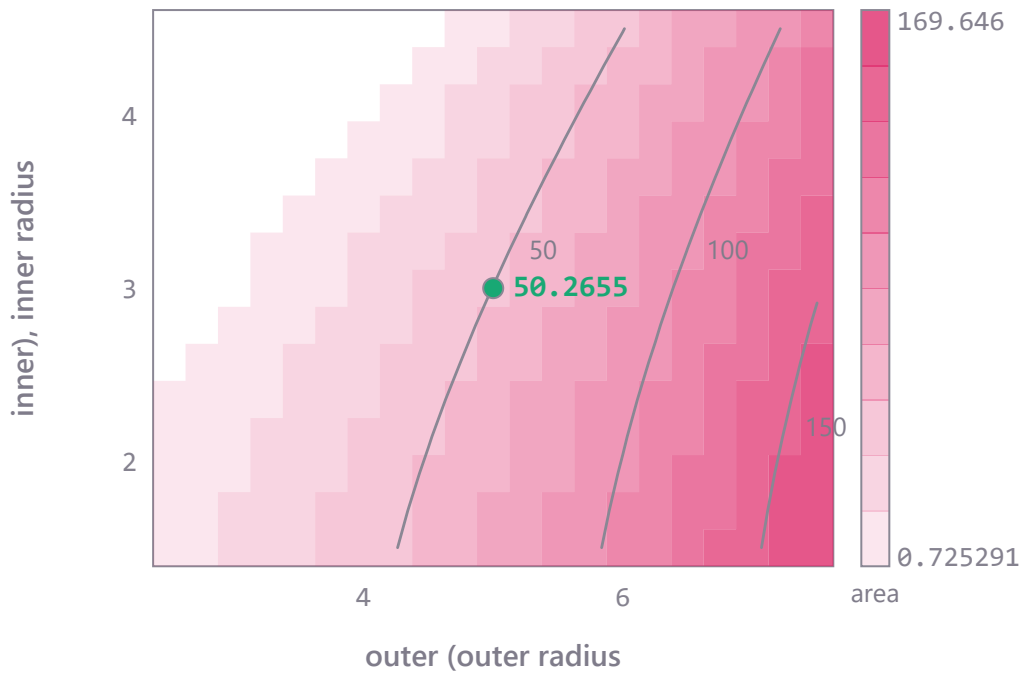
input	meaning	condition
`outer`	outer radius	
`inner`	inner radius	

output	meaning
`area`	pi $(R^2 - r^2)$

- **Method:** $A = pi$ (R - r)(R + r)
- **Assumes:** 0 < inner < outer
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



area (pi (R^2 - r^2)) against outer (outer radius) from 2.5 to 7.5. One curve for each value of inner (inner radius): 3, 1.5, 6. 27 points per curve computed by geometry-ai999x; marked: the worked example, outer = 5, area = 50.2655.



area (pi (R^2 - r^2)) over outer (outer radius) from 2.5 to 7.5 and inner (inner radius) from 1.5 to 4.5: each of the 21 × 15 cells is one run of geometry-ai999x, shaded by its value (light 0.725291 to dark 169.646); iso-lines at area = 50, 100, 150; blank cells: inputs the capability refuses. Marked: the worked example, outer = 5, inner = 3, area = 50.2655.

```
geometry_ai999x.solve('annulus_area', outer=5, inner=3)
# -> area = 50.2655   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the area of an annulus with outer radius 5 and inner radius 3.	SOLVED	area = 50.2655
A washer has an outer radius of 2 cm and an inner radius of 1 cm. What is its area?	SOLVED	area = 9.42478
Find the area of a ring with outer radius 3 and inner radius 5.	INVALID_INPUT	Area of an annulus (ring): the inner radius must be smaller than the outer radius.

Circular segment: area, chord, height, arc, `circular_segment`

Area, chord, height (sagitta) and arc length of the circular segment cut off by a chord with a given central angle.

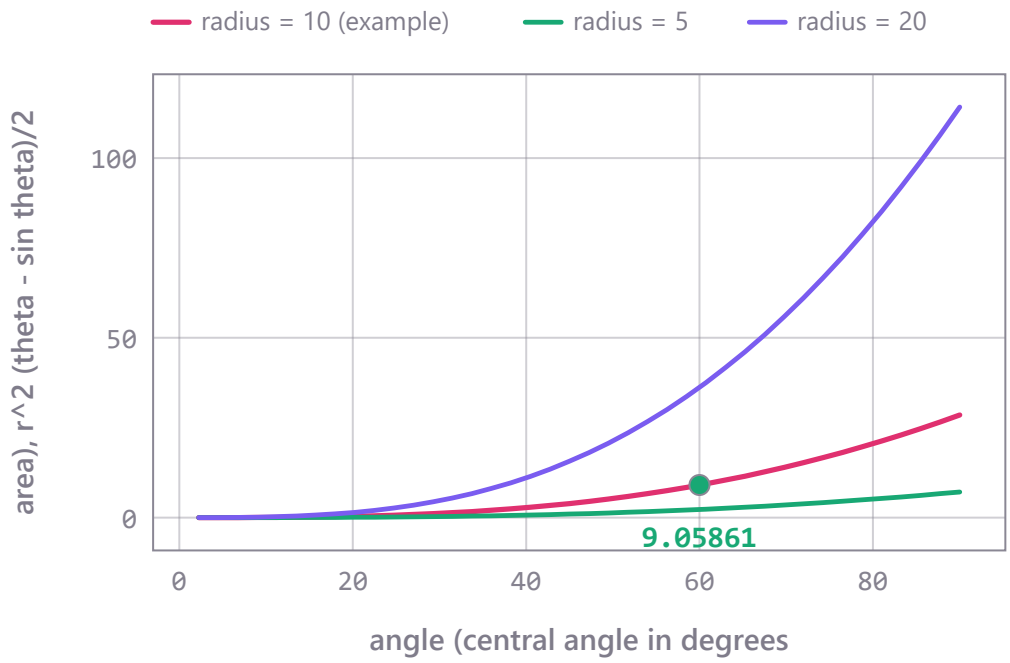
$$A = \tfrac{1}{2}r^2(\theta - \sin\theta), \quad c = 2r\sin\tfrac{\theta}{2}, \quad h = r\left(1 - \cos\tfrac{\theta}{2}\right)$$

input	meaning	condition
`radius`	radius	
`angle`	central angle in degrees	

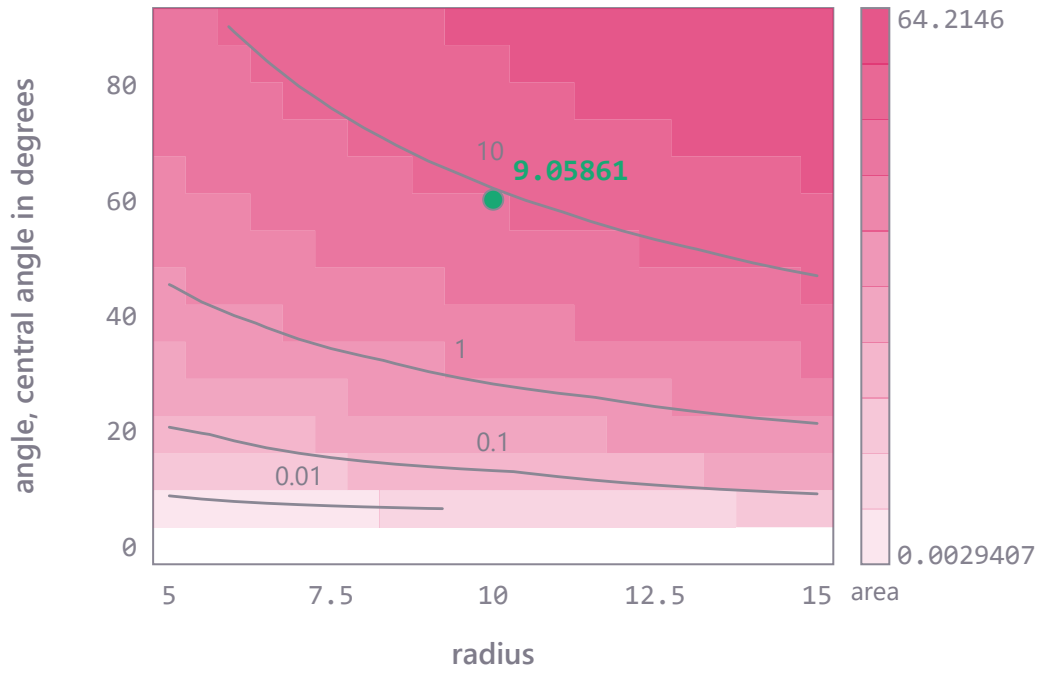
output	meaning
`area`	r^2 (theta - sin theta)/2

output	meaning
`chord`	2 r sin(theta/2)
`height`	r (1 - cos(theta/2))
`arc_length`	r theta

- **Method:** $A = r^2$ (theta - sin theta)/2 (series for small theta to avoid cancellation); $h = 2$ r sin^2 (theta/4)
- **Assumes:** radius > 0; 0 < angle < 360 degrees
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



area (r^2 (theta - sin theta)/2) against angle (central angle in degrees) from 0 to 90. One curve for each value of radius (radius): 10, 5, 20. 40 points per curve computed by geometry-ai999x; marked: the worked example, angle = 60, area = 9.05861.



area (r^2 (theta - sin theta)/2) over radius (radius) from 5 to 15 and angle (central angle in degrees) from 0 to 90: each of the 21 × 15 cells is one run of geometry-ai999x, shaded by its value (light 0.00294077 to dark 64.2146, logarithmic shading); iso-lines at area = 0.01, 0.1, 1, 10; blank cells: inputs the capability refuses. Marked: the worked example, radius = 10, angle = 60, area = 9.05861.

```
geometry_ai999x.solve('circular_segment', radius=10, angle=60)
# -> area = 9.05861, chord = 10, height = 1.33975, arc_length = 10.472   (SOLVED)
```

```
geometry_ai999x.solve('circular_segment', radius=2, angle=180)
# -> area = 6.28319, chord = 4, height = 2, arc_length = 6.28319   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the area of the circular segment with radius 10 and central angle 60 degrees.	SOLVED	area = 9.05861, chord = 10, height = 1.33975, arc_length = 10.472
A segment of a circle of radius 2 has a central angle of 180 degrees. Find its area.	SOLVED	area = 6.28319, chord = 4, height = 2, arc_length = 6.28319
Find the area of the circular segment with radius 10.	NEEDS_INPUT	circular_segment needs angle (central angle in degrees). Example: solve('circular_segment', radius=10, angle=60).

Frustum of a cone: volume and surface area, `frustum_measures`

Volume, slant height, lateral and total surface area of a frustum of a right circular cone.

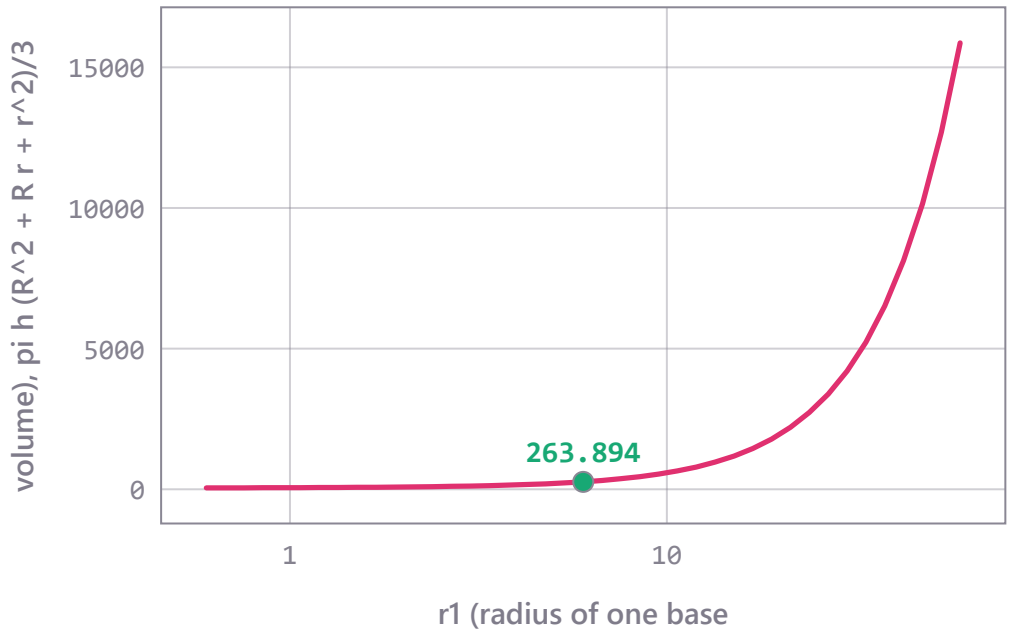
$$V = \tfrac{1}{3}\pi h(R^2 + Rr + r^2), \quad A_L = \pi(R+r)\sqrt{h^2 + (R-r)^2}$$

input	meaning	condition
`r1`	radius of one base	
`r2`	radius of the other base	

input	meaning	condition
`height`	perpendicular height	

output	meaning
`volume`	pi h (R^2 + R r + r^2)/3
`slant_height`	sqrt(h^2 + (R - r)^2)
`lateral_area`	pi (R + r) s
`total_area`	lateral + pi $(R^2 + r^2)$

- **Method:** $V = pi$ h (R^2 + R r + r^2)/3; $s = \sqrt{h^2 + (R - r)^2}$; lateral = pi (R + r) s
- **Assumes:** r1, r2, height > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



volume (pi h (R^2 + R r + r^2)/3) against r1 (radius of one base) from 0.6 to 60 on a logarithmic axis. Other inputs as in the worked example: r2 = 3, height = 4. 41 points computed by geometry-ai999x; marked: the worked example, r1 = 6, volume = 263.894.

```
geometry_ai999x.solve('frustum_measures', r1=6, r2=3, height=4)
# -> volume = 263.894, slant_height = 5, lateral_area = 141.372, total_area = 282.743   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the volume of a frustum with radii 3 and 6 and height 4.	SOLVED	volume = 263.894, slant_height = 5, lateral_area = 141.372, total_area = 282.743
A truncated cone has bottom radius 6, top radius 3 and height 4. Find its total surface area.	SOLVED	volume = 263.894, slant_height = 5, lateral_area = 141.372, total_area = 282.743
Find the volume of a frustum with radii 3 and 6.	NEEDS_INPUT	frustum_measures needs height (perpendicular height). Example: solve('frustum_measures', r1=6, r2=3, height=4).

Right rectangular (or square) pyramid: volume and surface area, `pyramid_measures`

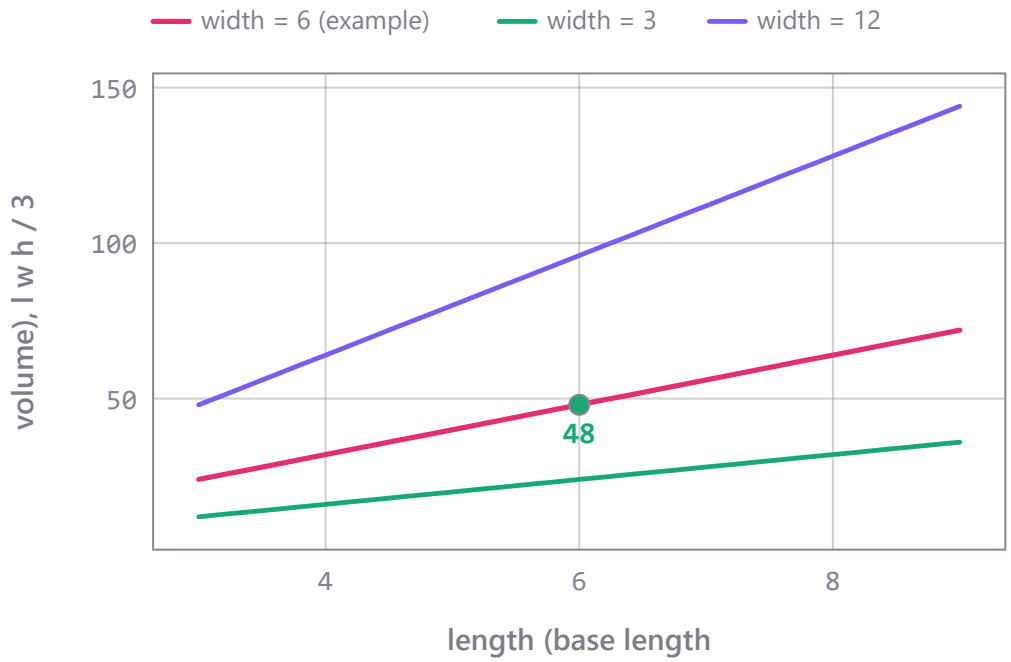
Volume, lateral area and total surface area of a right pyramid on a rectangular base (apex above the base centre).

$$V = \tfrac{1}{3}lwh, \quad A_L = l\sqrt{h^2 + (w/2)^2} + w\sqrt{h^2 + (l/2)^2}$$

input	meaning	condition
`length`	base length	
`width`	base width	
`height`	perpendicular height	

output	meaning
`volume`	l w $h/3$
`lateral_area`	area of the four triangular faces
`total_area`	lateral + base

- **Method:** V = lwh/3; each pair of faces has slant height sqrt(h^2 + (other side/2)^2)
- **Assumes:** length, width, height > 0; the apex is directly above the centre of the base
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



volume (l w h / 3) against length (base length) from 3 to 9. One curve for each value of width (base width): 6, 3, 12. Other inputs as in the worked example: height = 4. 31 points per curve computed by geometry-ai999x; marked: the worked example, length = 6, volume = 48.

```
geometry_ai999x.solve('pyramid_measures', length=6, width=6, height=4)
# -> volume = 48, lateral_area = 60, total_area = 96   (SOLVED)
```

```
geometry_ai999x.solve('pyramid_measures', length=6, width=8, height=12)
# -> volume = 192, lateral_area = 174.849, total_area = 222.849   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the volume of a square pyramid with base side 6 and height 4.	SOLVED	volume = 48, lateral_area = 60, total_area = 96
A pyramid has a 6 by 8 rectangular base and height 12. Find its volume.	SOLVED	volume = 192, lateral_area = 174.849, total_area = 222.849
Find the volume of a square pyramid with base side 6.	NEEDS_INPUT	pyramid_measures needs height (perpendicular height). Example: solve('pyramid_measures', length=6, width=6, height=4).

Right prism on a regular polygon base: volume and surface area, `regular_prism`

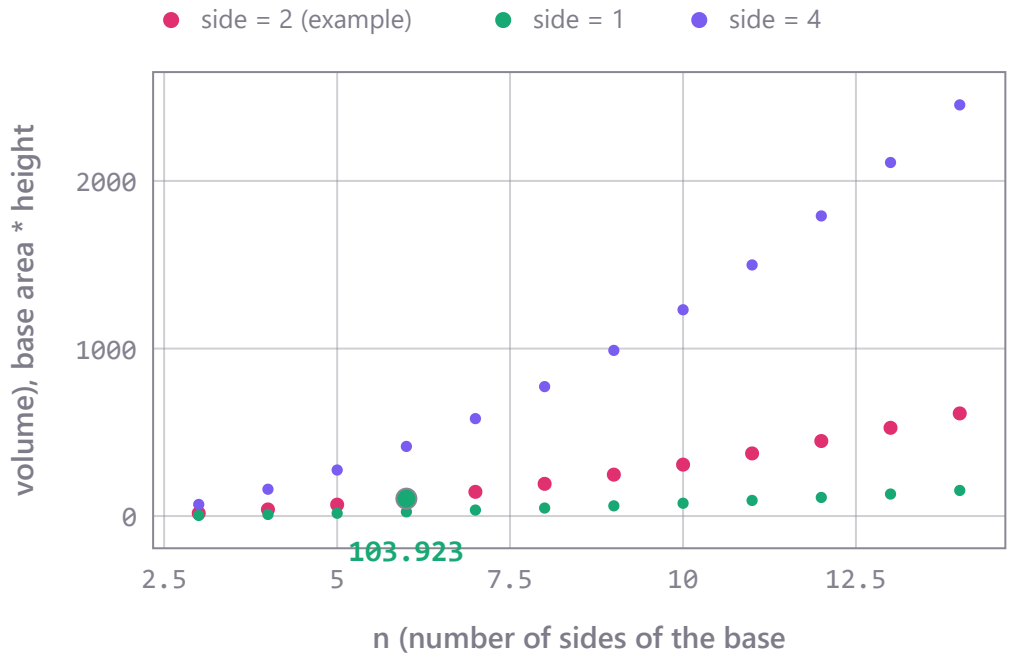
Volume, base area, lateral and total surface area of a right prism whose base is a regular n-gon.

$$B = \frac{ns^2}{4\tan(\pi/n)}, \quad V = Bh, \quad A = nsh + 2B$$

input	meaning	condition
`n`	number of sides of the base	
`side`	side length of the base	
`height`	height (length) of the prism	

output	meaning
`volume`	base area * height
`lateral_area`	n s h
`total_area`	lateral + 2 * base
`base_area`	n s^2 / (4 tan(pi/n))

- **Method:** base area n s^2/(4 tan(pi/n)); $V = B$ h; lateral = perimeter * h
- **Assumes:** $3 \le n \le 200$; side, height > 0; a triangular prism is read only when 'regular' or 'equilateral' is said
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



volume (base area * height) against n (number of sides of the base) from 1 to 14. One curve for each value of side (side length of the base): 2, 1, 4. Other inputs as in the worked example: height = 10. 12 points per curve computed by geometry-ai999x; marked: the worked example, n = 6, volume = 103.923.

```
geometry_ai999x.solve('regular_prism', n=6, side=2, height=10)
# -> volume = 103.923, lateral_area = 120, total_area = 140.785, base_area = 10.3923   (SOLVED)
```

```
geometry_ai999x.solve('regular_prism', n=3, side=4, height=10)
# -> volume = 69.282, lateral_area = 120, total_area = 133.856, base_area = 6.9282   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the volume of a regular hexagonal prism with side 2 and height 10.	SOLVED	volume = 103.923, lateral_area = 120, total_area = 140.785, base_area = 10.3923
Find the total surface area of an equilateral triangular prism with side 4 and length 10.	SOLVED	volume = 69.282, lateral_area = 120, total_area = 133.856, base_area = 6.9282

Platonic solids: counts, Euler characteristic, volume, surface, radii, `platonic_solid`

Vertices, edges, faces, Euler characteristic, volume, surface area, circumradius and inradius of a regular polyhedron of edge a.

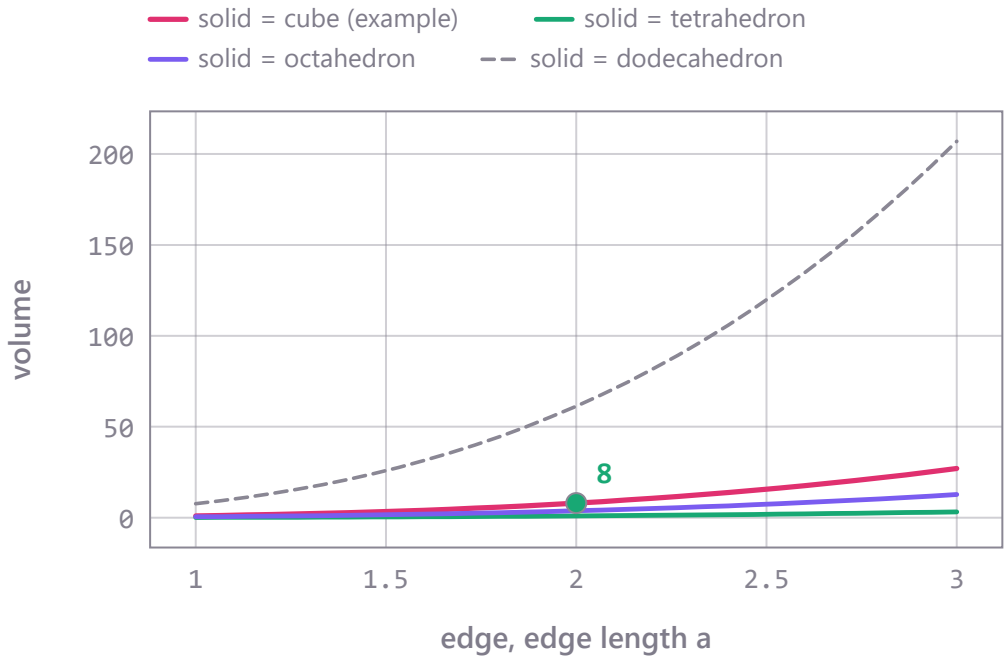
$$V - E + F = 2, \quad V_{\text{ico}} = \tfrac{5}{12}(3+\sqrt{5})\,a^3$$

input	meaning	condition
`solid`	which Platonic solid	
`edge`	edge length a	

output	meaning
`vertices`	V

output	meaning
`edges`	E
`faces`	F
`euler_characteristic`	V - E + F
`volume`	volume
`surface_area`	surface area
`circumradius`	radius of the circumscribed sphere
`inradius`	radius of the inscribed sphere

- **Method:** closed forms in the edge length (golden ratio for the dodecahedron and icosahedron); Euler: $V - E + F = 2$
- **Assumes:** edge > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



volume (volume) against edge (edge length a) from 1 to 3. One curve for each value of solid (which Platonic solid): cube, tetrahedron, octahedron, dodecahedron. 31 points per curve computed by geometry-ai999x; marked: the worked example, edge = 2, volume = 8.

```
geometry_ai999x.solve('platonic_solid', solid='cube', edge=2)
# -> vertices = 8, edges = 12, faces = 6, euler_characteristic = 2, volume = 8, surface_area = 24,
circumradius = 1.73205, inradius = 1   (SOLVED)
```

```
geometry_ai999x.solve('platonic_solid', solid='octahedron', edge=1)
# -> vertices = 6, edges = 12, faces = 8, euler_characteristic = 2, volume = 0.471405, surface_area
= 3.4641, circumradius = 0.707107, inradius = 0.408248   (SOLVED)
```

```
geometry_ai999x.solve('platonic_solid', solid='tetrahedron', edge=6)
# -> vertices = 4, edges = 6, faces = 4, euler_characteristic = 2, volume = 25.4558, surface_area =
62.3538, circumradius = 3.67423, inradius = 1.22474   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the volume and surface area of a regular icosahedron with edge 2.	SOLVED	vertices = 12, edges = 30, faces = 20, euler_characteristic = 2, volume = 17.4536, surface_area = 34.641, circumradius = 1.90211, inradius = 1.51152
What is the circumradius of a cube with edge 2?	SOLVED	vertices = 8, edges = 12, faces = 6, euler_characteristic = 2, volume = 8, surface_area = 24, circumradius = 1.73205, inradius = 1
How many faces, edges and vertices does a dodecahedron have?	NEEDS_INPUT	platonic_solid needs edge (edge length a). Example: solve('platonic_solid', solid='cube', edge=2).
Find the volume of a regular tetrahedron with edge 6.	SOLVED	vertices = 4, edges = 6, faces = 4, euler_characteristic = 2, volume = 25.4558, surface_area = 62.3538, circumradius = 3.67423, inradius = 1.22474

Spherical cap: volume, curved area, base radius, `spherical_cap`

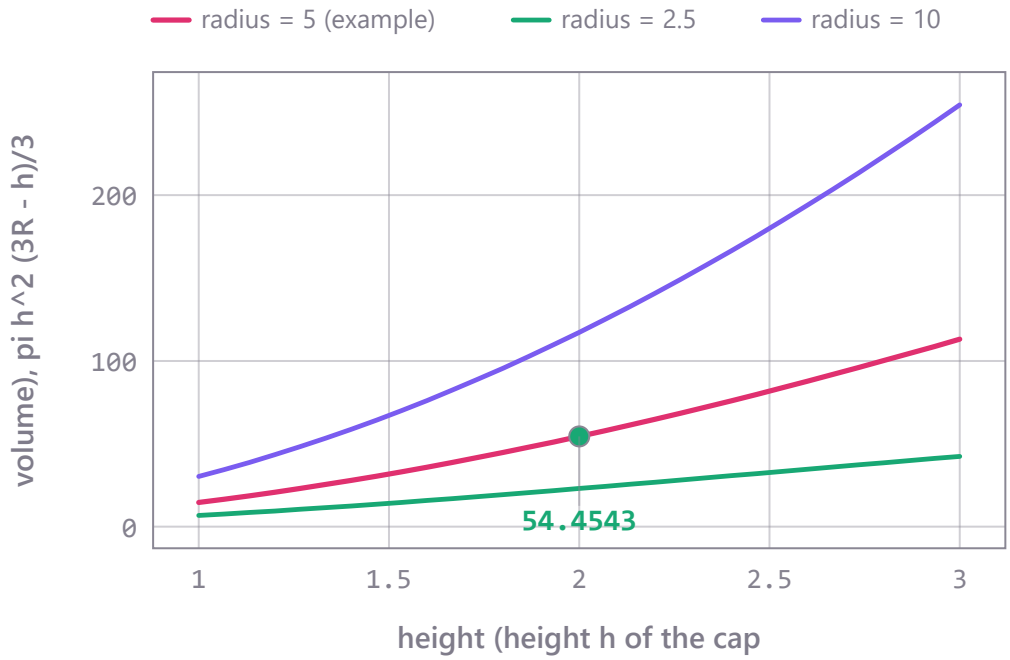
Volume, curved surface area and base radius of a cap of height h cut from a sphere of radius R.

$$V = \tfrac{1}{3}\pi h^2(3R - h), \quad A = 2\pi R h, \quad a = \sqrt{h(2R - h)}$$

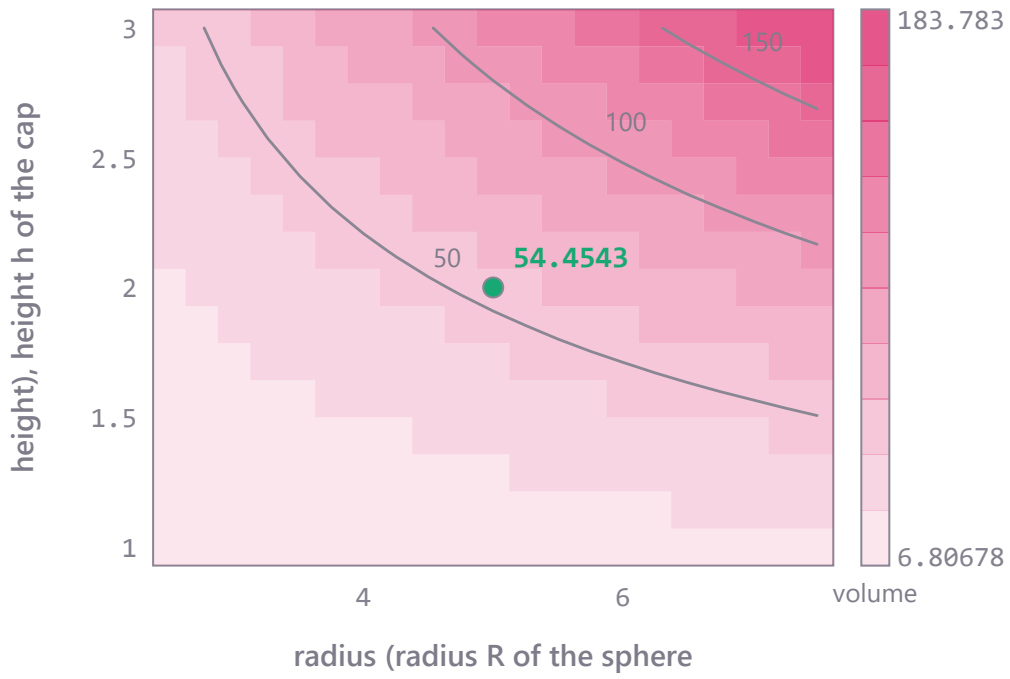
input	meaning	condition
`radius`	radius R of the sphere	
`height`	height h of the cap	

output	meaning
`volume`	pi $h^2\ (3R - h)/3$
`cap_area`	2 pi R h
`base_radius`	sqrt(h $(2R - h)$)

- **Method:** $V = pi\ h^2\ (3R - h)/3$; curved area 2 pi R h (Archimedes)
- **Assumes:** $R > 0$; $0 < h \leq 2R$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



volume (pi h^2 (3R - h)/3) against height (height h of the cap) from 1 to 3. One curve for each value of radius (radius R of the sphere): 5, 2.5, 10. 31 points per curve computed by geometry-ai999x; marked: the worked example, height = 2, volume = 54.4543.



volume (pi h^2 (3R - h)/3) over radius (radius R of the sphere) from 2.5 to 7.5 and height (height h of the cap) from 1 to 3: each of the 21 × 15 cells is one run of geometry-ai999x, shaded by its value (light 6.80678 to dark 183.783); iso-lines at volume = 50, 100, 150. Marked: the worked example, radius = 5, height = 2, volume = 54.4543.

```
geometry_ai999x.solve('spherical_cap', radius=5, height=2)
# -> volume = 54.4543, cap_area = 62.8319, base_radius = 4   (SOLVED)
```

```
geometry_ai999x.solve('spherical_cap', radius=3, height=3)
# -> volume = 56.5487, cap_area = 56.5487, base_radius = 3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the volume of a spherical cap of height 2 cut from a sphere of radius 5.	SOLVED	volume = 54.4543, cap_area = 62.8319, base_radius = 4
Find the volume of a spherical cap of height 12 on a sphere of radius 5.	INVALID_INPUT	Spherical cap: volume, curved area, base radius: the cap height cannot exceed the sphere's diameter.

Torus: volume and surface area, `torus_measures`

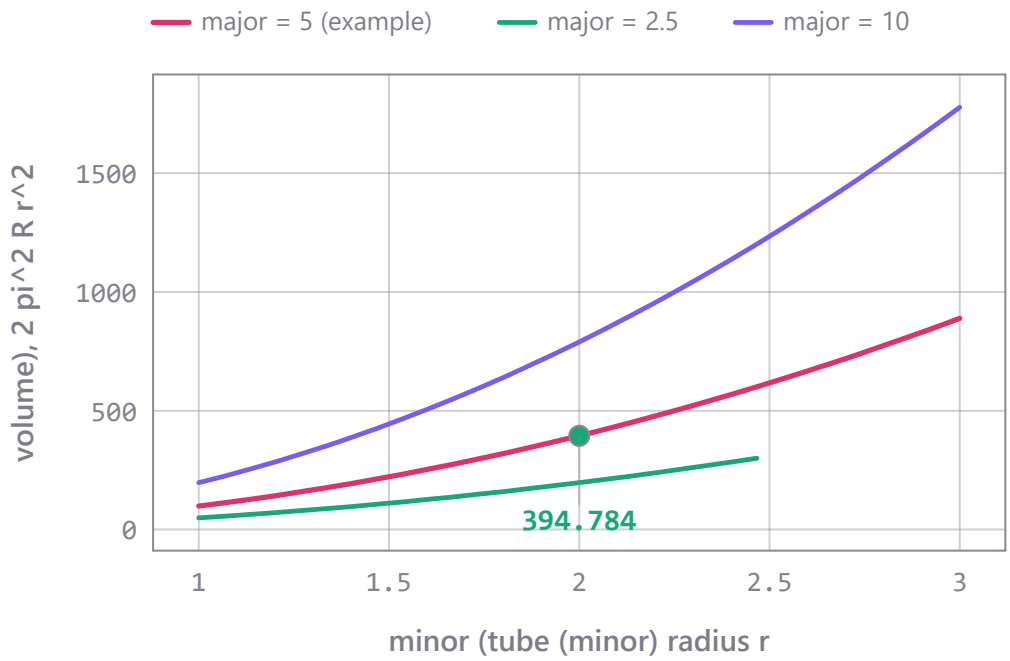
Volume and surface area of a ring torus with major radius R (centre to tube centre) and tube radius r.

$$V = 2\pi^2 R r^2, \quad A = 4\pi^2 R r$$

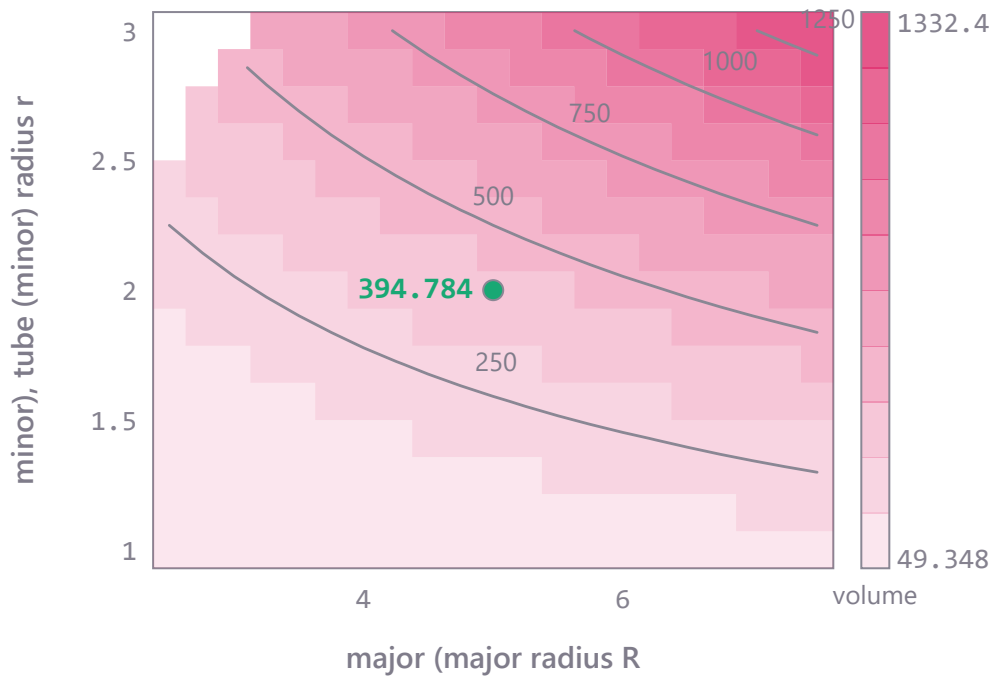
input	meaning	condition
`major`	major radius R	
`minor`	tube (minor) radius r	

output	meaning
`volume`	2 pi^2 R r^2
`surface_area`	4 pi^2 R r

- **Method:** Pappus's theorems: area/volume of the revolved circle times the path 2 pi R
- **Assumes:** $0 < r < R$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



volume (2 pi^2 R r^2) against minor (tube (minor) radius r) from 1 to 3. One curve for each value of major (major radius R): 5, 2.5, 10. 31 points per curve computed by geometry-ai999x; marked: the worked example, minor = 2, volume = 394.784.



volume (2 pi^2 R r^2) over major (major radius R) from 2.5 to 7.5 and minor (tube (minor) radius r) from 1 to 3: each of the 21 × 15 cells is one run of geometry-ai999x, shaded by its value (light 49.348 to dark 1332.4); iso-lines at volume = 250, 500, 750, 1000, 1250; blank cells: inputs the capability refuses. Marked: the worked example, major = 5, minor = 2, volume = 394.784.

```
geometry_ai999x.solve('torus_measures', major=5, minor=2)
# -> volume = 394.784, surface_area = 394.784   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the volume of a torus with major radius 5 and minor radius 2.	SOLVED	volume = 394.784, surface_area = 394.784
A doughnut has major radius 3 cm and tube radius 1 cm. What is its surface area?	SOLVED	volume = 59.2176, surface_area = 118.435
Find the volume of a torus with major radius 2 and minor radius 5.	INVALID_INPUT	Torus: volume and surface area: the tube radius must be smaller than the major radius (a ring torus).

Spheroid (ellipsoid of revolution): surface area and volume, `spheroid_measures`

Surface area and volume of an oblate or prolate spheroid with equatorial radius a and polar radius c.

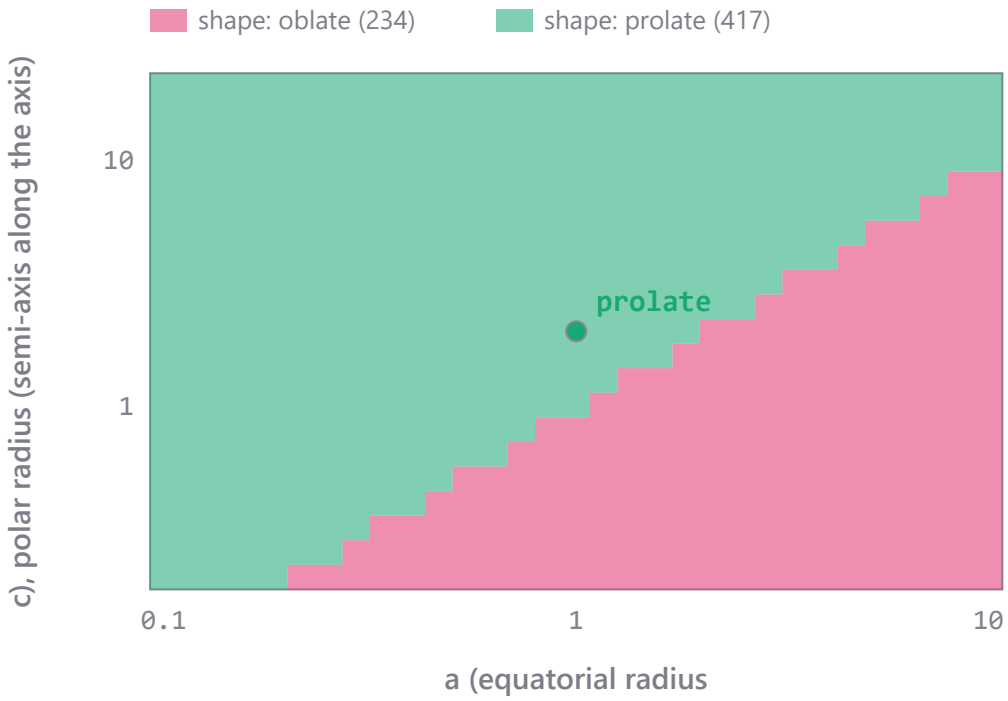
$$S_{\text{obl}} = 2\pi a^2 \left(1 + \frac{1-e^2}{e}\operatorname{artanh} e\right), \quad S_{\text{pro}} = 2\pi a^2 \left(1 + \frac{c}{ae}\arcsin e\right)$$

input	meaning	condition
`a`	equatorial radius (semi-axis in the equator)	
`c`	polar radius (semi-axis along the axis)	

output	meaning
`surface_area`	surface area

output	meaning
`volume`	$4/3$ pi a^2 c
`shape`	oblate / prolate / sphere

- **Method:** oblate: $S = 2$ pi a^2 (1 + (1 - e^2)/e atanh e); prolate: $S = 2$ pi a^2 (1 + c/(a e) asin e); e the eccentricity
- **Assumes:** a, $c > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



oblate / prolate / sphere (shape) over a (equatorial radius (semi-axis in the equator)) from 0.1 to 10 and c (polar radius (semi-axis along the axis)) from 0.2 to 20 (logarithmic axes): each of the 31 × 21 cells is one run of geometry-ai999x, coloured by the verdict it returned, oblate: 234 cells, prolate: 417 cells. Marked: the worked example, a = 1, c = 2 → prolate.

```
geometry_ai999x.solve('spheroid_measures', a=1, c=2)
# -> surface_area = 21.4784, volume = 8.37758, shape = prolate   (SOLVED)
```

```
geometry_ai999x.solve('spheroid_measures', a=3, c=3)
# -> surface_area = 113.097, volume = 113.097, shape = sphere   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the surface area of a prolate spheroid with a = 1 and c = 2.	SOLVED	surface_area = 21.4784, volume = 8.37758, shape = prolate
Find the volume of an oblate spheroid with equatorial radius 2 and polar radius 1.	SOLVED	surface_area = 34.6875, volume = 16.7552, shape = oblate

Perimeter of an ellipse (exact, by the complete elliptic integral), `ellipse_perimeter`

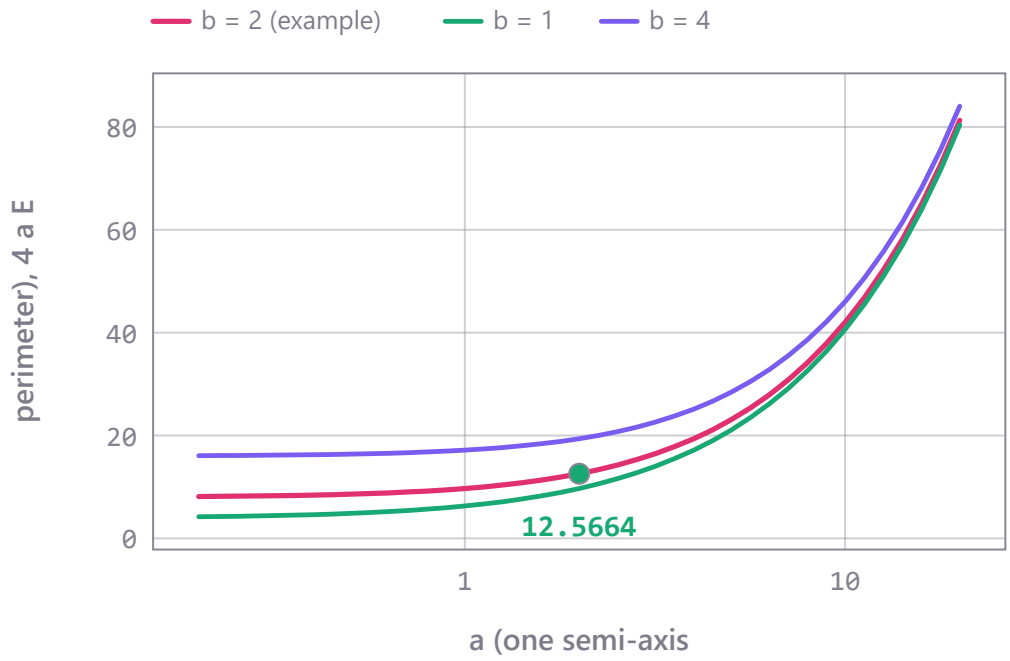
The exact perimeter (circumference) and area of an ellipse with semi-axes a and b.

$$P = 4a\,E(e), \quad E(m) = \frac{\pi}{2\,\mathrm{AGM}(1, \sqrt{1-m})}\left(1 - \sum_{n \geq 0} 2^{n-1} c_n^2\right)$$

input	meaning	condition
`a`	one semi-axis	
`b`	the other semi-axis	

output	meaning
`perimeter`	4 a E(e^2), a the larger semi-axis
`area`	pi a b

- **Method:** complete elliptic integral of the second kind by the arithmetic-geometric mean (Gauss), quadratically convergent
- **Assumes:** a, $b > 0$; axis ratio at least 0.001
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



perimeter (4 a E(e^2), a the larger semi-axis) against a (one semi-axis) from 0.2 to 20 on a logarithmic axis. One curve for each value of b (the other semi-axis): 2, 1, 4. 41 points per curve computed by geometry-ai999x; marked: the worked example, a = 2, perimeter = 12.5664.

```
geometry_ai999x.solve('ellipse_perimeter', a=2, b=2)
# -> perimeter = 12.5664, area = 12.5664   (SOLVED)
```

```
geometry_ai999x.solve('ellipse_perimeter', a=5, b=3)
# -> perimeter = 25.527, area = 47.1239   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the perimeter of an ellipse with semi-major axis 5 and semi-minor axis 3.	SOLVED	perimeter = 25.527, area = 47.1239
What is the circumference of the ellipse x^2/25 + y^2/9 = 1?	SOLVED	perimeter = 25.527, area = 47.1239

Triangle in space: area and perimeter from vertices, `triangle_area_3d`

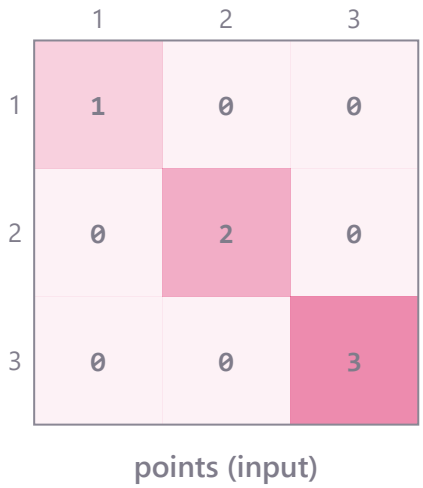
Area and perimeter of the triangle with vertices A, B, C in three-dimensional space.

$$K = \tfrac{1}{4}\sqrt{(a+b+c)(-a+b+c)(a-b+c)(a+b-c)}$$

input	meaning	condition
`points`	the vertices: xA yA zA, xB yB zB, xC yC zC	

output	meaning
`area`	area
`perimeter`	perimeter

- **Method:** side lengths, then Heron's formula in Kahan's stable form
- **Assumes:** exactly 3 points, not collinear
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): points (input). Returned with them by geometry-ai999x: area = 3.5, perimeter = 9.0039.

```
geometry_ai999x.solve('triangle_area_3d', points='1 0 0, 0 2 0, 0 0 3')
# -> area = 3.5, perimeter = 9.0039   (SOLVED)
```

```
geometry_ai999x.solve('triangle_area_3d', points='0 0 0, 3 0 0, 0 4 0')
# -> area = 6, perimeter = 12   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the area of the triangle with vertices (1, 0, 0), (0, 2, 0) and (0, 0, 3).	SOLVED	area = 3.5, perimeter = 9.0039
Find the area of the triangle with vertices (0, 0, 0), (1, 1, 1) and (2, 2, 2).	INVALID_INPUT	Triangle in space: area and perimeter from vertices: the three points are collinear.

Tetrahedron from four vertices: volume and surface area, `tetrahedron_measures`

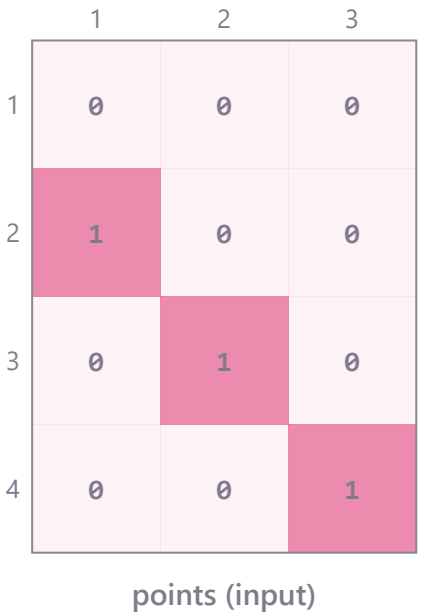
Volume and total surface area of the tetrahedron with four given vertices.

$$V = \tfrac{1}{6}\left|(B - A) \cdot \big((C - A) \times (D - A)\big)\right|$$

input	meaning	condition
`points`	the four vertices: x y z for each	

output	meaning
`volume`	|det(B - A, C - A, D - A)| / 6
`surface_area`	sum of the four face areas

- **Method:** scalar triple product; face areas from cross products
- **Assumes:** exactly 4 points; not coplanar
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): points (input). Returned with them by geometry-ai999x: volume = 0.166667, surface_area = 2.36603.

```
geometry_ai999x.solve('tetrahedron_measures', points='0 0 0, 1 0 0, 0 1 0, 0 0 1')
# -> volume = 0.166667, surface_area = 2.36603   (SOLVED)
```

```
geometry_ai999x.solve('tetrahedron_measures', points='0 0 0, 3 0 0, 0 4 0, 0 0 6')
# -> volume = 12, surface_area = 43.1555   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the volume of the tetrahedron with vertices (0, 0, 0), (1, 0, 0), (0, 1, 0) and (0, 0, 1).	SOLVED	volume = 0.166667, surface_area = 2.36603
Find the volume of the tetrahedron with vertices (0, 0, 0), (1, 0, 0), (0, 1, 0) and (1, 1, 0).	INVALID_INPUT	Tetrahedron from four vertices: volume and surface area: the four points are coplanar: they do not form a tetrahedron.

Plane through three points, `plane_through_points`

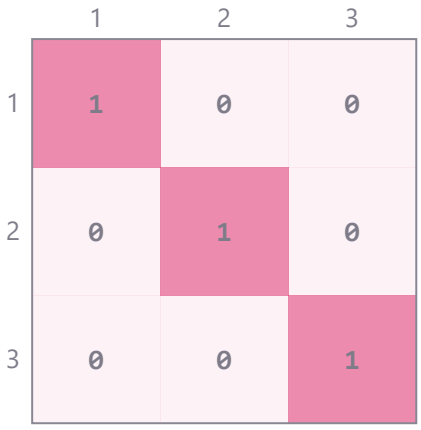
The equation a x + b y + c $z + d = 0$ of the plane through three points, with (a, b, c) a unit normal.

$$\mathbf{n} = (B - A) \times (C - A), \quad \mathbf{n} \cdot \mathbf{x} - \mathbf{n} \cdot A = 0$$

input	meaning	condition
`points`	the three points: x y z for each	

output	meaning
`a`	x-component of the unit normal
`b`	y-component
`c`	z-component
`d`	constant term

- **Method:** normal $n = (B - A)$ x (C - A) (exact), normalised; $d = -n.A$
- **Assumes:** exactly 3 points, not collinear; convention: (a, b, c) has length 1 and its first nonzero component is positive
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



points (input)

The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): points (input). Returned with them by geometry-ai999x: a = 0.57735, b = 0.57735, c = 0.57735, d = -0.57735.

```
geometry_ai999x.solve('plane_through_points', points='1 0 0, 0 1 0, 0 0 1')
# -> a = 0.57735, b = 0.57735, c = 0.57735, d = -0.57735   (SOLVED)
```

```
geometry_ai999x.solve('plane_through_points', points='0 0 2, 1 0 2, 0 1 2')
# -> a = 0, b = 0, c = 1, d = -2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the equation of the plane through (1, 0, 0), (0, 1, 0) and (0, 0, 1).	SOLVED	a = 0.57735, b = 0.57735, c = 0.57735, d = -0.57735
Find the plane passing through (0, 0, 2), (1, 0, 2) and (0, 1, 2).	SOLVED	a = 0, b = 0, c = 1, d = -2
Find the plane through (0, 0, 0), (1, 1, 1) and (2, 2, 2).	INVALID_INPUT	Plane through three points: the three points are collinear: they do not determine a plane.

Distance from a point to a plane, `point_plane_distance`

The distance from a point to the plane a x + b y + c $z + d = 0$ and the foot of the perpendicular.

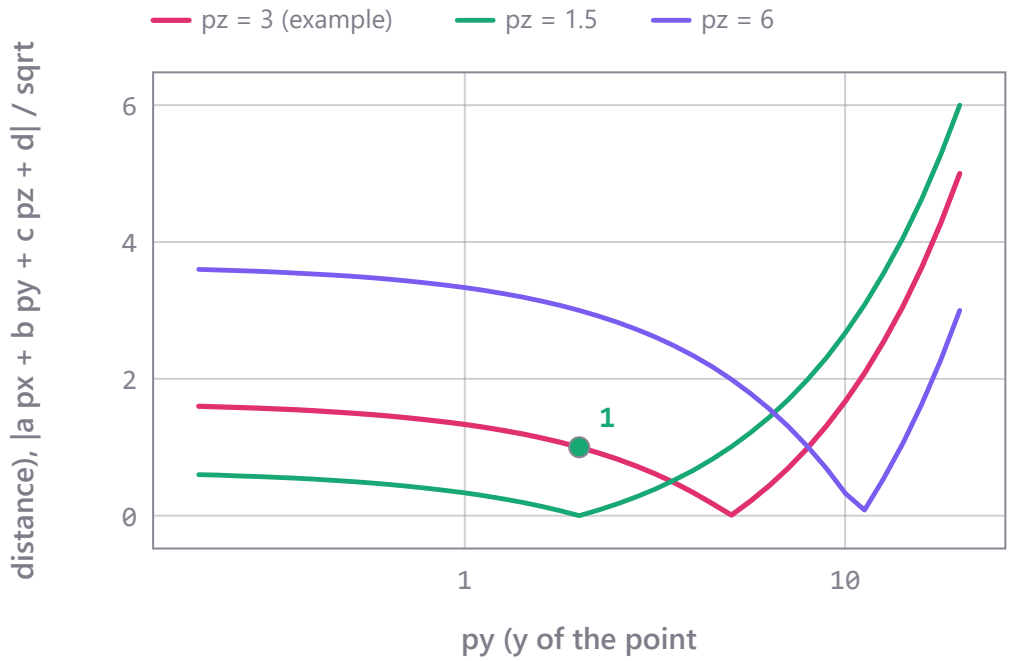
$$d = \frac{|ap_x + bp_y + cp_z + d|}{\sqrt{a^2 + b^2 + c^2}}$$

input	meaning	condition
`px`	x of the point	
`py`	y of the point	
`pz`	z of the point	
`a`	coefficient of x	
`b`	coefficient of y	

input	meaning	condition
`c`	coefficient of z	
`d`	constant term	

output	meaning
`distance`	|a px + b py + c pz + d| / sqrt(a^2 + b^2 + c^2)
`fx`	x of the foot
`fy`	y of the foot
`fz`	z of the foot

- **Method:** signed value of the plane's equation divided by the normal's length
- **Assumes:** a, b, c not all 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



distance (|a px + b py + c pz + d| / sqrt(a^2 + b^2 + c^2)) against py (y of the point) from 0.2 to 20 on a logarithmic axis. One curve for each value of pz (z of the point): 3, 1.5, 6. Other inputs as in the worked example: px = 1, a = 2, b = -1, c = 2, d = -3. 41 points per curve computed by geometry-ai999x; marked: the worked example, py = 2, distance = 1.

```
geometry_ai999x.solve('point_plane_distance', px=1, py=2, pz=3, a=2, b=-1, c=2, d=-3)
# -> distance = 1, fx = 0.333333, fy = 2.33333, fz = 2.33333   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the distance from the point (1, 2, 3) to the plane 2x - y + 2z = 3.	SOLVED	distance = 1, fx = 0.333333, fy = 2.33333, fz = 2.33333
Find the distance from the point (0, 0, 5) to the plane z = 1.	SOLVED	distance = 4, fx = 0, fy = 0, fz = 1

Intersection of a line and a plane, `line_plane_intersection`

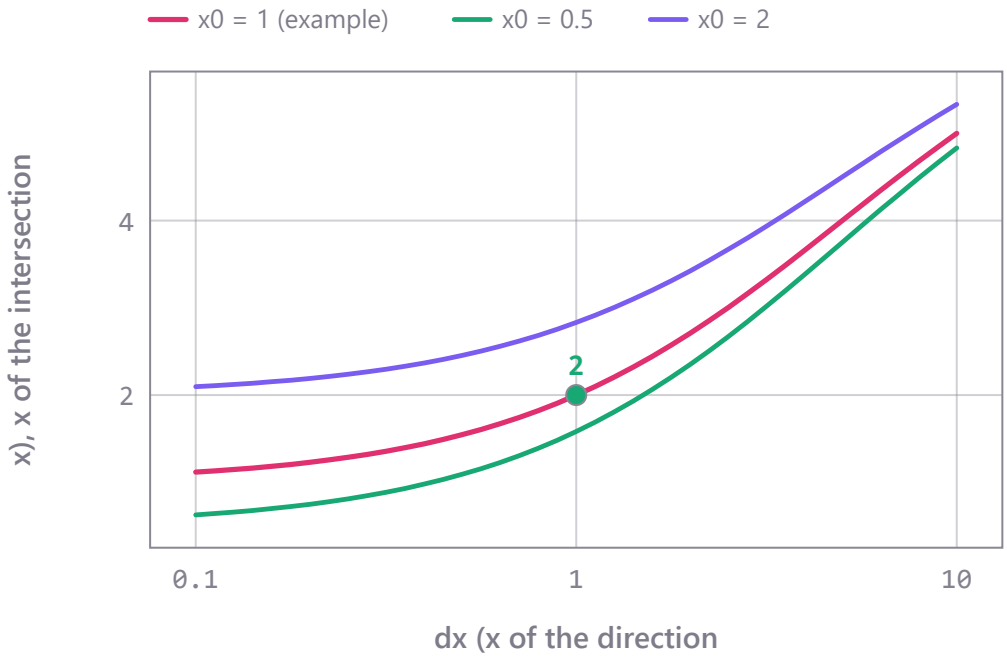
The point where the line $P_0 + t$ v meets the plane a x + b y + c $z + d = 0$.

$$t = -\frac{\mathbf{n} \cdot P_0 + d}{\mathbf{n} \cdot \mathbf{v}}, \quad X = P_0 + t\,\mathbf{v}$$

input	meaning	condition
`x0`	x of a point on the line	
`y0`	y of the point	
`z0`	z of the point	
`dx`	x of the direction	
`dy`	y of the direction	
`dz`	z of the direction	
`a`	coefficient of x	
`b`	coefficient of y	
`c`	coefficient of z	
`d`	constant term	

output	meaning
`x`	x of the intersection
`y`	y
`z`	z
`t`	line parameter at the intersection

- **Method:** t = -$(n.P_0 + d)/(n.v)$ in exact rational arithmetic
- **Assumes:** direction not zero; a, b, c not all 0; line not parallel to the plane (exact test)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



x (x of the intersection) against dx (x of the direction) from 0.1 to 10 on a logarithmic axis. One curve for each value of x0 (x of a point on the line): 1, 0.5, 2. Other inputs as in the worked example: y0 = 0, z0 = 0, dy = 2, dz = 3, a = 1, b = 1, c = 1, d = -7. 41 points per curve computed by geometry-ai999x; marked: the worked example, dx = 1, x = 2.

```
geometry_ai999x.solve('line_plane_intersection', x0=1, y0=0, z0=0, dx=1, dy=2, dz=3, a=1, b=1, c=1, d=-7)
# -> x = 2, y = 2, z = 3, t = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Where does the line through (1, 0, 0) with direction (1, 2, 3) meet the plane x + y + z = 7?	SOLVED	x = 2, y = 2, z = 3, t = 1
Where does the line through (0, 0, 0) with direction (1, -1, 0) meet the plane x + y + z = 7?	INVALID_INPUT	Intersection of a line and a plane: the line is parallel to the plane (or lies in it): no single intersection point.

Distance between two lines in space (skew or parallel), `skew_lines_distance`

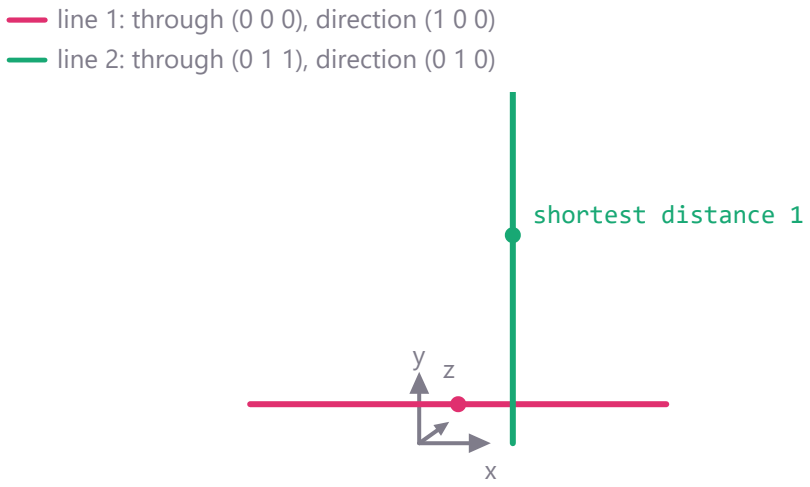
The shortest distance between the lines $P_1 + s$ d1 and $P_2 + t$ d2 in three dimensions.

$$d = \frac{|(P_2 - P_1) \cdot (\mathbf{d_1} \times \mathbf{d_2})|}{\|\mathbf{d_1} \times \mathbf{d_2}\|}$$

input	meaning	condition
`p1`	a point on line 1	
`d1`	direction of line 1	
`p2`	a point on line 2	
`d2`	direction of line 2	

output	meaning
`distance`	shortest distance
`parallel`	yes / no (exact test)

- **Method:** $|(P_2 - P_1)$.(d1 x d2)| / |d1 x d2|; parallel lines: $|(P_2 - P_1)$ x d1| / |d1|
- **Assumes:** nonzero directions
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The two stated lines in an oblique 3-D view: line 1 through (0 0 0) along (1 0 0), line 2 through (0 1 1) along (0 1 0). The package returned their shortest distance 1 (parallel: no). Marked: the stated points of each line.

```
geometry_ai999x.solve('skew_lines_distance', p1='0 0 0', d1='1 0 0', p2='0 1 1', d2='0 1 0')
# -> distance = 1, parallel = no   (SOLVED)
```

```
geometry_ai999x.solve('skew_lines_distance', p1=[0, 0, 0], d1=[1, 0, 0], p2=[0, 3, 4], d2=[2, 0, 0])
# -> distance = 5, parallel = yes   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the distance between the lines through (0, 0, 0) with direction (1, 0, 0) and through (0, 1, 1) with direction (0, 1, 0).	SOLVED	distance = 1, parallel = no
Find the distance between the lines through (0, 0, 0) with direction (0, 0, 0) and through (0, 1, 1) with direction (0, 1, 0).	INVALID_INPUT	Distance between two lines in space (skew or parallel): a direction vector is zero.

Angle between two planes, `angle_between_planes`

The acute (dihedral) angle between two planes given by their equations (only the normals matter).

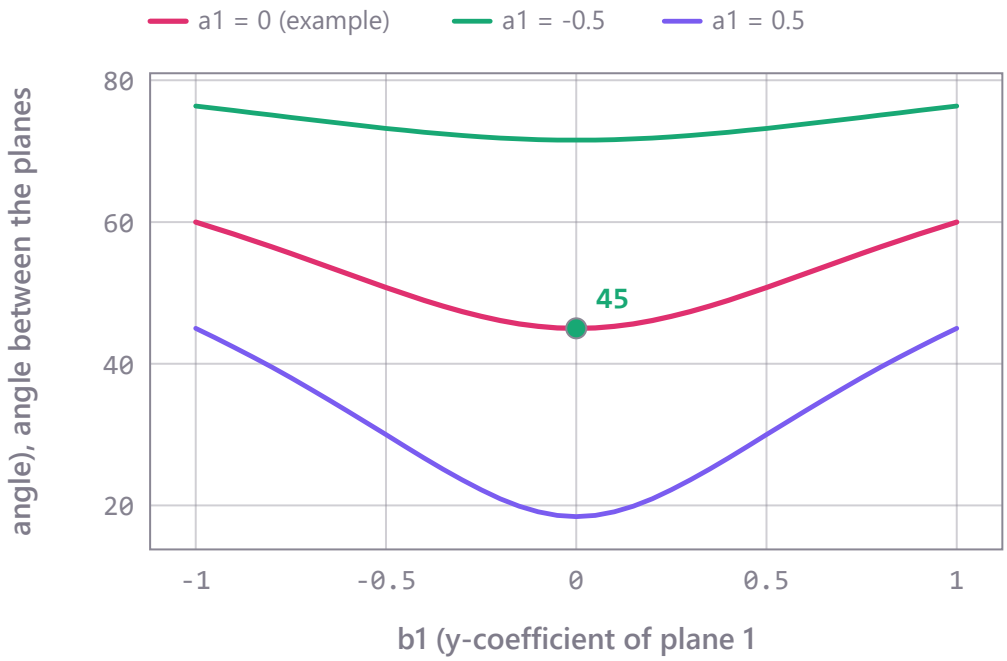
$$\cos\theta = \frac{|\mathbf{n}_1 \cdot \mathbf{n}_2|}{\|\mathbf{n}_1\|\|\mathbf{n}_2\|}$$

input	meaning	condition
`a1`	x-coefficient of plane 1	
`b1`	y-coefficient of plane 1	
`c1`	z-coefficient of plane 1	
`a2`	x-coefficient of plane 2	
`b2`	y-coefficient of plane 2	

input	meaning	condition
`c2`	z-coefficient of plane 2	

output	meaning
`angle`	angle between the planes, degrees, 0 to 90

- **Method:** angle between the normals, folded to [0, 90]: atan2(|n1 x n2|, |n1.n2|)
- **Assumes:** nonzero normals; the constants do not affect the angle
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



angle (angle between the planes, degrees, 0 to 90) against b1 (y-coefficient of plane 1) from -1 to 1. One curve for each value of a1 (x-coefficient of plane 1): 0, -0.5, 0.5. Other inputs as in the worked example: c1 = 1, a2 = 1, b2 = 0, c2 = 1. 41 points per curve computed by geometry-ai999x; marked: the worked example, b1 = 0, angle = 45.

```
geometry_ai999x.solve('angle_between_planes', a1=0, b1=0, c1=1, a2=1, b2=0, c2=1)
# -> angle = 45   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the angle between the planes z = 0 and x + z = 2.	SOLVED	angle = 45
Find the angle between the planes x + y + z = 1 and x - y = 0 + 0z.	SOLVED	angle = 90

Rotation of a point in space about an axis, `rotate_point_3d`

The image of a point rotated by an angle about an axis through the origin (right-hand rule).

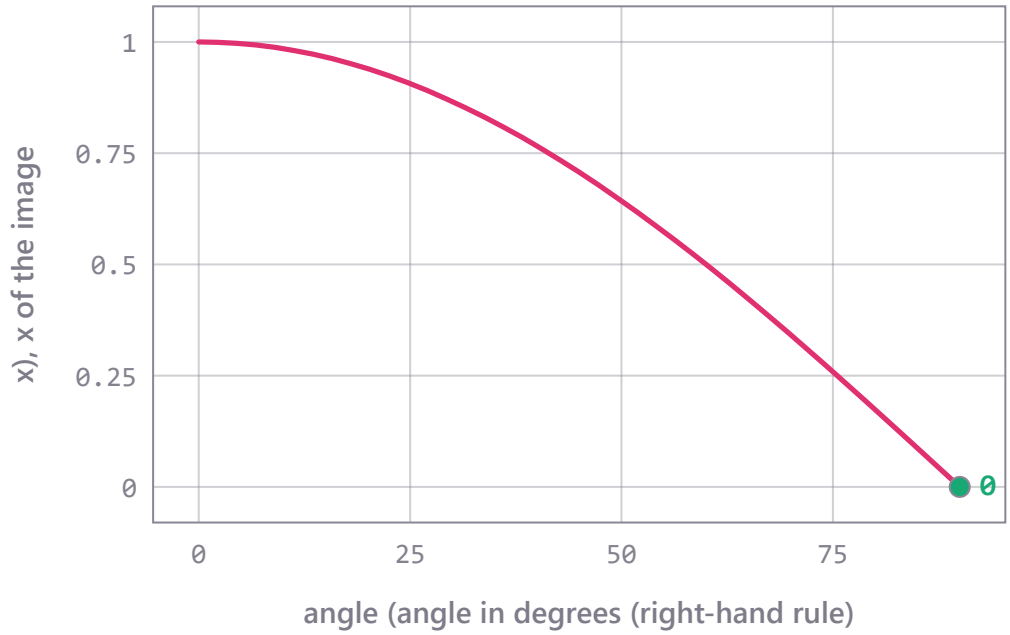
$$\mathbf{v}' = \mathbf{v}\cos\theta + (\mathbf{k}\times\mathbf{v})\sin\theta + \mathbf{k}(\mathbf{k}\cdot\mathbf{v})(1-\cos\theta)$$

input	meaning	condition
`point`	the point x y z	

input	meaning	condition
`axis`	axis direction (through the origin)	
`angle`	angle in degrees (right-hand rule)	

output	meaning
`x`	x of the image
`y`	y of the image
`z`	z of the image

- **Method:** Rodrigues' formula v' = v cos t + (k x v) sin t + k (k.v)(1 - cos t), exact for multiples of 90 degrees
- **Assumes:** axis not zero; it passes through the origin; positive angle: counterclockwise looking from the axis tip toward the origin
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



x (x of the image) against angle (angle in degrees (right-hand rule)) from 0 to 90. Other inputs as in the worked example: point = 1 0 0, axis = 0 0 1. 41 points computed by geometry-ai999x; marked: the worked example, angle = 90, x = 0.

```
geometry_ai999x.solve('rotate_point_3d', point='1 0 0', axis='0 0 1', angle=90)
# -> x = 0, y = 1, z = 0   (SOLVED)
```

```
geometry_ai999x.solve('rotate_point_3d', point=[1, 2, 3], axis=[1, 1, 1], angle=120)
# -> x = 3, y = 1, z = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rotate the point (1, 0, 0) by 90 degrees about the z-axis.	SOLVED	x = 0, y = 1, z = 0
Rotate (1, 2, 3) by 120 degrees about the axis (1, 1, 1).	SOLVED	x = 3, y = 1, z = 2

Sphere through four points (circumsphere of a tetrahedron), `sphere_four_points`

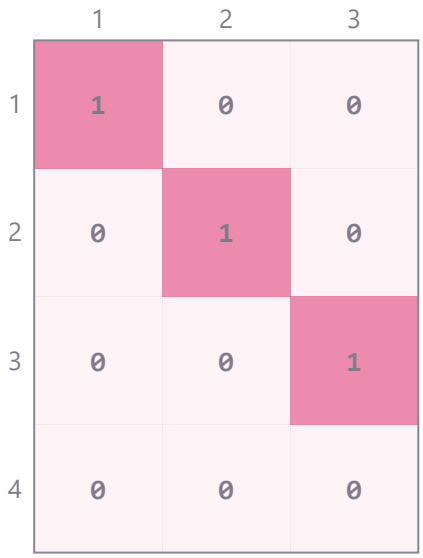
Centre and radius of the sphere through four non-coplanar points.

$$|X - P_i|^2 = R^2 \ (i = 1..4) \Rightarrow 2(P_i - P_1) \cdot X = |P_i|^2 - |P_1|^2$$

input	meaning	condition
`points`	the four points: x y z for each	

output	meaning
`cx`	x of the centre
`cy`	y of the centre
`cz`	z of the centre
`radius`	radius

- **Method:** subtract the first sphere equation from the others: a 3x3 linear system, solved exactly (Cramer)
- **Assumes:** exactly 4 points, not coplanar
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



points (input)

The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): points (input). Returned with them by geometry-ai999x: cx = 0.5, cy = 0.5, cz = 0.5, radius = 0.866025.

```
geometry_ai999x.solve('sphere_four_points', points='1 0 0, 0 1 0, 0 0 1, 0 0 0')
# -> cx = 0.5, cy = 0.5, cz = 0.5, radius = 0.866025   (SOLVED)
```

```
geometry_ai999x.solve('sphere_four_points', points='2 0 0, -2 0 0, 0 2 0, 0 0 2')
# -> cx = 0, cy = 0, cz = 0, radius = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the sphere through (1, 0, 0), (0, 1, 0), (0, 0, 1) and (0, 0, 0).	SOLVED	cx = 0.5, cy = 0.5, cz = 0.5, radius = 0.866025
Find the sphere passing through (0, 0, 0), (1, 0, 0), (0, 1, 0) and (1, 1, 0).	INVALID_INPUT	Sphere through four points (circumsphere of a tetrahedron): the four points are coplanar: no unique sphere passes throug

Great-circle distance between two points on a sphere, `great_circle_distance`

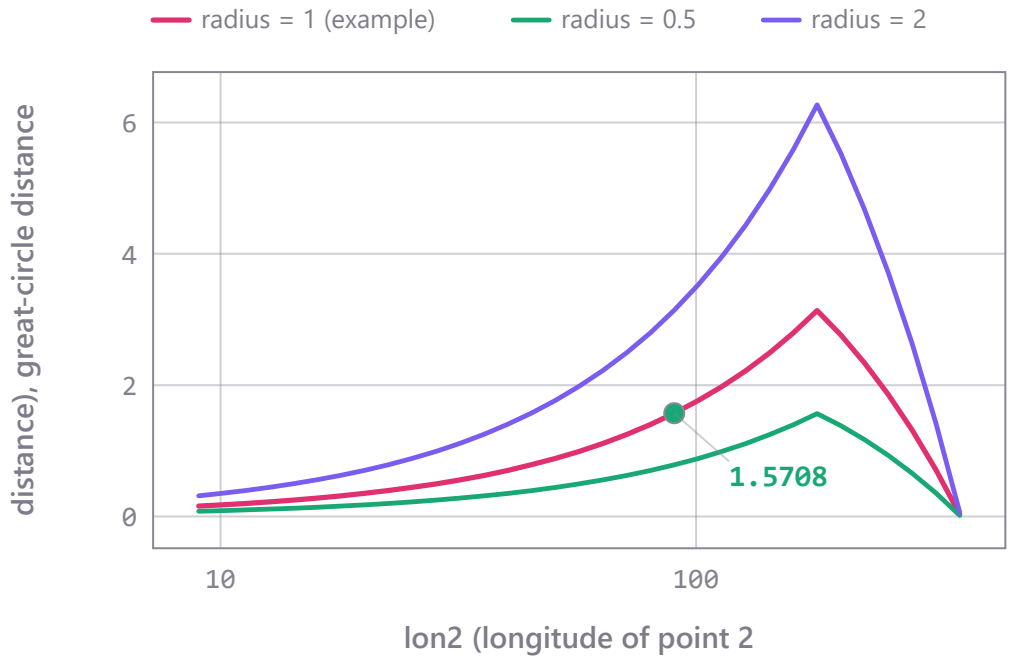
The central angle and great-circle distance between two points given by latitude and longitude (on the Earth unless a radius is given).

$$h = \sin^2 \tfrac{\Delta\varphi}{2} + \cos\varphi_1 \cos\varphi_2 \sin^2 \tfrac{\Delta\lambda}{2}, \quad d = 2R \operatorname{atan2}(\sqrt{h}, \sqrt{1-h})$$

input	meaning	condition
`lat1`	latitude of point 1, degrees	
`lon1`	longitude of point 1, degrees	
`lat2`	latitude of point 2	
`lon2`	longitude of point 2	
`radius`	sphere radius (default 6371, the mean Earth radius in km)	

output	meaning
`central_angle`	central angle, degrees
`distance`	great-circle distance (radius units)

- **Method:** haversine formula in its atan2 form
- **Assumes:** points written (latitude, longitude) in signed decimal degrees; radius defaults to 6371 km (mean Earth radius; the Earth is treated as a sphere)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



distance (great-circle distance (radius units)) against lon2 (longitude of point 2) from 9 to 900 on a logarithmic axis. One curve for each value of radius (sphere radius (default 6371, the mean Earth radius in km)): 1, 0.5, 2. Other inputs as in the worked example: lat1 = 0, lon1 = 0, lat2 = 0. 33 points per curve computed by geometry-ai999x; marked: the worked example, lon2 = 90, distance = 1.5708.

```
geometry_ai999x.solve('great_circle_distance', lat1=0, lon1=0, lat2=0, lon2=90, radius=1)
# -> central_angle = 90, distance = 1.5708   (SOLVED)
```

```
geometry_ai999x.solve('great_circle_distance', lat1=90, lon1=0, lat2=0, lon2=45)
# -> central_angle = 90, distance = 10007.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the central angle between (0, 0) and (0, 90) on a sphere of radius 1?	SOLVED	central_angle = 90, distance = 1.5708
Find the great-circle distance between (90, 0) and (0, 45) on the Earth.	SOLVED	central_angle = 90, distance = 10007.5
Find the great-circle distance between (95, 0) and (0, 45) on the Earth.	INVALID_INPUT	Great-circle distance between two points on a sphere: point 1: latitude must be between -90 and 90 degrees.

Initial bearing (forward azimuth) along a great circle, `initial_bearing`

The initial compass bearing from point 1 to point 2 along the great circle, degrees clockwise from north.

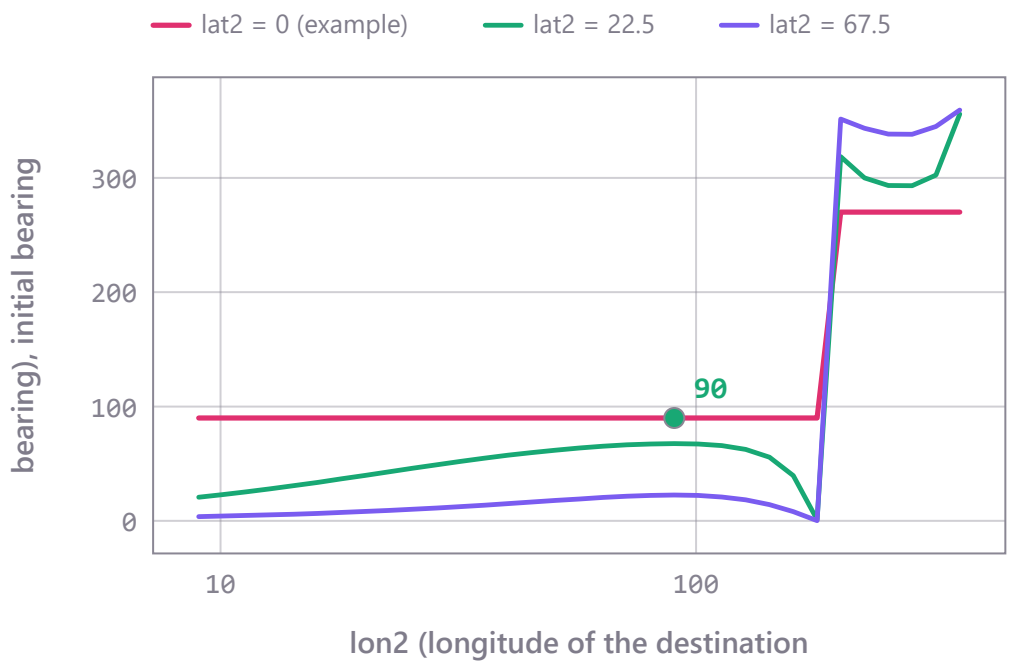
$$\theta = \operatorname{atan2}\left(\sin\Delta\lambda\cos\varphi_2,\ \cos\varphi_1\sin\varphi_2 - \sin\varphi_1\cos\varphi_2\cos\Delta\lambda\right)$$

input	meaning	condition
`lat1`	latitude of the start, degrees	
`lon1`	longitude of the start	
`lat2`	latitude of the destination	

input	meaning	condition
`lon2`	longitude of the destination	

output	meaning
`bearing`	initial bearing, degrees in [0, 360), clockwise from north

- **Method:** theta = atan2(sin dl cos f2, cos f1 sin f_2 − sin f1 cos f2 cos dl)
- **Assumes:** (latitude, longitude) in signed decimal degrees; the start is not a pole; the points are neither equal nor antipodal
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



bearing (initial bearing, degrees in [0, 360), clockwise from north) against lon2 (longitude of the destination) from 9 to 900 on a logarithmic axis. One curve for each value of lat2 (latitude of the destination): 0, 22.5, 67.5. Other inputs as in the worked example: lat1 = 0, lon1 = 0. 33 points per curve computed by geometry-ai999x; marked: the worked example, lon2 = 90, bearing = 90.

```
geometry_ai999x.solve('initial_bearing', lat1=0, lon1=0, lat2=0, lon2=90)
# -> bearing = 90   (SOLVED)
```

```
geometry_ai999x.solve('initial_bearing', lat1=0, lon1=0, lat2=45, lon2=0)
# -> bearing = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the initial bearing from (0, 0) to (0, 90).	SOLVED	bearing = 90
What is the bearing from (10, 20) to (10, 20)?	INVALID_INPUT	Initial bearing (forward azimuth) along a great circle: the points coincide or are antipodal: the initial bearing is not

Area and spherical excess of a spherical triangle, `spherical_triangle_area`

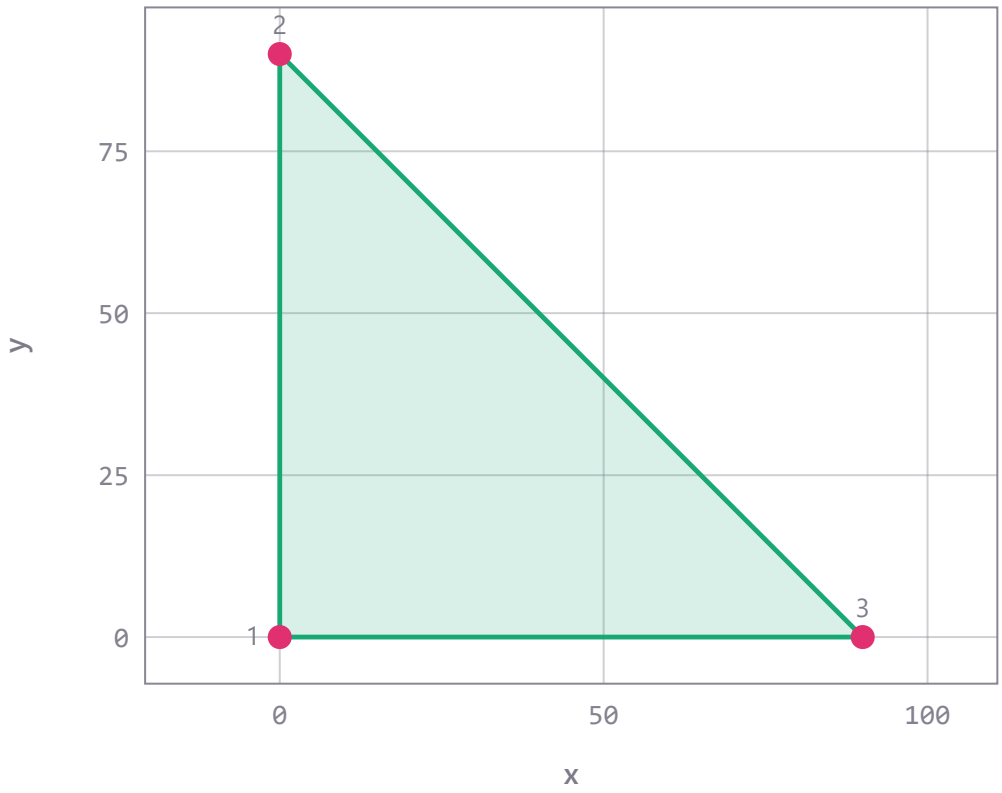
The spherical excess and area of the triangle with three vertices given by latitude and longitude on a sphere of radius R.

$$\tan\frac{E}{4}=\sqrt{\tan\frac{s}{2}\tan\frac{s-a}{2}\tan\frac{s-b}{2}\tan\frac{s-c}{2}},\quad A=ER^2$$

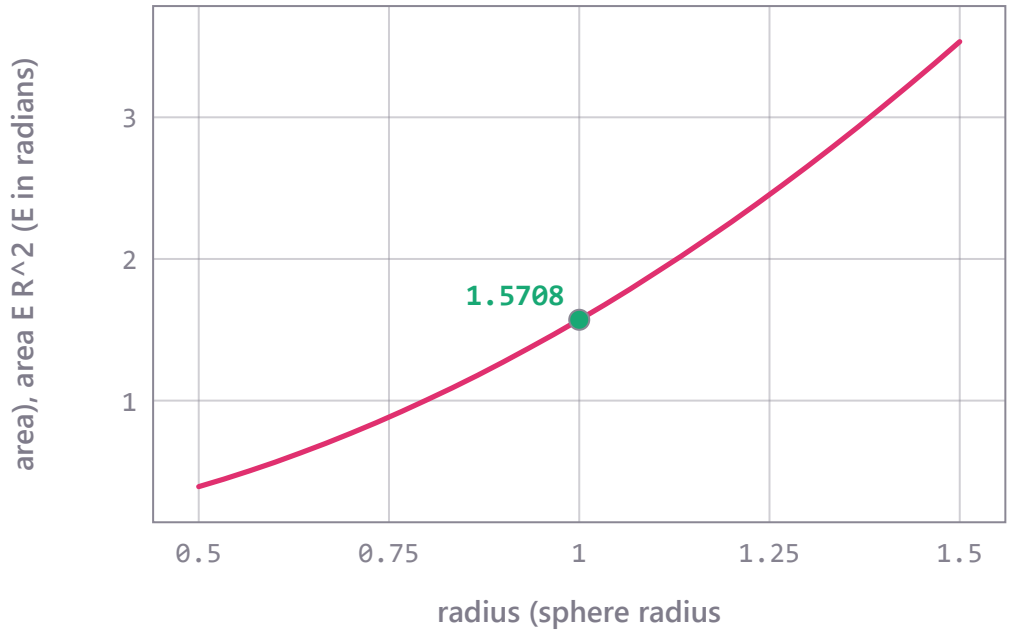
input	meaning	condition
`points`	the vertices (lat lon): lat1 lon1, lat2 lon2, lat3 lon3, degrees	
`radius`	sphere radius (default 1; 'Earth' means 6371 km)	

output	meaning
`excess`	spherical excess $A+B+C-180$, degrees
`area`	area E R^2 (E in radians)

- **Method:** side arcs by the haversine formula, then L'Huilier's theorem
- **Assumes:** the smaller triangle (each side less than 180 degrees); vertices not on one great circle; radius defaults to 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The rows of points (the vertices (lat lon): lat1 lon1, lat2 lon2, lat3 lon3, degrees) drawn as points (numbered in order), joined in order as a polygon. Returned: excess = 90, area = 1.5708. Other inputs: radius = 1.



area (area E R^2 (E in radians)) against radius (sphere radius (default 1; 'Earth' means 6371 km)) from 0.5 to 1.5. Other inputs as in the worked example: points = 0 0, 0 90, 90 0. 31 points computed by geometry-ai999x; marked: the worked example, radius = 1, area = 1.5708.

```
geometry_ai999x.solve('spherical_triangle_area', points='0 0, 0 90, 90 0', radius=1)
# -> excess = 90, area = 1.5708   (SOLVED)
```

```
geometry_ai999x.solve('spherical_triangle_area', points='0 0, 0 90, 90 0', radius=2)
# -> excess = 90, area = 6.28319   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the area of the spherical triangle with vertices (0, 0), (0, 90) and (90, 0) on a sphere of radius 1.	SOLVED	excess = 90, area = 1.5708
Find the area of the spherical triangle with vertices (0, 0), (0, 30) and (0, 60) on a sphere of radius 1.	INVALID_INPUT	Area and spherical excess of a spherical triangle: the vertices lie on one great circle (or two coincide): no spherical

Solid angle of a cone, `solid_angle_cone`

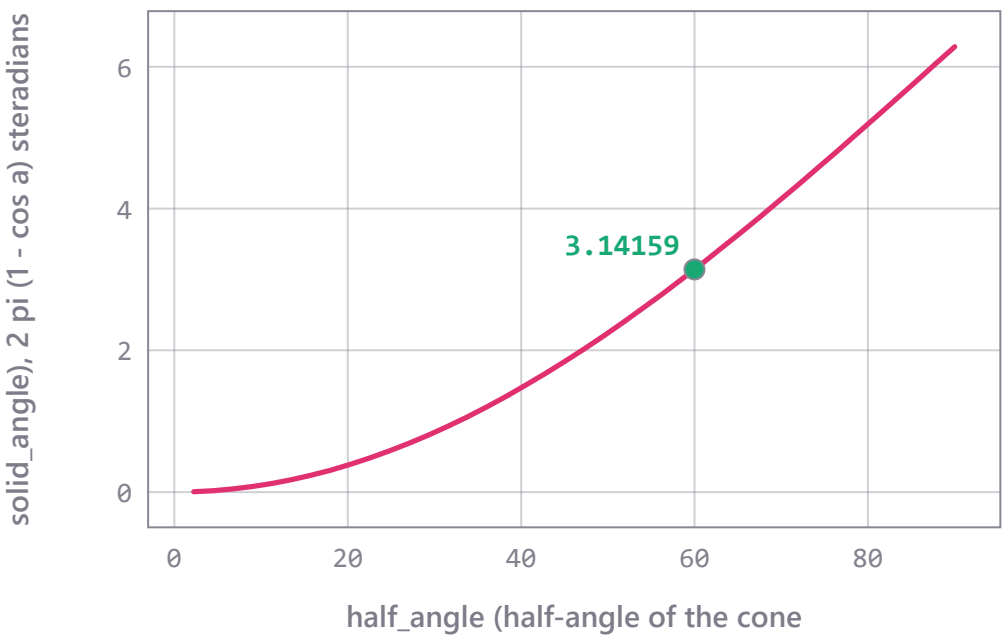
The solid angle (steradians) subtended at the apex by a right circular cone of given half-angle.

$$\Omega = 2\pi(1 - \cos\alpha) = 4\pi \sin^2 \tfrac{\alpha}{2}$$

input	meaning	condition
`half_angle`	half-angle of the cone (axis to side), degrees	

output	meaning
`solid_angle`	2 pi (1 - cos a) steradians
`fraction_of_sphere`	solid angle / (4 pi)

- **Method:** $Omega = 2$ pi (1 - cos a) = 4 pi sin^2(a/2) (no cancellation for small a)
- **Assumes:** 0 < half-angle <= 180 degrees; an apex (full opening) angle in the question is halved
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



solid_angle (2 pi (1 - cos a) steradians) against half_angle (half-angle of the cone (axis to side), degrees) from 0 to 90. 40 points computed by geometry-ai999x; marked: the worked example, half_angle = 60, solid_angle = 3.14159.

```
geometry_ai999x.solve('solid_angle_cone', half_angle=60)
# -> solid_angle = 3.14159, fraction_of_sphere = 0.25   (SOLVED)
```

```
geometry_ai999x.solve('solid_angle_cone', half_angle=90)
# -> solid_angle = 6.28319, fraction_of_sphere = 0.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the solid angle of a cone with half-angle 60 degrees?	SOLVED	solid_angle = 3.14159, fraction_of_sphere = 0.25
Find the solid angle of a cone with apex angle 180 degrees.	SOLVED	solid_angle = 6.28319, fraction_of_sphere = 0.5
What is the solid angle of a cone with half-angle 200 degrees?	INVALID_INPUT	Solid angle of a cone: the half-angle must be between 0 and 180 degrees.

Solid angle subtended by a triangle, `solid_angle_triangle`

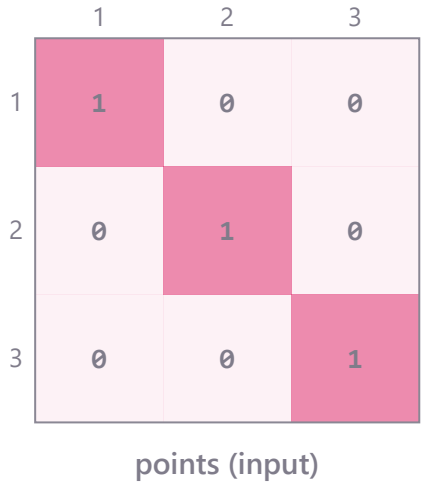
The solid angle (steradians) that the triangle with vertices A, B, C subtends at the origin.

$$\tan\frac{\Omega}{2} = \frac{|\mathbf{a}\cdot(\mathbf{b}\times\mathbf{c})|}{abc + (\mathbf{a}\cdot\mathbf{b})c + (\mathbf{a}\cdot\mathbf{c})b + (\mathbf{b}\cdot\mathbf{c})a}$$

input	meaning	condition
`points`	the vertices as seen from the origin: x y z for each	

output	meaning
`solid_angle`	solid angle, steradians

- **Method:** Van Oosterom-Strackee: tan(Omega/2) = |a.(b x c)| / (abc + (a.b)c + (a.c)b + (b.c)a)
- **Assumes:** no vertex at the origin; the triangle's plane does not pass through the origin; observer at the origin
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



points (input)

The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): points (input). Returned with them by geometry-ai999x: solid_angle = 1.5708.

```
geometry_ai999x.solve('solid_angle_triangle', points='1 0 0, 0 1 0, 0 0 1')
# -> solid_angle = 1.5708   (SOLVED)
```

```
geometry_ai999x.solve('solid_angle_triangle', points='2 0 0, 0 3 0, 0 0 5')
# -> solid_angle = 1.5708   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What solid angle does the triangle with vertices (1, 0, 0), (0, 1, 0), (0, 0, 1) subtend at the origin?	SOLVED	solid_angle = 1.5708
What solid angle does the triangle with vertices (1, 0, 0), (0, 1, 0), (1, 1, 0) subtend at the origin?	INVALID_INPUT	Solid angle subtended by a triangle: the triangle's plane passes through the origin: it subtends no solid angle.

Curvature of a polynomial graph at a point, `curvature_of_graph`

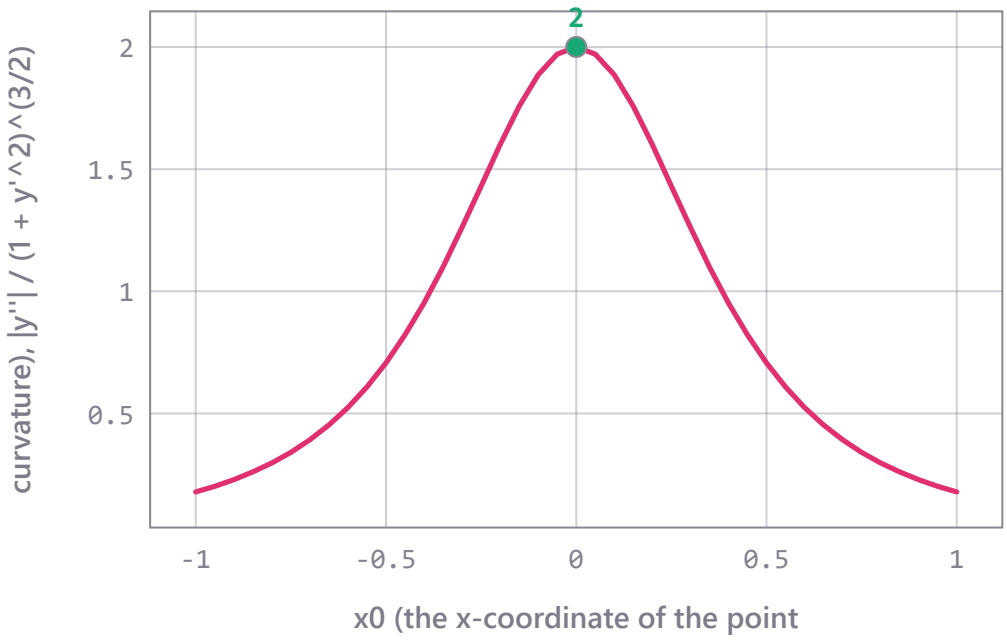
The curvature and radius of curvature of $y = p(x)$ at $x = x_0$ for a polynomial p.

$$\kappa = \frac{|y''|}{(1+y'^2)^{3/2}}, \quad R = \frac{1}{\kappa}$$

input	meaning	condition
`coefficients`	coefficients from the highest power down: a_n, ..., a_1, a_0	
`x0`	the x-coordinate of the point	

output	meaning
`curvature`	|y''| / (1 + y'^2)^(3/2)
`radius_of_curvature`	1 / curvature (none where the curvature is 0)
`slope`	y'(x0)

- **Method:** exact derivatives of the polynomial (Horner), then k = |y''|/(1 + y'^2)^(3/2)
- **Assumes:** polynomial of degree at most 10
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



curvature (|y''| / (1 + y'^2)^(3/2)) against x0 (the x-coordinate of the point) from -1 to 1. Other inputs as in the worked example: coefficients = 1 0 0. 41 points computed by geometry-ai999x; marked: the worked example, x0 = 0, curvature = 2.

```
geometry_ai999x.solve('curvature_of_graph', coefficients='1 0 0', x0=0)
# -> curvature = 2, radius_of_curvature = 0.5, slope = 0   (SOLVED)
```

```
geometry_ai999x.solve('curvature_of_graph', coefficients=[1, 0, -2, 0], x0=1)
# -> curvature = 2.12132, radius_of_curvature = 0.471405, slope = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the curvature of y = x^2 at x = 0.	SOLVED	curvature = 2, radius_of_curvature = 0.5, slope = 0

question	outcome	result
What is the radius of curvature of y = x^3 - 2x at x = 1?	SOLVED	curvature = 2.12132, radius_of_curvature = 0.471405, slope = 1
Find the curvature of y = x^2.	NEEDS_INPUT	curvature_of_graph needs x0 (the x-coordinate of the point). Example: solve('curvature_of_graph', coefficients='1 0 0',

Helix: curvature, torsion and length per turn, `helix_measures`

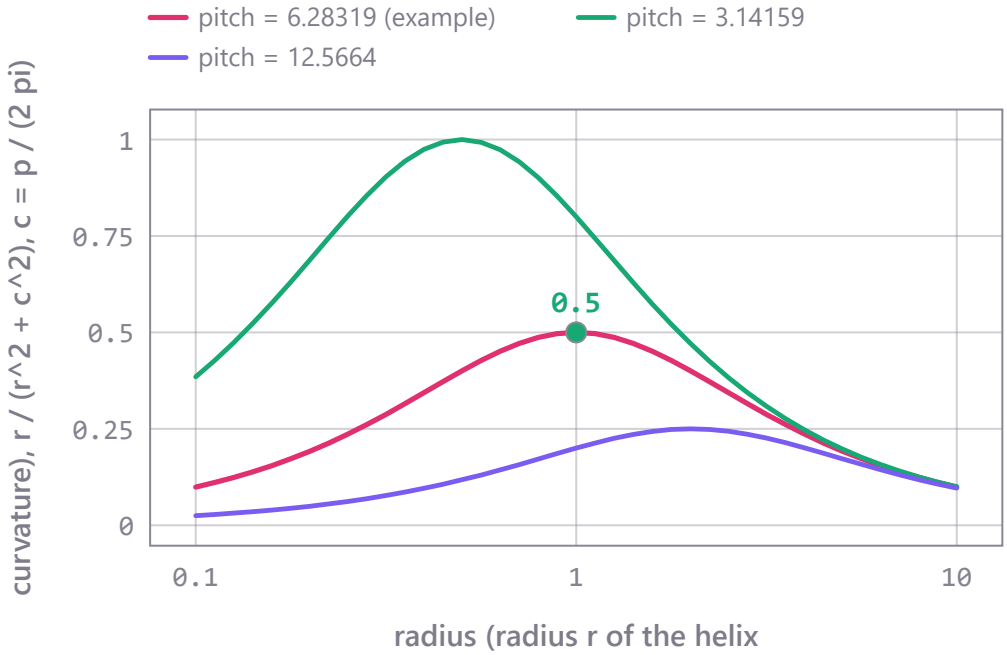
Curvature, torsion and arc length per turn of the circular helix of radius r and pitch p (rise per turn).

$$\kappa = \frac{r}{r^2 + c^2}, \quad \tau = \frac{c}{r^2 + c^2}, \quad c = \frac{p}{2\pi}$$

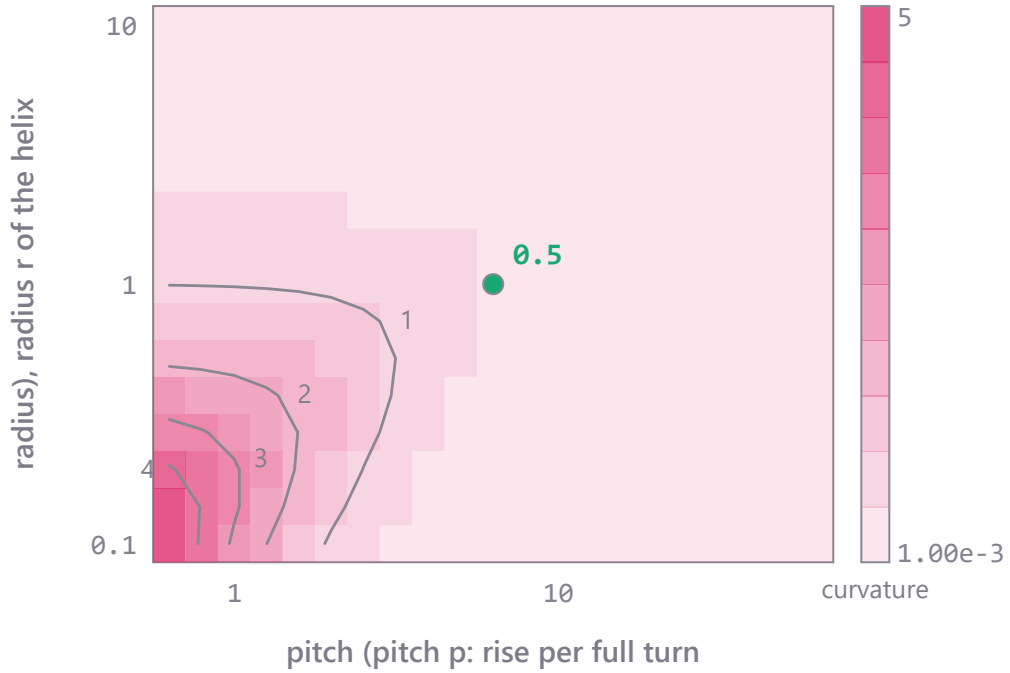
input	meaning	condition
`radius`	radius r of the helix	
`pitch`	pitch p: rise per full turn (negative: left-handed)	

output	meaning
`curvature`	$r/(r^2 + c^2)$, c = p / (2 pi)
`torsion`	$c/(r^2 + c^2)$
`length_per_turn`	sqrt((2 pi r)^2 + p^2)

- **Method:** Frenet-Serret closed forms for the circular helix
- **Assumes:** radius > 0; parametrised (r cos t, r sin t, c t)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



curvature (r / (r^2 + c^2), c = p / (2 pi)) against radius (radius r of the helix) from 0.1 to 10 on a logarithmic axis. One curve for each value of pitch (pitch p: rise per full turn (negative: left-handed)): 6.28319, 3.14159, 12.5664. 41 points per curve computed by geometry-ai999x; marked: the worked example, radius = 1, curvature = 0.5.



curvature (r / (r^2 + c^2), c = p / (2 pi)) over pitch (pitch p: rise per full turn (negative: left-handed)) from 0.628319 to 62.8319 and radius (radius r of the helix) from 0.1 to 10 (logarithmic axes): each of the 21 × 15 cells is one run of geometry-ai999x, shaded by its value (light 0.0009999 to dark 5); iso-lines at curvature = 1, 2, 3, 4. Marked: the worked example, pitch = 6.28319, radius = 1, curvature = 0.5.

```
geometry_ai999x.solve('helix_measures', radius=1, pitch=6.283185307179586)
# -> curvature = 0.5, torsion = 0.5, length_per_turn = 8.88577   (SOLVED)
```

```
geometry_ai999x.solve('helix_measures', radius=3, pitch=25.132741228718345)
# -> curvature = 0.12, torsion = 0.16, length_per_turn = 31.4159   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the curvature and torsion of a helix with radius 1 and pitch 6.283185307179586.	SOLVED	curvature = 0.5, torsion = 0.5, length_per_turn = 8.88577
A spring is a helix of radius 3 cm and pitch 25.132741228718345 cm. Find its curvature.	SOLVED	curvature = 0.12, torsion = 0.16, length_per_turn = 31.4159
Find the curvature of a helix with radius 2.	NEEDS_INPUT	helix_measures needs pitch (pitch p: rise per full turn (negative: left-handed)). Example: solve('helix_measures', radiu

Curvature of an ellipse at a point, `ellipse_curvature`

The curvature and radius of curvature of the ellipse (a cos t, b sin t) at parameter t (eccentric anomaly).

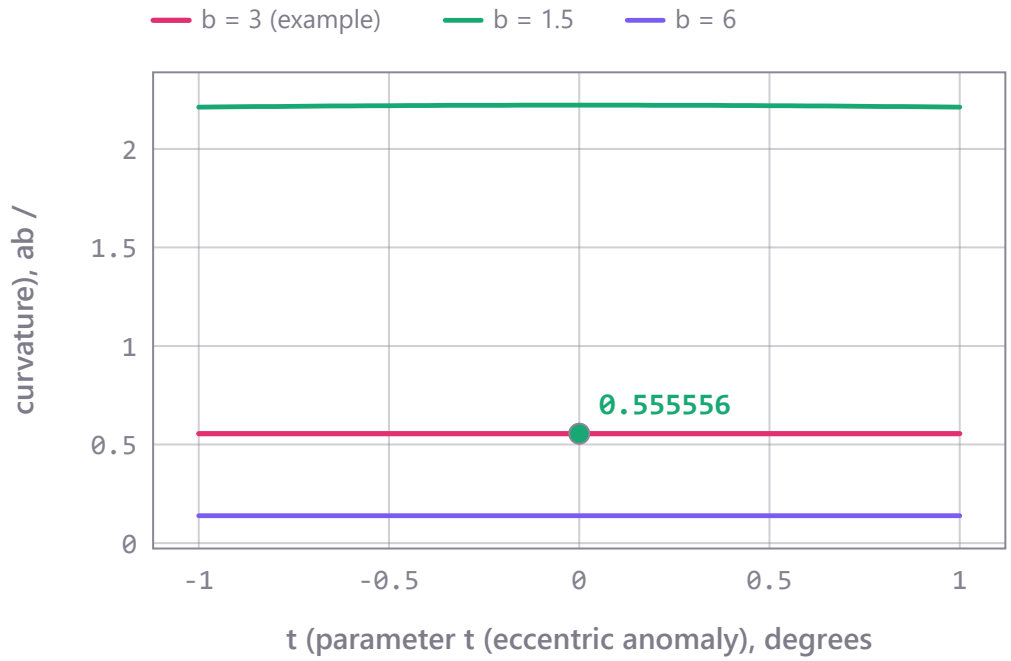
$$\kappa(t) = \frac{ab}{(a^2 \sin^2 t + b^2 \cos^2 t)^{3/2}}$$

input	meaning	condition
a	semi-axis along x	

input	meaning	condition
`b`	semi-axis along y	
`t`	parameter t (eccentric anomaly), degrees	

output	meaning
`curvature`	ab / (a^2 $sin^2\ t + b^2\ cos^2$ t)^(3/2)
`radius_of_curvature`	1 / curvature
`x`	x of the point
`y`	y of the point

- **Method:** parametric curvature |x'y'' - y'x''| / (x'^$2 + y$'^2)^(3/2) in closed form
- **Assumes:** a, $b > 0$; the point is (a cos t, b sin t); 'end of the major/minor axis' means $t = 0/90$ degrees
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



curvature (ab / (a^2 sin^2 t + b^2 cos^2 t)^(3/2)) against t (parameter t (eccentric anomaly), degrees) from -1 to 1. One curve for each value of b (semi-axis along y): 3, 1.5, 6. Other inputs as in the worked example: a = 5. 41 points per curve computed by geometry-ai999x; marked: the worked example, t = 0, curvature = 0.555556.

```
geometry_ai999x.solve('ellipse_curvature', a=5, b=3, t=0)
# -> curvature = 0.555556, radius_of_curvature = 1.8, x = 5, y = 0   (SOLVED)
```

```
geometry_ai999x.solve('ellipse_curvature', a=5, b=3, t=90)
# -> curvature = 0.12, radius_of_curvature = 8.33333, x = 3.06162e-16, y = 3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the curvature of the ellipse with semi-major axis 5 and semi-minor axis 3 at the end of the major axis.	SOLVED	curvature = 0.555556, radius_of_curvature = 1.8, x = 5, y = 0

question	outcome	result
Find the radius of curvature of the ellipse x^2/25 + y^2/9 = 1 at the end of the minor axis.	SOLVED	curvature = 0.12, radius_of_curvature = 8.33333, x = 3.06162e-16, y = 3

Gaussian and mean curvature of a quadratic surface z = f(x, y), `surface_curvature`

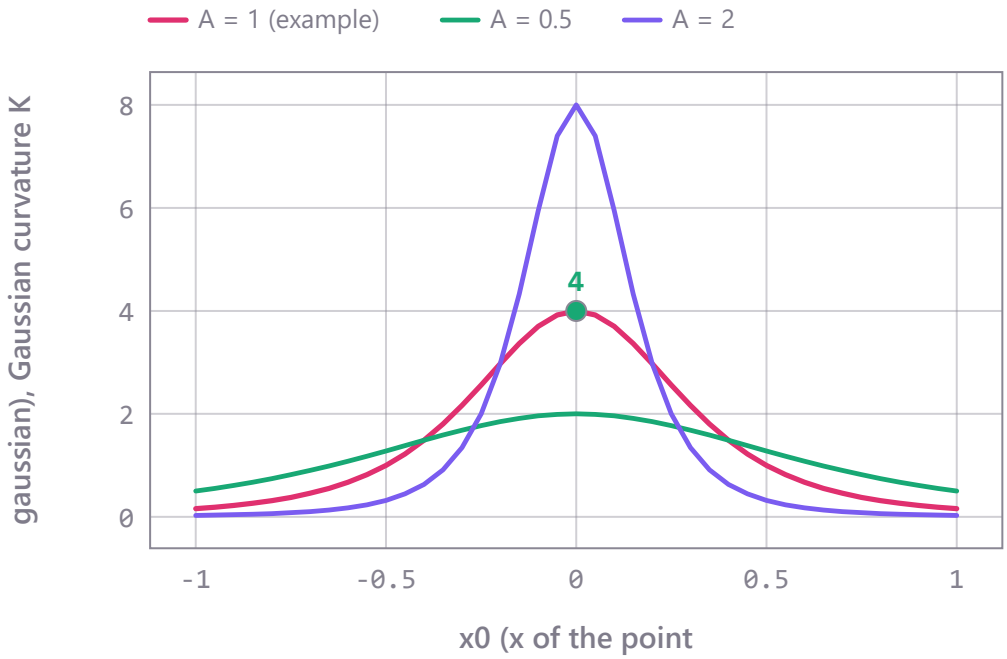
Gaussian curvature K and mean curvature H of $z = A\,x^2 + B$ xy + C $y^2 + D$ x + E y + F at the point above (x0, y0).

$$K = \frac{f_{xx} f_{yy} - f_{xy}^2}{(1 + f_x^2 + f_y^2)^2}, \quad H = \frac{(1 + f_y^2) f_{xx} - 2 f_x f_y f_{xy} + (1 + f_x^2) f_{yy}}{2(1 + f_x^2 + f_y^2)^{3/2}}$$

input	meaning	condition
`A`	coefficient of x^2	
`B`	coefficient of xy	
`C`	coefficient of y^2	
`D`	coefficient of x	
`E`	coefficient of y	
`x0`	x of the point	
`y0`	y of the point	
`F`	constant term (does not affect curvature)	

output	meaning
`gaussian`	Gaussian curvature K
`mean`	mean curvature H (upward normal: positive where the surface opens upward)

- **Method:** Monge patch: K = (f_xx f_yy - f_xy^2)/(1 + f_x^2 + f_y^2)^2, H = ($(1 + f_y^2)\, f_x x - 2$ f_x f_y $f_x y + (1 + f_x^2)\, f_y y)/(2\,(1 + f_x^2 + f_y^2)^{3/2}$)
- **Assumes:** the unit normal points upward, (-f_x, -f_y, 1)/|...|
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



gaussian (Gaussian curvature K) against x0 (x of the point) from -1 to 1. One curve for each value of A (coefficient of x^2): 1, 0.5, 2. Other inputs as in the worked example: B = 0, C = 1, D = 0, E = 0, y0 = 0. 41 points per curve computed by geometry-ai999x; marked: the worked example, x0 = 0, gaussian = 4.

```
geometry_ai999x.solve('surface_curvature', A=1, B=0, C=1, D=0, E=0, x0=0, y0=0)
# -> gaussian = 4, mean = 2   (SOLVED)
```

```
geometry_ai999x.solve('surface_curvature', A=1, B=0, C=-1, D=0, E=0, x0=0, y0=0)
# -> gaussian = -4, mean = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Gaussian and mean curvature of z = x^2 + y^2 at (0, 0).	SOLVED	gaussian = 4, mean = 2
Find the Gaussian curvature of z = x^2 - y^2 + 2x at (1, 1).	SOLVED	gaussian = -0.00907029, mean = -0.124696
Find the Gaussian curvature of z = x^2 + y^2.	NEEDS_INPUT	surface_curvature needs x0, y0 (x of the point; y of the point). Example: solve('surface_curvature', A=1, B=0, C=1, D=0,

Curvature of a torus at a point, `torus_curvature`

Gaussian, mean and principal curvatures of a ring torus at tube angle v (v = 0 on the outer equator, 180 on the inner).

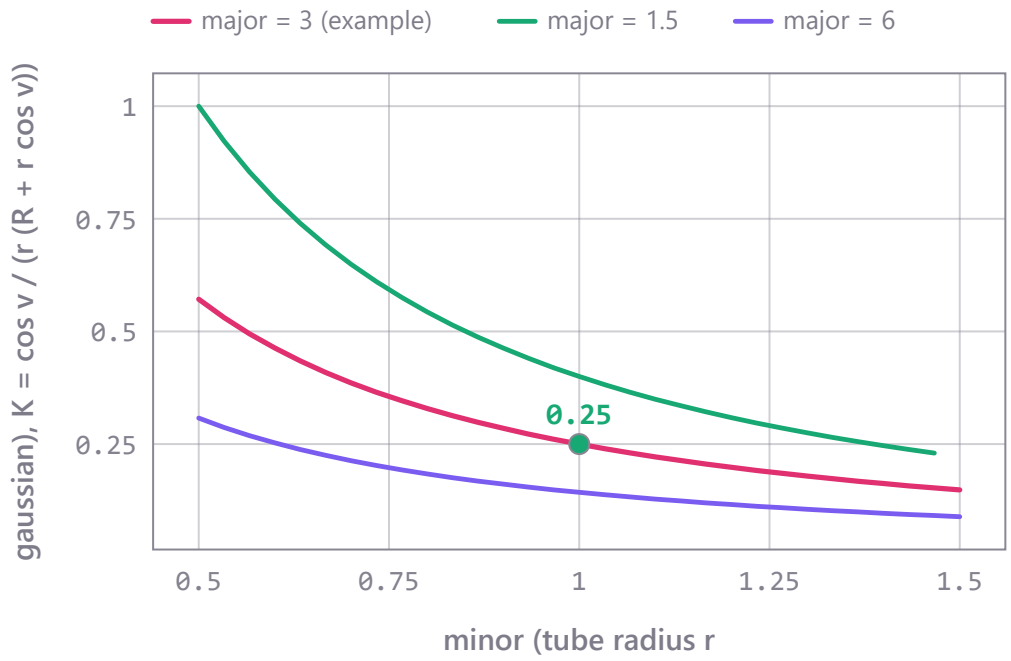
$$K = \frac{\cos v}{r(R + r\cos v)}, \quad H = \frac{R + 2r\cos v}{2r(R + r\cos v)}$$

input	meaning	condition
`major`	major radius R	
`minor`	tube radius r	

input	meaning	condition
`v`	tube angle v, degrees (0 = outer equator)	

output	meaning
`gaussian`	$K = cos$ v / (r (R + r cos v))
`mean`	H = (R + 2 r cos $v)/(2$ r (R + r cos v))
`k1`	principal curvature around the tube, $1/r$
`k2`	principal curvature along the ring, cos v / (R + r cos v)

- **Method:** principal curvatures of the surface of revolution: $1/r$ around the tube and cos v/(R + r cos v) along the ring
- **Assumes:** $0 < r < R$; signs: curvatures positive where the surface bends away from the outward normal (a sphere is positive)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



gaussian (K = cos v / (r (R + r cos v))) against minor (tube radius r) from 0.5 to 1.5. One curve for each value of major (major radius R): 3, 1.5, 6. Other inputs as in the worked example: v = 0. 31 points per curve computed by geometry-ai999x; marked: the worked example, minor = 1, gaussian = 0.25.

```
geometry_ai999x.solve('torus_curvature', major=3, minor=1, v=0)
# -> gaussian = 0.25, mean = 0.625, k1 = 1, k2 = 0.25   (SOLVED)
```

```
geometry_ai999x.solve('torus_curvature', major=3, minor=1, v=180)
# -> gaussian = -0.5, mean = 0.25, k1 = 1, k2 = -0.5   (SOLVED)
```

```
geometry_ai999x.solve('torus_curvature', major=3, minor=1, v=90)
# -> gaussian = 2.04108e-17, mean = 0.5, k1 = 1, k2 = 2.04108e-17   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Gaussian curvature of a torus with major radius 3 and minor radius 1 at tube angle 0 degrees.	SOLVED	gaussian = 0.25, mean = 0.625, k1 = 1, k2 = 0.25
Find the mean curvature of a torus with major radius 3 and minor radius 1 at tube angle 180 degrees.	SOLVED	gaussian = -0.5, mean = 0.25, k1 = 1, k2 = -0.5

Arc length of a parabola, `parabola_arc_length`

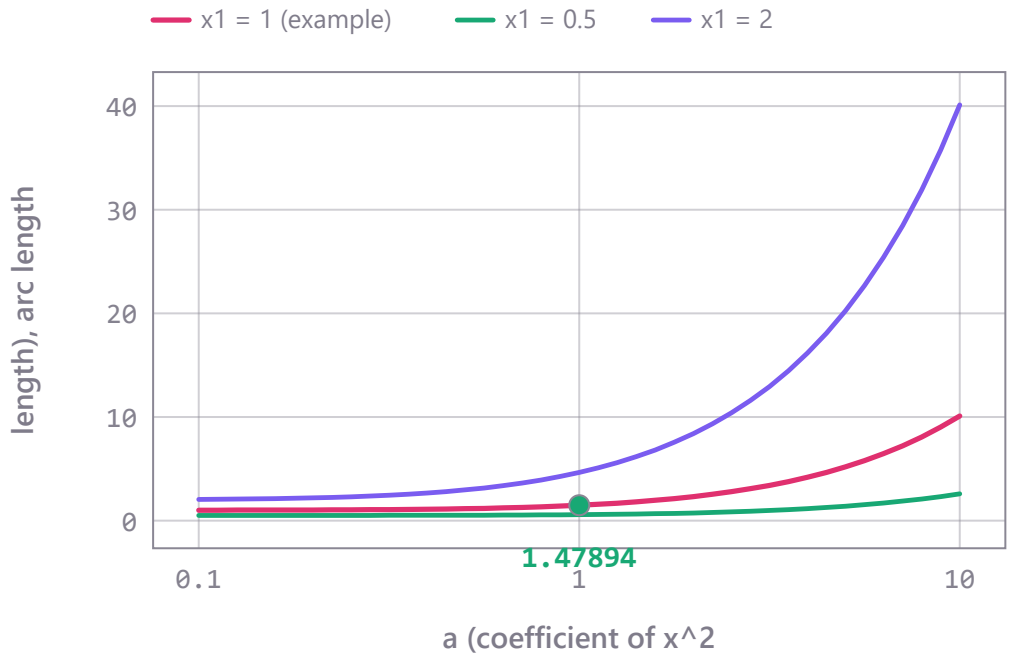
The length of the curve $y = a\,x^2 + b$ x + c between $x = x_0$ and $x = x_1$.

$$L = \frac{1}{2a}\left[\tfrac{1}{2}\left(u\sqrt{1+u^2} + \operatorname{arsinh} u\right)\right]_{u_0}^{u_1}, \quad u = 2ax + b$$

input	meaning	condition
`a`	coefficient of x^2	
`b`	coefficient of x	
`x0`	start x	
`x1`	end x	
`c`	constant (does not affect the length)	

output	meaning
`length`	arc length

- **Method:** closed form of the integral of sqrt(1 + u^2), $u = 2ax + b$: (u sqrt(1+u^2) + asinh u)/2, divided by 2a
- **Assumes:** $x_0 \neq x_1$ (either order)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



length (arc length) against a (coefficient of x^2) from 0.1 to 10 on a logarithmic axis. One curve for each value of x1 (end x): 1, 0.5, 2. Other inputs as in the worked example: b = 0, x0 = 0. 41 points per curve computed by geometry-ai999x; marked: the worked example, a = 1, length = 1.47894.

```
geometry_ai999x.solve('parabola_arc_length', a=1, b=0, x0=0, x1=1)
# -> length = 1.47894   (SOLVED)
```

```
geometry_ai999x.solve('parabola_arc_length', a=0, b=1, x0=0, x1=3)
# -> length = 4.24264   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the arc length of y = x^2 from x = 0 to x = 1.	SOLVED	length = 1.47894
Find the arc length of y = x^2 + 3 from x = 2 to x = 2.	INVALID_INPUT	Arc length of a parabola: the interval is empty (x0 = x1).

Smallest enclosing circle of points, `min_enclosing_circle`

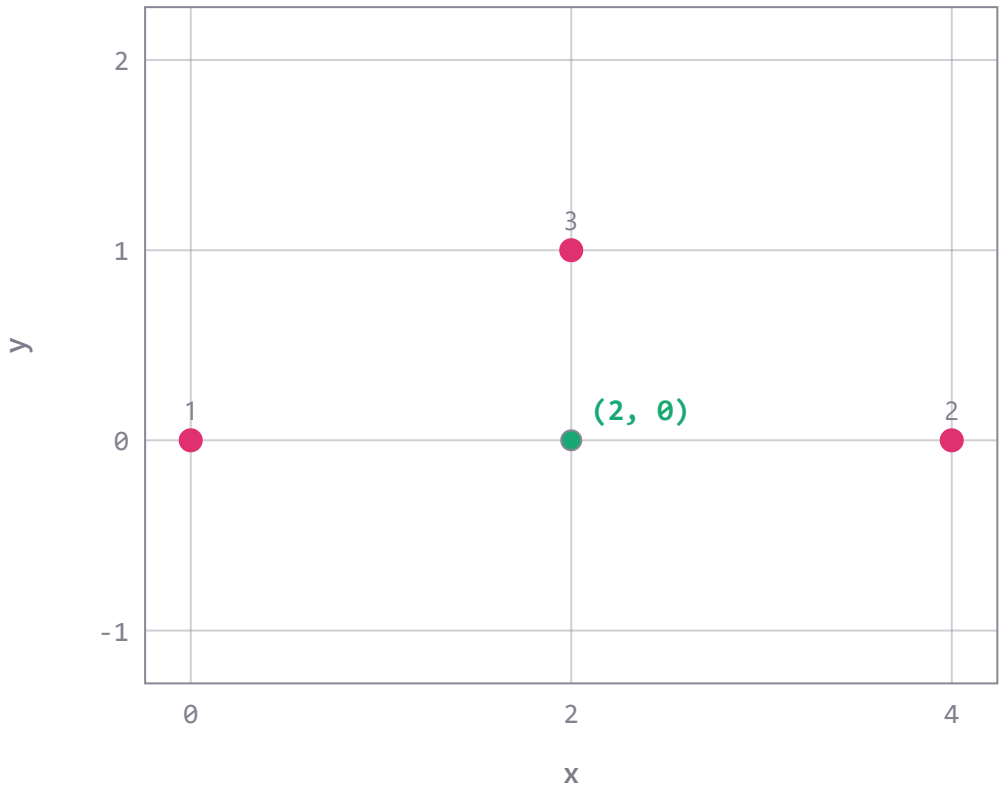
Centre and radius of the smallest circle that contains every given point.

$$\min_{c,r} r \;\; \text{s.t. } \|p_i - c\| \le r \;\forall i$$

input	meaning	condition
`points`	the points: x1 y1, x2 y2, ...	

output	meaning
`cx`	x of the centre
`cy`	y of the centre
`radius`	radius

- **Method:** Welzl's incremental algorithm (fixed shuffled order), circles through 2 or 3 support points
- **Assumes:** 1 to 2000 points; the smallest enclosing circle is unique
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The rows of points (the points: x1 y1, x2 y2, ...) drawn as points (numbered in order), returned (x, y) pairs marked. Returned: cx = 2, cy = 0, radius = 2.

```
geometry_ai999x.solve('min_enclosing_circle', points='0 0, 4 0, 2 1')
# -> cx = 2, cy = 0, radius = 2   (SOLVED)
```

```
geometry_ai999x.solve('min_enclosing_circle', points='0 0, 2 0, 1 2')
# -> cx = 1, cy = 0.75, radius = 1.25   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the smallest enclosing circle of the points (0, 0), (4, 0), (2, 1).	SOLVED	cx = 2, cy = 0, radius = 2
What is the smallest circle containing the points (0, 0), (2, 0) and (1, 2)?	SOLVED	cx = 1, cy = 0.75, radius = 1.25

Minimum-area enclosing rectangle, `min_area_rectangle`

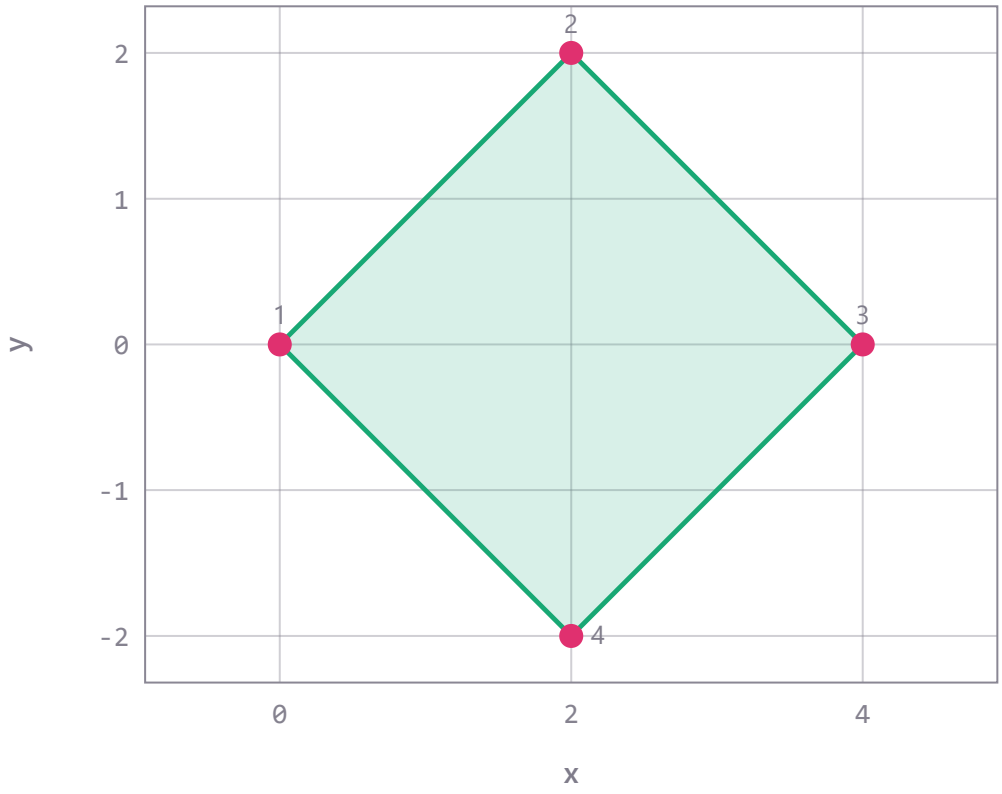
The area (and sides) of the smallest-area rectangle, in any orientation, enclosing the points.

$$A^* = \min_{e \in \text{hull edges}} w_e \, h_e$$

input	meaning	condition
`points`	the points: x1 y1, x2 y2, ...	

output	meaning
`area`	minimum area
`width`	shorter side of that rectangle
`length`	longer side

- **Method:** convex hull, then one side of the optimal rectangle lies along a hull edge: try every edge (rotating calipers)
- **Assumes:** at least 3 points, not all collinear; when two orientations tie in area, the sides of the first found are reported
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The rows of points (the points: x1 y1, x2 y2, ...) drawn as points (numbered in order), joined in order as a polygon. Returned: area = 8, width = 2.82843, length = 2.82843.

```
geometry_ai999x.solve('min_area_rectangle', points='0 0, 2 2, 4 0, 2 -2')
# -> area = 8, width = 2.82843, length = 2.82843   (SOLVED)
```

```
geometry_ai999x.solve('min_area_rectangle', points='0 0, 3 0, 3 1, 0 1, 1 0.5')
# -> area = 3, width = 1, length = 3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the minimum area bounding rectangle of the points (0, 0), (2, 2), (4, 0), (2, -2).	SOLVED	area = 8, width = 2.82843, length = 2.82843
Find the minimum area bounding rectangle of the points (0, 0), (1, 1), (2, 2).	INVALID_INPUT	Minimum-area enclosing rectangle: the points are collinear: the enclosing rectangle has no area.

Delaunay triangulation and Voronoi diagram: counts, `delaunay_voronoi_counts`

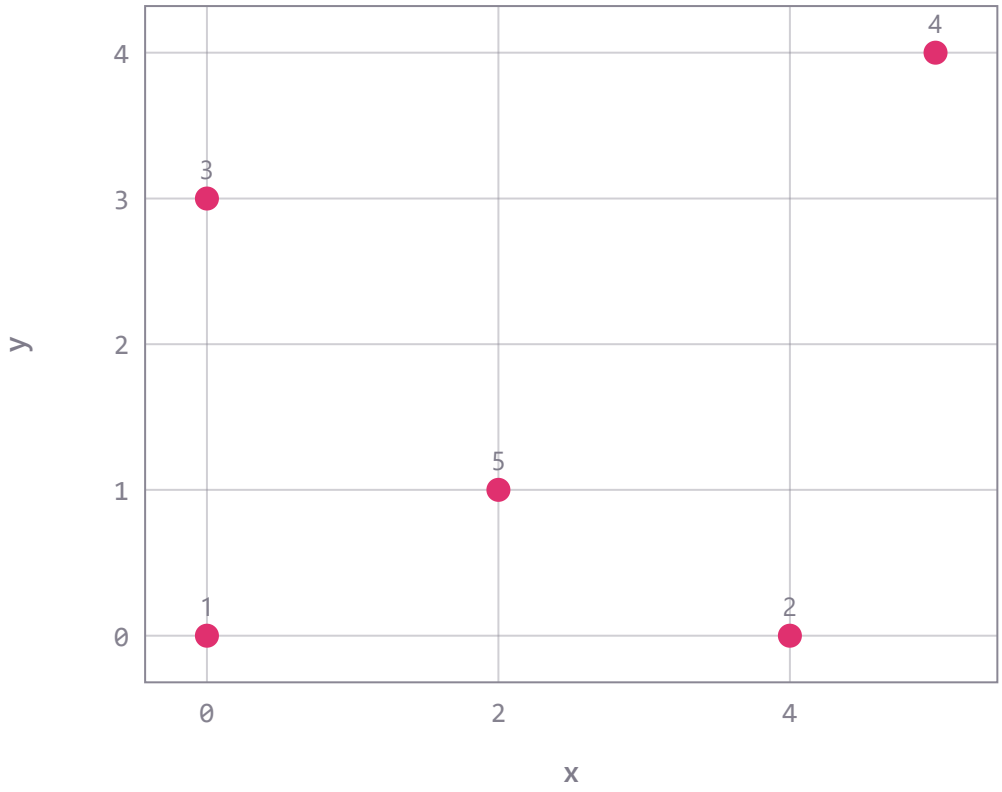
Numbers of Delaunay triangles and edges, hull vertices, Voronoi vertices and unbounded Voronoi cells of points in general position.

$$T = 2n - 2 - h, \quad E = 3n - 3 - h$$

input	meaning	condition
`points`	the points: x1 y1, x2 y2, ... (3 to 25)	

output	meaning
`triangles`	Delaunay triangles $2n - 2 - h$
`edges`	Delaunay edges $3n - 3 - h$
`hull_vertices`	h, vertices of the convex hull
`voronoi_vertices`	Voronoi vertices (one per triangle)
`unbounded_cells`	unbounded Voronoi cells (one per hull vertex)

- **Method:** exact general-position checks, the convex hull, then Euler's formula for a triangulation; Voronoi is the dual
- **Assumes:** 3 to 25 distinct points; general position: no three collinear, no four concyclic (checked exactly)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The rows of points (the points: x1 y1, x2 y2, ... (3 to 25)) drawn as points (numbered in order). Returned: triangles = 4, edges = 8, hull_vertices = 4, voronoi_vertices = 4, unbounded_cells = 4.

```
geometry_ai999x.solve('delaunay_voronoi_counts', points='0 0, 4 0, 0 3, 5 4, 2 1')
# -> triangles = 4, edges = 8, hull_vertices = 4, voronoi_vertices = 4, unbounded_cells = 4
(SOLVED)
```

```
geometry_ai999x.solve('delaunay_voronoi_counts', points='0 0, 1 0, 0 1')
# -> triangles = 1, edges = 3, hull_vertices = 3, voronoi_vertices = 1, unbounded_cells = 3
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
How many triangles are in the Delaunay triangulation of the points (0, 0), (4, 0), (0, 3), (5, 4), (2, 1)?	SOLVED	triangles = 4, edges = 8, hull_vertices = 4, voronoi_vertices = 4, unbounded_cells = 4
Count the Voronoi vertices for the points (0, 0), (1, 0), (1, 1), (0, 1).	INVALID_INPUT	Delaunay triangulation and Voronoi diagram: counts: four points are concyclic: the Delaunay triangulation is not unique.

Area of the intersection of two convex polygons, `convex_polygon_intersection`

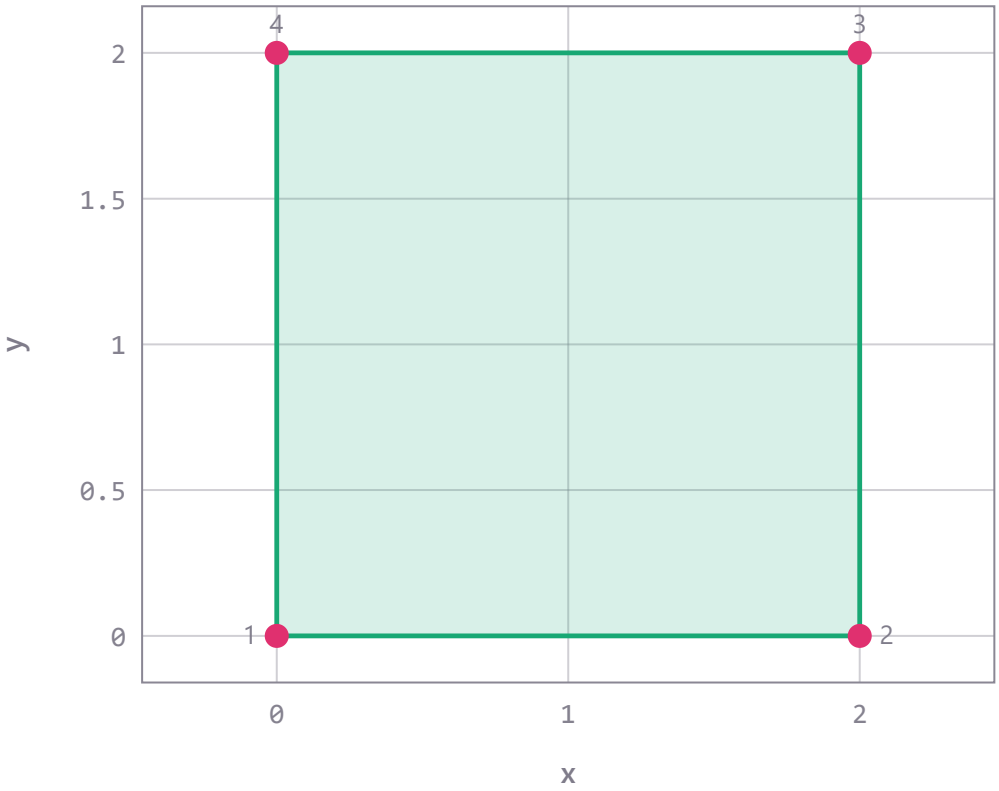
The area common to two convex polygons given by their vertices.

$$A(P \cap Q)$$

input	meaning	condition
`p`	first convex polygon's vertices in order	
`q`	second convex polygon's vertices in order	

output	meaning
`area`	area of the intersection (0 if they do not overlap)

- **Method:** Sutherland-Hodgman clipping of one polygon by the other's edges, then the shoelace area
- **Assumes:** both polygons simple and convex
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The rows of p (first convex polygon's vertices in order) drawn as points (numbered in order), joined in order as a polygon. Returned: area = 1. Other inputs: q = 1 1, 3 1, 3 3, 1 3.

```
geometry_ai999x.solve('convex_polygon_intersection', p='0 0, 2 0, 2 2, 0 2', q='1 1, 3 1, 3 3, 1
3')
# -> area = 1   (SOLVED)
```

```
geometry_ai999x.solve('convex_polygon_intersection', p='0 0, 1 0, 0 1', q='5 5, 6 5, 5 6')
# -> area = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the area of intersection of the polygon with vertices (0, 0), (2, 0), (2, 2), (0, 2) and the polygon with vertices (1, 1), (3, 1), (3, 3), (1, 3).	SOLVED	area = 1

question	outcome	result
Find the overlap of the polygon with vertices (0, 0), (4, 0), (4, 4), (2, 1), (0, 4) and the polygon with vertices (1, 1), (3, 1), (3, 3).	INVALID_INPUT	Area of the intersection of two convex polygons: p: the polygon is not convex.

Hausdorff distance between two point sets, `hausdorff_distance`

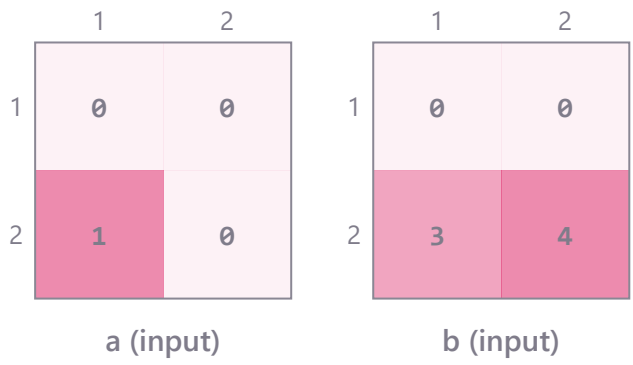
The Hausdorff distance between two finite point sets and both directed distances.

$$d_H(A,B) = \max\{\sup_a \inf_b \|a-b\|,\ \sup_b \inf_a \|a-b\|\}$$

input	meaning	condition
`a`	first set: x1 y1, x2 y2, ...	
`b`	second set	

output	meaning
`distance`	max of the two directed distances
`from_a`	max over A of the distance to the nearest point of B
`from_b`	max over B of the distance to the nearest point of A

- **Method:** directed distances by exhaustive nearest-point search
- **Assumes:** 1 to 1000 points per set
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): a (input); b (input). Returned with them by geometry-ai999x: distance = 4.47214, from_a = 1, from_b = 4.47214.

```
geometry_ai999x.solve('hausdorff_distance', a='0 0, 1 0', b='0 0, 3 4')
# -> distance = 4.47214, from_a = 1, from_b = 4.47214   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Hausdorff distance between the set (0, 0), (1, 0) and the set (0, 0), (3, 4).	SOLVED	distance = 4.47214, from_a = 1, from_b = 4.47214

Nearest neighbour of a query point, `nearest_neighbor`

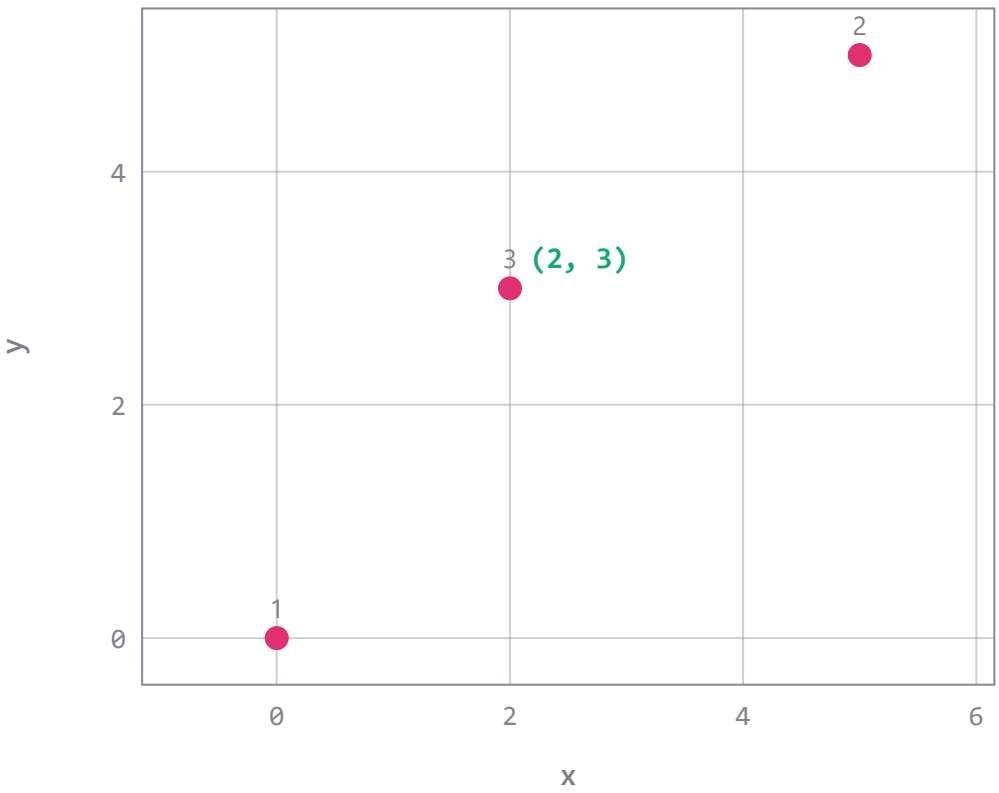
Which of a list of points is nearest to a query point, and how far it is.

$$k^* = \arg\min_k \|p_k - q\|$$

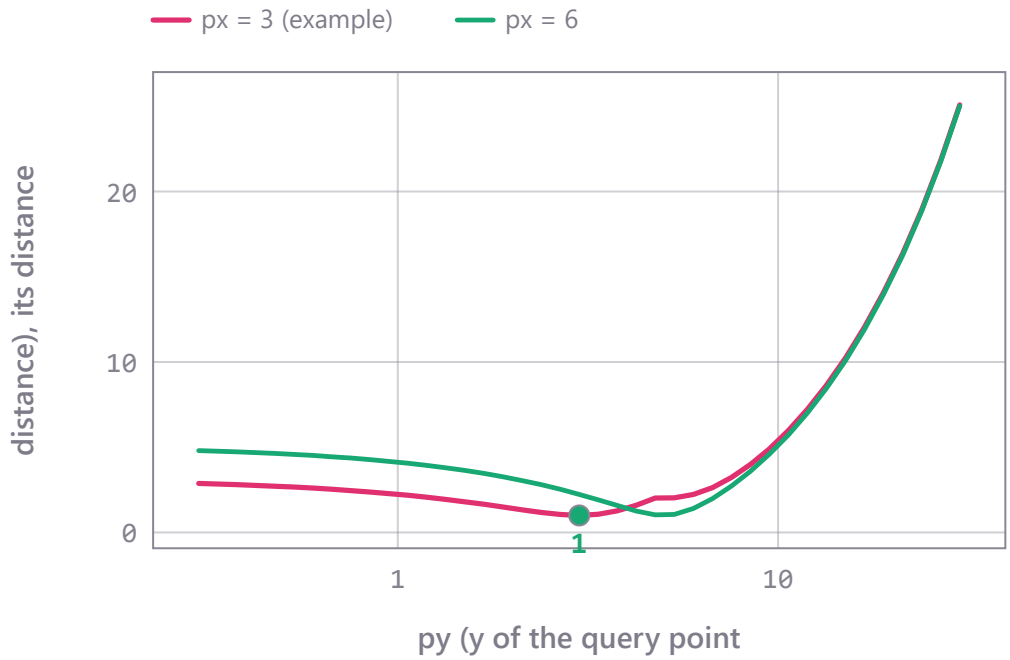
input	meaning	condition
`px`	x of the query point	
`py`	y of the query point	
`points`	the candidate points: x1 y1, x2 y2, ...	

output	meaning
`index`	position of the nearest point in the list (1 = first)
`distance`	its distance
`nx`	its x
`ny`	its y

- **Method:** exhaustive search on squared distances
- **Assumes:** ties go to the earliest point in the list
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The rows of points (the candidate points: x1 y1, x2 y2, ...) drawn as points (numbered in order), returned (x, y) pairs marked. Returned: index = 3, distance = 1, nx = 2, ny = 3. Other inputs: px = 3, py = 3.



distance (its distance) against py (y of the query point) from 0.3 to 30 on a logarithmic axis. One curve for each value of px (x of the query point): 3, 6. Other inputs as in the worked example: points = 0 0, 5 5, 2 3. 41 points per curve computed by geometry-ai999x; marked: the worked example, py = 3, distance = 1.

```
geometry_ai999x.solve('nearest_neighbor', px=3, py=3, points='0 0, 5 5, 2 3')
# -> index = 3, distance = 1, nx = 2, ny = 3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Which of the points (0, 0), (5, 5), (2, 3) is nearest to the point (3, 3)?	SOLVED	index = 3, distance = 1, nx = 2, ny = 3

Barycentric coordinates of a point in a triangle, `barycentric_coordinates`

The barycentric coordinates (l1, l2, l3) of a point with respect to triangle ABC: $P = l_1\,A + l_2\,B + l_3$ C, $l_1 + l_2 + l_3 = 1$.

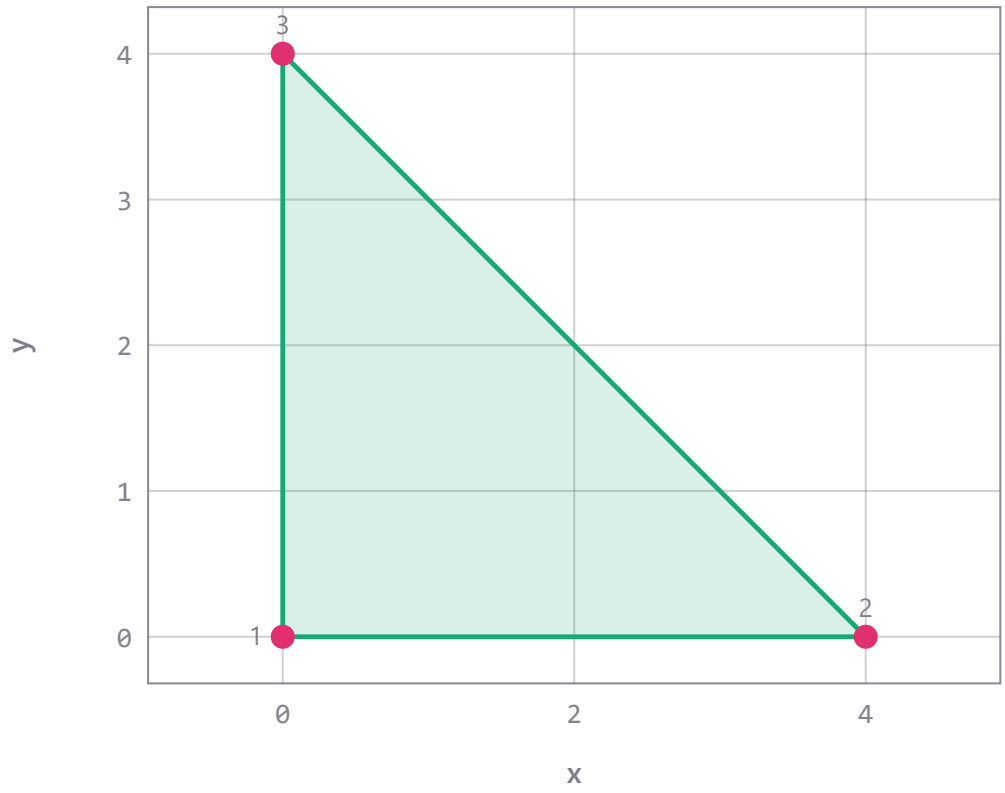
$$\lambda_1 = \frac{[PBC]}{[ABC]},\ \lambda_2 = \frac{[APC]}{[ABC]},\ \lambda_3 = \frac{[ABP]}{[ABC]}$$

input	meaning	condition
`px`	x of the point	
`py`	y of the point	
`points`	the triangle's vertices A, B, C	

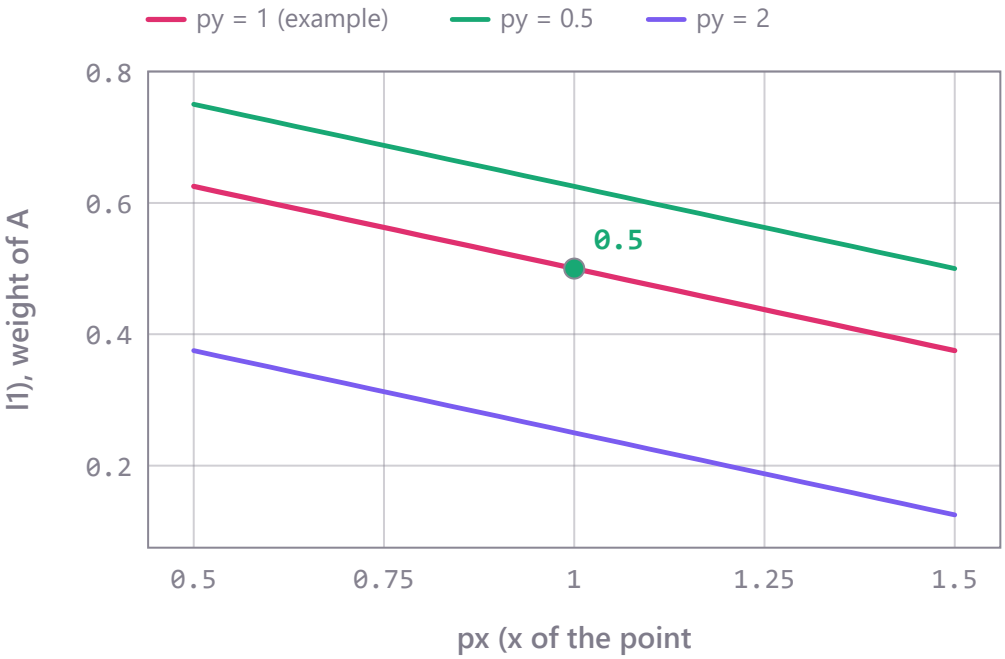
output	meaning
`l1`	weight of A
`l2`	weight of B
`l3`	weight of C

- **Method:** ratios of signed areas: l1 = [PBC]/[ABC], l2 = [APC]/[ABC], l3 = [ABP]/[ABC]

- **Assumes:** A, B, C not collinear; all weights positive exactly when P is inside
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The rows of points (the triangle's vertices A, B, C) drawn as points (numbered in order), joined in order as a polygon. Returned: l1 = 0.5, l2 = 0.25, l3 = 0.25. Other inputs: px = 1, py = 1.



l1 (weight of A) against px (x of the point) from 0.5 to 1.5. One curve for each value of py (y of the point): 1, 0.5, 2. Other inputs as in the worked example: points = 0 0, 4 0, 0 4. 31 points per curve computed by geometry-ai999x; marked: the worked example, px = 1, l1 = 0.5.

```
geometry_ai999x.solve('barycentric_coordinates', px=1, py=1, points='0 0, 4 0, 0 4')
# -> l1 = 0.5, l2 = 0.25, l3 = 0.25   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the barycentric coordinates of the point (1, 1) in the triangle with vertices (0, 0), (4, 0), (0, 4).	SOLVED	l1 = 0.5, l2 = 0.25, l3 = 0.25

Interior angles of a polygon from its vertices, `polygon_interior_angles`

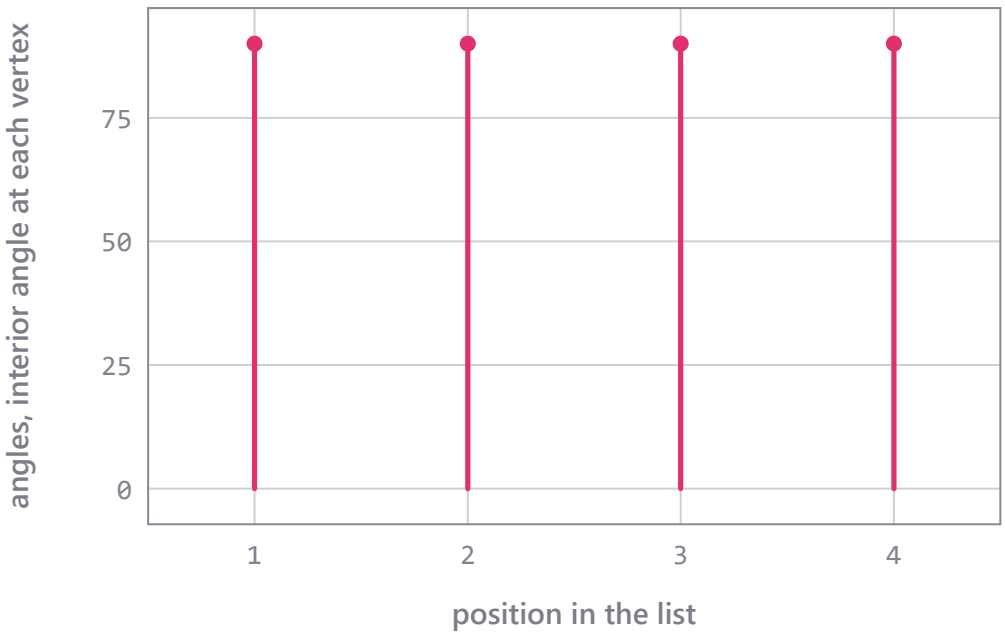
The interior angle at every vertex of a simple polygon (reflex angles included), their sum and the number of reflex vertices.

$$\theta_i = 180^\circ - \operatorname{atan2}(\mathbf{u} \times \mathbf{v}, \mathbf{u} \cdot \mathbf{v}), \quad \sum \theta_i = (n-2)180^\circ$$

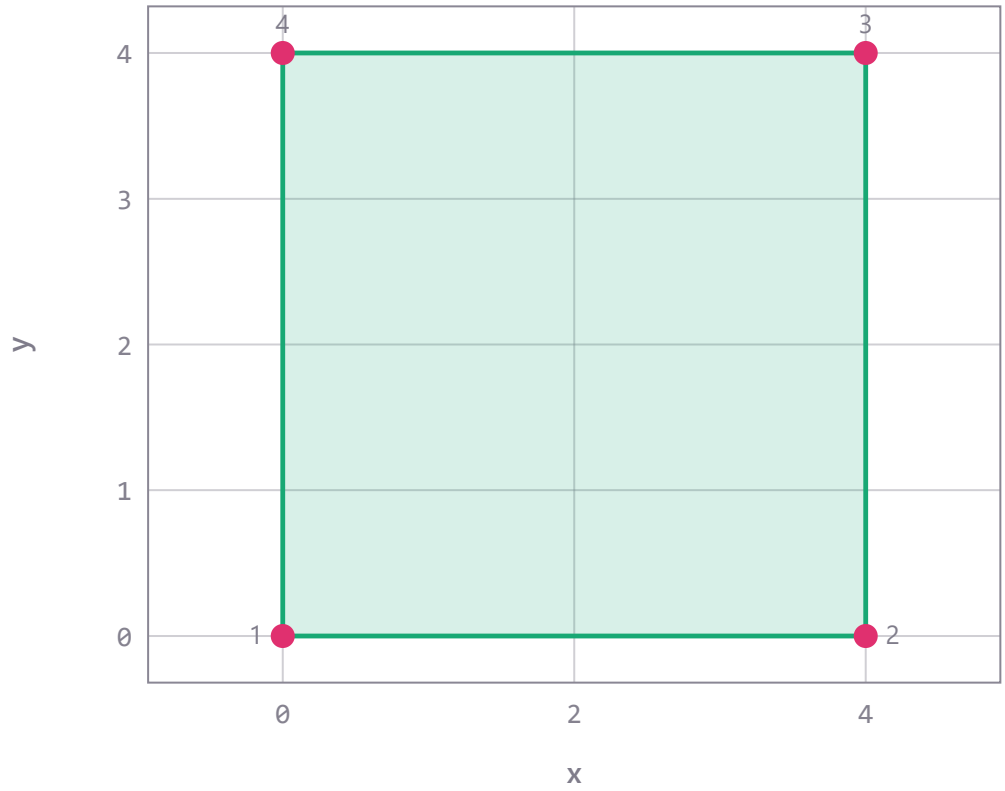
input	meaning	condition
`points`	vertices in order: x1 y1, x2 y2, ...	

output	meaning
`angles`	interior angle at each vertex, degrees, in the order given
`angle_sum`	$(n-2)$ 180
`reflex_vertices`	number of angles above 180 degrees

- **Method:** turning angle at each vertex by atan2 of the cross and dot products of consecutive edges, interior = 180 - turn (orientation-corrected)
- **Assumes:** simple polygon; either orientation
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



interior angle at each vertex, degrees, in the order given (angles) returned by geometry-ai999x for the worked example, one value for each position in the list (4 values, first 90, last 90, largest 90). Inputs: points = 0 0, 4 0, 4 4, 0 4.



The rows of points (vertices in order: x1 y1, x2 y2, ...) drawn as points (numbered in order), joined in order as a polygon. Returned: angle_sum = 360, reflex_vertices = 0.

```
geometry_ai999x.solve('polygon_interior_angles', points='0 0, 4 0, 4 4, 0 4')
# -> angles = [90.0, 90.0, 90.0, 90.0], angle_sum = 360, reflex_vertices = 0   (SOLVED)
```

```
geometry_ai999x.solve('polygon_interior_angles', points='0 0, 2 0, 2 2, 1 1, 0 2')
# -> angles = [90.0, 90.0, 45.0, 270.0, 45.0], angle_sum = 540, reflex_vertices = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the interior angles of the polygon with vertices (0, 0), (2, 0), (2, 2), (1, 1), (0, 2).	SOLVED	angles = [90.0, 90.0, 45.0, 270.0, 45.0], angle_sum = 540, reflex_vertices = 1